

对话记录

正文按可见用户/助手消息顺序收录；“用户 / 助手”与消息编号为整理标识，不属于原消息正文。

#001 用户

Original custom instructions no longer available

#002 用户

我想 Fork Tutti Open Source 来改造出 Self Hosted 的 Tutti VM。参考 [tutti-os/tutti: Where people and agents build in tune.](https://github.com/tutti-os/tutti) 开源仓库与目前 Tutti VM https://tutti.sh/ 的官方宣传，我们来头脑风暴一下实现方式与技术栈等。

#003 助手

我会先把官方开源仓库和 Tutti VM 当前宣传里的“能力边界”对齐，再按你的规则一次只问一个问题，逐步收敛到架构、运行时、隔离、存储、网络和部署方式 $\mathcal{A}(\underline{\cdot} \cdot \cdot \cdot \underline{\cdot})$ ♡

#004 助手

先抓到一个很关键的信号：这件事大概率不是“把 Tutti 塞进 Docker”这么简单喵 (๑ • ๒ •) ♡ 开源版当前是 Electron 桌面壳 + 本地 Go 常驻 daemon (`tuttid`) ；而仓库里已经有不少明确写着同时给 Tutti / TSH 使用的共享边界，比如 `device-link`、`workbench`、`workspace`、Device Authority 客户端等。

官方 Tutti VM 的目标又明确包含“多人/多设备共享同一个持久工作现场”，同时 Agent 仍主要运行在各自本机，房间只同步工作状态与产物。也就是说，我们如果追求接近官方 Tutti VM，真正要补的是一套 self-hosted 的 Room / Control Plane / Realtime / Relay / Storage 后端，而不是单纯远程桌面化。

#005 助手

第一问，也是会决定后面几乎所有技术选型的问题——你的 Self Hosted Tutti VM 希望采用哪种「Agent 运行模型」？ (。 • ̎-) ♡

A. 「复刻官方路线」：Claude Code / Codex / OpenCode 等 Agent 仍运行在每个用户自己的电脑上；自托管服务器只提供 Room、共享状态、文件/产物、Realtime、Relay、权限和 localhost Preview。最接近官方 Tutti VM，但工程难度最高。

B. 「真正的云端 VM」：Agent、代码、Terminal、localhost 服务全部运行在你自己的服务器 VM / Container 里；用户电脑只是 Tutti 客户端。实现会简单很多，也更像 Codespaces / Coder / Gitpod。

C. 「Hybrid 混合模式」：Room 和持久 Workspace 在服务器；既可以启动服务器上的 Agent，也允许用户把本机 Claude Code / Codex 之类“接入”Room，并继续使用自己的订阅。工程复杂度介于两者之间，但可能最强大 $\mathcal{A}(\geq \nabla \leq) \mathcal{A}$

你更想做 **A、B 还是 C**？也可以说一个你自己的变体～

#006 用户

A

#007 助手

好哒，那我们锁定 **A：尽量复刻 Tutti 官方的分布式路线** ♡ ๑ (´ ω `) ♡

这个选择意味着核心原则是： **Agent 算力、账号凭据、MCP、Skills 等留在用户设备；Self Hosted Server 负责 Room、

协作状态、协调与共享服务。官方目前也明确写着 Agent 实际运行在用户自己的机器、而且是在一个 managed local VM 中；Room 负责共享 working state，并支持跨设备恢复 sessions、tasks、files、running previews 等。(tutti.sh)

这也和开源仓库的结构非常契合：`tuttid` 已经是本地 business core，而 `device-link` 明确包含 ICE、QUIC、Relay stream、WebSocket/yamux 等设备间连接基础设施，所以 Fork 时我们应该尽量沿着这些已有 seam 往外补 Server，而不是推翻重写客户端。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/architecture/project-structure.md?utm_source=chatgpt.com))

那么第二问来啦 (◡‿‿◡)♡

你的 Self Hosted Tutti VM，第一版是否也要实现官方所说的「Managed Local VM」？

也就是每台用户电脑上：

A. 严格复刻

`tuttid` → Local VM → Agent + Workspace`

Agent 不直接操作 macOS / Windows 宿主机目录，而是在 Tutti 管理的 Linux VM 里工作。这样隔离、跨平台一致性、文件同步、Agent 借用都会漂亮很多，但工程量明显变大。

B. 先复用 Open Source 的本地运行方式

`tuttid` → Host OS → Agent + Local Workspace`

先专注把 **Room / 多人 / 实时协作 / 跨设备 / Agent borrowing** 做出来，等整个 distributed architecture 跑通以后，再加 Managed Local VM。

C. 从一开始设计双 Runtime

给 `tuttid` 抽象一个类似：

`WorkspaceRuntime`
→ `HostRuntime`
→ `ManagedVMRuntime`

MVP 先实现 Host，架构上保证以后能无痛塞入 Managed Local VM。

我目前会比较倾向 **C**，但先不把它写进最终方案——你希望选 **A**、**B** 还是 **C** 呢？(≥▽≤)♡

#008 用户

使用 Docker Desktop

#009 助手

那我们吧这条设计正式锁定啦：**Managed Local VM 不自己造虚拟机层，而是直接建立在 Docker Desktop 上** (≥▽≤)/ ✨

这样整体可以理解成：

`Tutti Desktop / tuttid`
→ `Docker Desktop API`
→ `Docker Desktop Linux VM`
→ `Tutti Workspace Container`
→ `Claude Code / Codex / OpenCode / Terminal / localhost services`

这和当前 Tutti Open Source 的结构也比较好衔接：Electron 继续只做桌面壳，业务和 Runtime 编排继续放 `tuttid`；只是我们给 `tuttid` 新增一层 `ContainerRuntime`，而不是让 Electron 直接调用 Docker。当前仓库本身就明确要求 Electron 不成为第二个 business core。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/architecture/project-structure.md?utm_source=chatgpt.com))

而且我建议我们**不要直接把现有 Tutti 的 managed Node/Python runtime 删除**。现在 `tuttid` 已经有 daemon-managed runtime、App bootstrap、App data/runtime/log/database directories 等契约。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/conventions/workspace-app-runtime.md?utm_source=chatgpt.com)) 后面完全可以变成：

```
`WorkspaceRuntime`  
→ `HostRuntime` (保留兼容)  
→ `DockerDesktopRuntime` (我们的默认)  
→ 以后甚至可以有 `RemoteDockerRuntime`
```

这样 Fork 对上游更新也友好很多 (｡•̀-)-✧

那么接下来第三问，会决定 Docker Runtime 的核心模型：

****一个 Tutti Room 在一台用户电脑上，应该对应怎样的 Docker 隔离粒度？****

A. ****一个 Room = 一个长期运行 Container****

```
```text id="jbf7cn"  
Room A
├── tutti-workspace-a
│ ├── workspace/
│ ├── Claude Code
│ ├── Codex
│ ├── OpenCode
│ ├── terminals
│ └── localhost apps
...`
```

所有 Agent 共用一个 Linux 环境。简单，也最像“大家围着同一台工作电脑”。

B. **\*\*一个 Agent = 一个 Container，共享 Workspace Volume\*\***

```
```text id="bf0e6o"  
Room A  
├── claude-container ─┐  
├── codex-container ─┴── tutti-workspace-volume  
└── opencode-container┘  
...`
```

隔离更强，但文件锁、Git、端口、环境变量、进程和 Agent 之间交接都会复杂很多。

C. ****一个 Room = 一个 Docker Compose Project****

```
```text id="6n3tza"  
tutti-room-abc/
├── workspace ← 长期存在，Agent 都 exec 到这里
```

```

├── preview-router
├── optional services
├── volumes
...

```

Agent 本身通常通过 `docker exec` 进入 `workspace` container；以后数据库、Browser、Preview Proxy、MCP 服务等需要隔离时，再自然增加 sidecar。

**\*\*我目前最倾向 C：一个 Room = 一个 Compose Project，核心只有一个长期 `workspace` container。它很适合后面实现 Tutti VM 那种 localhost Preview、持久环境和 Room 生命周期，同时不会一开始就把系统拆得过碎 (〰️> 〰️<〰️)〰️**

你希望选 **\*\*A、B 还是 C\*\***？

#### #010 用户

HostRuntime 只保留代码，但是前端不可达。因为如果要让他人「借用」电脑上的 Agent，那么会出现权限安全问题，可以肆无忌惮的获取共享者电脑上的所有敏感文件。

一个 Room = 一个 Compose Project，名为 open-tutti-vm-roomid

#### #011 助手

收到，这两条我会作为硬约束写进方案里 (๑ • ๓ • )๑ ✧

当前架构基线变成：

```

```text id="fmb2m6"
Tutti Desktop
|
▼
tuttid
|
├── HostRuntime
|   └── 仅保留实现代码
|       前端不可选择 / 不可创建 / 不可借用
|
└── DockerDesktopRuntime ← 唯一公开 Runtime
    |
    ▼
    Docker Compose Project
    open-tutti-vm-<roomId>
    |
    ├── workspace
    ├── preview / sidecars...
    └── persistent volumes
...

```

这里我非常赞同把 `HostRuntime` 从产品能力层彻底隐藏掉：****“借用 Agent”意味着远端用户实际上能够驱动这个 Agent 执行任意 shell/tool 操作****。只要 Agent 跑在 Host 上，即使 Tutti UI 自己有限权，也很难阻止它去读取 `~/.ssh`、浏览器数据、Cloud Credentials、其他项目目录等敏感内容。放进 Docker Desktop 管理的隔离环境之后，安全边界就清楚得多了 (๑ • ๓ •)๑ ✧

而且我们应该顺势定一条非常重要的规则：

> `workspace` 容器默认不得挂载 Host Home、Docker Socket、SSH Agent Socket 或任何非 Room 专属路径。

Compose Project 名称就按你定的：

```
```text id="sj7xa1"
open-tutti-vm-<roomId>
```
```

实际实现时只需要对 `roomId` 做 Compose project name 的合法化/稳定映射即可，例如：

```
```text id="x9pcyk"
Room ID:
019d1234-abcd-...
```

```
Compose Project:
open-tutti-vm-019d1234abcd
```
```

下一问就直接碰这个安全模型最核心的地方啦 ฅ(̎̈> ̎̈< ̎̈)ฅ

****Room 的 Workspace 文件最终存在哪里？****

A. ****Docker Named Volume****

```
```text id="2rozaq"
open-tutti-vm-<roomId>-workspace
```
```

源码完全生活在 Docker Desktop VM 中，Host 文件系统默认不可见。安全隔离最好，也最接近真正的 “Tutti VM”。

B. ****Host Bind Mount****

```
```text id="vggi85"
~/OpenTutti/Rooms/<roomId>/
↓
/workspace
```
```

用户能直接用宿主机 IDE、Finder、git 工具访问文件，但安全边界弱一些。

C. ****Named Volume 为默认，但允许显式 Export / Import****
平时：

```
```text id="xdc7c5"
Docker Named Volume
|
▼
/workspace
```
```

只有用户主动执行：

```
```text id="vd3y8i"
Import Folder
```

Export Workspace  
Clone from Git  
Push to Git  
...

才和 Host 交换文件。

**\*\*我会优先选 C\*\***：这样“VM”才是真的安全边界，而不是表面上跑在容器里、实际上整个项目仍然 bind mount 到宿主主机 ( •̀ω•́ )✧

你希望选 **\*\*A、B 还是 C\*\***？

#### #012 用户

```
open-tutti-vm-<roomId>-workspace
```

#### #013 助手

好哒，这个名字正式锁定 (๑ •̀ㅂ•́) ✧

Compose Project:

```
`open-tutti-vm-<roomId>`
```

Workspace Named Volume:

```
`open-tutti-vm-<roomId>-workspace`
```

实现时建议在 Compose 里显式指定 `name:`，不要依赖 Docker Compose 自动生成名称，否则默认通常会变成类似 `open-tutti-vm-<roomId>-workspace`：

```
```yaml id="calw2o"
volumes:
  workspace:
    name: open-tutti-vm-${ROOM_ID}-workspace
```
```

并且这个 Volume 应该独立于 Container 生命周期：`compose down` 不删除，只有真正执行「Delete Room / Delete Local Room Data」时才删除它 (｡•̀-｡)✧

下一问非常关键啦——它会决定我们后面的 **\*\*文件同步协议和 Server 架构\*\***：

同一个 Room 被多台电脑加入后，`open-tutti-vm-<roomId>-workspace` 在每台电脑上都会有一份。那么这些 Workspace 之间应该怎么保持一致？

A. **\*\*某一台设备是 Host / Source of Truth\*\***

其他成员实际上远程调用 Host 上的 Agent 和 Workspace，不在自己的电脑复制完整 Workspace。

B. **\*\*服务器是 Source of Truth，本机 Volume 是 Replica\*\***

每台加入 Room 的电脑都有自己的 Docker Volume；文件变更实时同步到 Self Hosted Server，再同步到其他设备。类似：  
`Local Volume ↔ Sync Engine ↔ Server ↔ Sync Engine ↔ Local Volume`

C. **\*\*P2P Replica，服务器只协调\*\***

设备之间直接同步 Workspace；Server 负责 Room membership、信令、版本元数据和 Relay，文件主体尽量不经过服务器。

你希望 \*\*A、B 还是 C\*\* 呢？(≥▽≤)♡

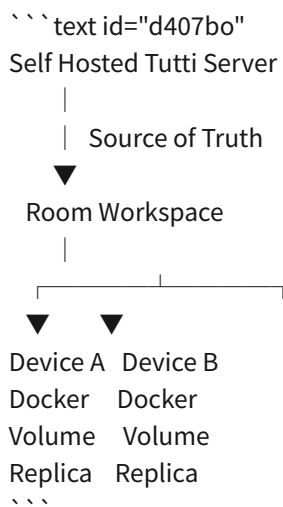
#### #014 用户

服务器是 Source of Truth，本机 Volume 是 Replica。不然在其他人的电脑上远程运行 Agent 会有争议，例如部分提供商只允许在用户本机运行，禁止登到别人的电脑上用

#### #015 助手

那就把这一条也锁死啦 (๑ • ๒ • )₉ ✧

现在的核心拓扑是：



每台设备都只在自己的 `open-tutti-vm-<roomId>-workspace` 里运行自己的 Agent；“借用 Agent”本质上是把任务/指令发给对方设备上的 Agent，由对方设备在自己的本地 Replica 上执行，而不是登录对方主机，也不是让别人的凭据跑到服务器上 (↔>□<)↔

这样安全边界会很清晰：

- Server 保存 Room 的权威 Workspace 状态
- 每台设备维护本地 Replica
- Agent 永远只访问本机 Docker Workspace
- Agent 凭据不上传 Server
- 不跨设备复制 `~/.config`、`~/.ssh`、token、浏览器状态
- 借用的是 “Agent execution capability”，不是 “对方电脑权限”

接下来最关键的问题就是：\*\*服务器端的 Source of Truth 要不要做成真正的“文件系统状态”，还是只做版本对象仓库？\*\*

A. 服务器维护一个真实 Workspace 文件树

例如每个 Room 在服务器上都有类似：

```
`/var/lib/open-tutti/rooms/<roomId>/workspace`
```

本地 Volume 和它做增量同步。

B. 服务器只保存 content-addressed objects + metadata

更像 Git/CAS:

`chunks/blobs + file tree + revisions`

客户端根据版本树还原本地 Workspace，不要求服务器上存在一个真实可挂载目录。

C. 两层同时存在

底层是 CAS / revision store，服务端额外 materialize 一份当前 Workspace，方便搜索、预览、diff、Web UI、备份等。

我会偏向 \*\*C\*\*，因为以后做多人并发、历史版本、冲突处理、Room 恢复和增量同步会舒服很多 (。・ヾ-)◇

你想选 \*\*A、B 还是 C\*\*？

#### #016 用户

B，服务器尽量只做调度

#### #017 助手

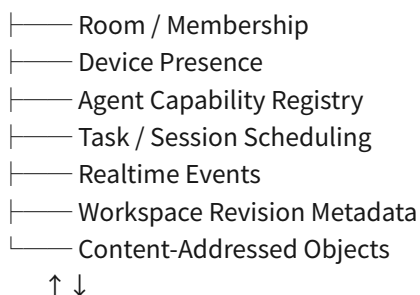
好哒，那服务器端正式锁定为 \*\*“调度 + 元数据 + CAS/Object Store”

，而不是远程文件系统\*\* (๑・ ٠・ ٠)ゞ◇

也就是说服务器不会长期 materialize 一个可执行的 Workspace，更不会在服务器上运行用户 Agent：

```text id="s9tg24"

Self Hosted Server



Device A Replica Device B Replica

Docker Volume Docker Volume

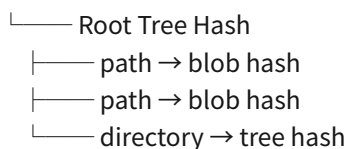
Agent A Agent B

```

Workspace 可以抽象成：

```text id="v0yn2y"

Revision



```

服务器只需要知道“Room 当前权威 revision 是什么”，真正的 Linux 文件系统只存在于各设备自己的：

`open-tutti-vm-<roomId>-workspace`



这会让 Server 很轻，也避免把它慢慢演化成一个隐藏的 Cloud IDE ( $\geq \nabla \leq$ )♡

下一问就决定 **CAS 的存储层** 啦：

你希望 Self Hosted Server 的对象数据默认放在哪里？

**A. 本机磁盘**

例如 `/var/lib/open-tutti/objects/sha256/...`，部署最简单。

**B. S3-compatible Object Storage**

例如 MinIO / SeaweedFS / Ceph RGW / AWS S3，Server 自己只维护 metadata + presigned upload/download。

**C. 做 Storage Provider 抽象**

默认 Local Filesystem，但接口设计成：

``ObjectStore` → LocalFS / S3Compatible``

小型 Self Hosted 一台机器直接跑，大规模部署以后切 MinIO/S3 不改上层逻辑。

我会比较倾向 **C** (〰 > □ <〰)🐼

你选 **A**、**B** 还是 **C**？

#### #018 用户

第一版先做单机部署

#### #019 助手

那第一版就收敛成 **单机 Self Hosted Server + 本地文件系统 CAS** 啦 (๑ • ๒ • )₯💎

也就是先不引入 S3 / MinIO 这类外部依赖：

```
```text id="itr94u"
open-tutti-server
├── metadata database
├── realtime / scheduler
├── room state
└── object store
    └── /var/lib/open-tutti/objects/
...```
```

Workspace 对象依然按内容寻址存储，例如：

```
```text id="6u65co"
/var/lib/open-tutti/objects/sha256/ab/cdef...
...```
```

以后如果要横向扩展，再把 ``ObjectStore`` 抽象出来迁移到 S3-compatible storage 即可；第一版没必要一开始就把部署复杂度拉高 (｡ • ̮ • )💎

下一问：

**\*\*第一版服务器的 Metadata Database 你希望用什么? \*\***

### A. **\*\*SO Lite\*\***

单二进制 + 单机部署非常舒服，Room、Device、Revision、Task、Presence metadata 都可以先放进去。

## B. **\*\*PostgreSQL\*\***

部署多一个服务，但以后多人并发、事务、查询和扩展会更稳，基本不用迁数据库架构。

### C. \*\*代码层抽象 Repository, 第一版 SQLite\*\*

业务层不直接依赖 SQLite，以后可以加 PostgreSQL Driver。

我会倾向 **\*\*C: 第一版 SQLite, 但从第一天就把数据库访问隔离在 Repository 层\*\*** ㄟ(ゝ▽ 〇 <)ㄟ

你想选 \*\*A、B 还是 C\*\*？

## #020 用户

C

## #021 助手

好哒，数据库这一层就确定为：

`Repository Interface → SQLite v1 → PostgreSQL future adapter`

业务层不能直接出现 SQLite-specific SQL/类型，这样第一版保持单机简单，以后横向扩展也不用重写核心逻辑 (๑ • ๓) ♡

目前服务器大致已经收敛成：

``text id="t30b47"

open-tutti-server

Room / Membership

Device Registry / Presence

## Agent Capability Registry

— Scheduler

Realtime

Workspace Revision / CAS Metadata

Repository

```
| └─ SQLiteRepository
```

└─ LocalObjectStore

```
└── /var/lib/open-tutti/objects/
```

下一问我们来决定一个会影响大量代码复用的东西——\*\*Self Hosted Server 主体用什么语言/技术栈？\*\*\*(。・ヾ-)◇

### A. \*\*Go 单体服务\*\*

尽量和现有 `tuttid` 统一语言，协议模型、Device Link、Workspace、认证等都有机会共享 Go package。第一版编译成一个 `open-tutti-server` 二进制。

## B. **\*\*TypeScript / Node.js\*\***

Web/API/Realtime 开发比较快，但会和现有 Go daemon 形成两套实现，共享协议主要靠 protobuf/OpenAPI。



protocol\*\*，而本地`renderer`→`tuttid`本身就是 HTTP + SSE/WebSocket；官方架构文档还特别要求 business events 走共享 event-stream contract，而不是各功能自己发明 socket payload。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/architecture/project-structure.md?utm\_source=chatgpt.com))

所以接下来第四问（？）我们来决定\*\*`tuttid` ↔ `open-tutti-server`的主协议\*\*：

**\*\*A. HTTP/OpenAPI + WebSocket\*\***

普通 CRUD、Room、Revision、CAS metadata 走 HTTP；Presence、Agent 状态、Task、Workspace revision notification 等走一个长期 WebSocket。最大程度贴合现有 Tutti 的协议风格。

**\*\*B. gRPC + bidirectional stream\*\***

两个 Go 服务通信非常舒服，但会单独形成一套远端协议体系，以后 Web Console / 第三方客户端还得加 gateway。

**\*\*C. Hybrid：HTTP/OpenAPI + WebSocket 为产品协议，内部允许 QUIC/device-link 用于特定数据路径\*\***

也就是 Control Plane 永远走 HTTPS/WS；以后大对象传输、设备间 Relay 等才复用现有`packages/device-link`的 QUIC / Relay 能力。上游的`device-link`本身就被定义为共享的 ICE/QUIC、Relay、WebSocket/yamux 基础设施，很适合保持在数据面而不是拿来承载整个业务 API。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/architecture/project-structure.md?utm\_source=chatgpt.com))

我目前最倾向 **\*\*C\*\*** (｡•̀ゝ)◇

你希望选 **\*\*A、B 还是 C\*\***？

## #025 用户

A

## #026 助手

锁定为 **\*\*A：HTTP/OpenAPI + 单条长期 WebSocket\*\*** 啦～(๑•̀ㅂ•́)و✧

我建议协议职责直接切成：

```text id="yv1lyc"

tuttid

```
|
|—— HTTPS / REST
|   Room CRUD
|   Membership
|   Device enrollment
|   Agent capabilities
|   Workspace revisions
|   CAS object upload/download
|   Task commands / history
|
|—— WebSocket
|   Presence
|   Agent state
|   Task state changes
|   Workspace revision committed
|   Borrowed-agent control events
|   Reconnect / resume
```

```

而且不要另起一套随意 JSON 格式，尽可能沿用上游现在的 **schema-first business event protocol** 思路。当前 Tutti 已经要求事件 topic 和 payload 由共享 schema 作为唯一事实来源，WebSocket 生命周期则留在具体 host/service 中。  
([github.com])(https://github.com/tutti-os/tutti/blob/main/docs/architecture/project-structure.md?utm\_source=chatgpt.com))

这样我们最终可以有类似：

```
```text id="43ut17"
packages/events/protocol
  ↑
  | shared schemas
  ↓
tuttid ←————→ open-tutti-server
```
```

HTTP API 则维护一份 OpenAPI schema，生成 Go client，让 `tuttid` 不手写 request/response DTO。这样 Fork 上游时也比较不容易越来越乱啦 (〰 > < ~) 〰

另外上游已经有 `device-authority-go`，其职责里包含 **Ed25519 enrollment / owner-token signing / credential binding validation**，所以我们的设备认证其实有一个很值得借鉴甚至复用的现成边界。([github.com])(https://github.com/tutti-os/tutti/blob/main/docs/architecture/project-structure.md?utm\_source=chatgpt.com))

那么下一问就问 **身份与设备认证模型**：

你希望第一版 Self Hosted Open Tutti VM 怎么登录？

**A. 内建账号系统**

Server 自己管理 User、密码、Session；新设备登录账号后生成独立 Ed25519 Device Identity。

**B. 只做管理员生成 Invite Token**

服务器没有传统注册/密码。管理员创建邀请链接/Token，Desktop 用 Token enroll，然后设备凭 Ed25519 key 长期认证。非常适合小团队 Self Hosted。

**C. OIDC / OAuth2**

例如接 Authentik、Keycloak、Pocket ID、Google 等；登录后再 enroll Device Identity。企业化最好，但第一版部署明显复杂一点。

我目前会比较倾向 **B**：特别符合我们现在追求的“单机、自托管、Server 尽量只调度”的方向，而且还能把 **User Identity** 和 **Device Identity** 清楚分开 (。・ ̎ )

你想选 **A**、**B** 还是 **C**？

## #027 用户

服务器可选配置服务器邀请码，输入邀请码的用户可以在服务器内创建房间，否则只能通过文本加入他人的房间类似于：

http(s)://<my\_server>/share/...  
房间密码：123456

## #028 助手

可以，这样身份模型会变得很适合小团队 Self Hosted，而且比传统「每个人都注册服务器账号」轻很多～(๑ • ๒ • )๑ ✧

我先把规则记成这样：

```
```text id="9j8sv9"
Server
├── 可选 SERVER_INVITE_CODE
│
├── 创建 Room
│   ├── 配置了服务器邀请码 → 必须先输入正确邀请码
│   └── 未配置 → 任意客户端都可创建 Room
│
└── 加入 Room
    └── 不需要服务器邀请码
        ↓
        https://example.com/share/<share-id>
        + Room Password
```
```

这里我建议把两种凭据严格区分：

```
```text id="ksgau0"
Server Invite Code
→ 只授予「创建房间」权限

Room Password
→ 只授予「加入指定房间」权限
```
```

并且 Share URL 中的 `<share-id>` 应该是足够长的随机 ID，而不是直接暴露 `roomId`：

```
```text id="9kneld"
/share/r_ZV4kq7Yx9W...
```
```

房间密码也不要明文存 SQLite，Server 只保存 Argon2id 哈希；URL 本身不携带密码，避免密码进入浏览器历史、反向代理日志和聊天软件链接预览里 (｡• ̯ •) ✧

加入成功后，我还建议客户端生成/使用自己的 \*\*Device Identity (Ed25519)\*\*。也就是说密码只是第一次进入 Room 的“门票”，后续 Server 实际识别的是设备：

```
```text id="i7ekwv"
Room Password
|
▼
首次 Join
|
▼
Device Public Key
|
▼
Room Membership
```
```

这样将来借用 Agent 时，我们才能明确显示：

```
```text id="ia4yxn"
安小娜的 MacBook
├── Claude Code ✓ available
├── Codex      ✓ available
└── OpenCode   ✗ unavailable
```
```

而不是只看到一堆匿名 WebSocket 连接 🌐(↗️ □ <~)🌐

下一问就围绕这个模型：

**\*\*通过 Share URL + 房间密码加入后，这个设备是否应该永久成为 Room Member? \*\***

A. **\*\*永久成员\*\***：首次验证密码后记录 Device Membership，以后无需再次输入密码，除非 Owner 主动踢出设备。

B. **\*\*每次验证\*\***：每次重新连接 Room 都要求密码，简单但体验比较烦。

C. **\*\*可撤销的长期成员\*\***：首次输入密码后获得长期 Membership Credential；Owner 可以查看设备列表、单独踢设备，也可以执行「Reset Room Access」使所有旧 Credential 失效。

我比较倾向 **\*\*C\*\*** (≥▽≤)♡ 你希望选哪个？

#### #029 用户

Room 是所有人退出就解散的，像会议一样

#### #030 助手

明白啦～那我们把 Room 定义成真正的 **\*\*临时协作会话\*\***，而不是长期项目空间 (≥▽≤)♡

这会让身份模型进一步简化：

```
```text id="og16e4"
Room 生命周期
创建
↓
至少 1 名在线参与者
↓
其他人通过 Share URL + Room Password 加入
↓
所有参与者陆续退出
↓
最后一个参与者退出
↓
Room 结束 / 解散
```
```

因此第一版可以不做“永久 Room Member”。Device Identity 仍然存在，但 Membership 只在当前 Room 生命周期内有效。

Server 只需要维护：

```
```text id="ilf5tn"
```

Device Identity 长期存在于客户端

Room Membership 临时

Room Session Token 临时

Agent Availability 临时

Presence 临时

```
```
```

房间被解散后：

```
```text id="mwrcr1"
```

Room Membership → 删除

Session Token → 失效

Share URL → 失效

Room Password Hash → 删除

WebSocket State → 删除

```
```
```

这也符合“借 Agent 像开会”的感觉：加入会议的人可以临时把自己机器里的 Agent 能力贡献给会议；会议结束，授权关系随之彻底消失 (｡•̀-)-✧

那接下来有一个非常关键的问题：

**\*\*最后一个人退出、Room 解散时，Workspace 数据要怎么办？\*\***

A. **\*\*彻底销毁\*\***

Server 上该 Room 的 revision/CAS 引用删除，各设备上的

`open-tutti-vm-<roomId>-workspace`

也删除。真正像一次临时会议。

B. **\*\*Room 解散，但 Workspace 保留在参与者本机\*\***

Server 删除 Room 和 CAS；每台电脑保留自己的最后 Replica，之后可以手动导出/继续使用。

C. **\*\*结束会议时由最后一个人选择\*\***

`Delete Workspace / Keep Local Copy / Export`

我觉得这个选择会直接决定我们后面的 CAS GC、Volume 生命周期和“会议结束”UX，所以这一题先定下来比较好喵 (˘> ㉿ <˘)ฅ

### #031 用户

每个人退出会议的时候都能选择导出还是放弃。房主退出则必须导出，否则不让他退出房间。

### #032 助手

好啦，这条生命周期规则锁定啦~(｡•̀-)-✧

当前退出规则变成：

```
```text id="49kdvv"
```

普通参与者退出

|—— Export Workspace → 导出当前 Replica 后退出
|—— Discard → 删除本机 Room Volume 后退出

房主退出

|—— Export Workspace → 允许退出
|—— Discard → 禁止
...`

也就是说房主承担一个很明确的责任：**至少保证会议最终成果有一份落到真实可持久保存的位置**。这样即使 Server 的 CAS 在 Room 解散后被 GC，也不会出现所有人一关会议、成果一起蒸发的情况 (。・ ̎-)◇

最后一个参与者退出后：

```text id="pdyrmp"  
Room → Destroy  
Share URL → Invalid  
Room Password → Delete  
Membership / Session → Delete  
CAS references → Release  
Unreferenced CAS objects → GC  
```

而各客户端自己的 `open-tutti-vm-<roomId>-workspace`，如果选择 Discard 就删除；选择 Export 后也可以删除这个临时 Volume，只保留导出的结果。

下一问就变得很关键啦：

如果房主先导出并退出，但 Room 里还有其他参与者，房主身份怎么办？

- A. 自动把房主转移给仍在线最久的参与者。
- B. 房主退出后 Room 进入“无房主模式”，其他人继续协作，但没人拥有房主特权。
- C. 房主退出前必须指定一个新的房主，否则不能退出。

你希望选 **A、B 还是 C**？ (≥▽≤)♡

#033 用户

房主退出前可以选择解散或者转让：指定一个新的房主，否则不能退出。

#034 助手

锁定啦～房主身份采用**显式转让制**，不会因为退出或掉线自动漂移给别人 (๑・ ̎・)9◇

规则现在是：

```text id="gkv60a"  
房主请求退出  
|—— 先完成 Workspace Export  
|  
|—— 解散房间  
| |—— Room 立即结束，所有参与者被移出  
|  
|—— 转让房主

- └—— 指定一名当前参与者
- └—— 对方成为新 Owner
- └—— 原房主随后退出

未导出

→ 不允许房主退出

未解散且未完成转让

→ 不允许房主退出

...

这里还要区分 \*\*“主动退出”\*\* 和 \*\*“意外断线”\*\*，否则房主电脑睡眠、Docker Desktop 重启、网络中断时，会绕过我们刚定下的规则 (。・\_・。)

所以下一问：

\*\*房主意外掉线 / Tutti 崩溃时，Room 怎么处理？\*\*

A. 给房主一个例如 \*\*5 分钟重连宽限期\*\*；期间 Room 继续运行，但不能解散或转让。超过时间后 Room 自动冻结。

B. 给房主重连宽限期；超时后自动把房主转让给仍在线最久的参与者。

C. 给房主重连宽限期；超时后其他参与者可以发起 \*\*“接管房主”\*\*，由在线参与者确认后完成接管。

我比较倾向 \*\*C\*\*，这样异常情况下不会把会议彻底卡死，又不会让 Server 悄悄决定新的房主 (˘> ౧ <˘)౧

你想选 \*\*A、B 还是 C\*\*？

#### #035 用户

给房主一个例如 \*\*5 分钟重连宽限期\*\*；期间 Room 继续运行，但不能解散或转让。超过时间后 Room 自动把房主转让给仍在线最久的参与者。

#### #036 助手

好哒，这条规则就确定为：

```text id="z77d2m"

房主意外断线

↓

进入 5 分钟 Grace Period

↓

期间 Room 继续运行

但禁止解散 / 房主转让

5 分钟内房主重连

→ 恢复正常

5 分钟后仍未重连

→ 自动转让给「当前仍在线时间最长」的参与者

...

这个机制很适合会议模型，因为 Server 只需要维护每个参与者的 `joined\_at / presence / last\_seen`，不需要复杂选举协

议 (｡• ㉿•) ♡

下一问只处理一个边界情况：

****如果房主断线 5 分钟后，Room 里已经没有任何其他在线参与者，该怎么办？****

- A. 直接解散 Room，并开始释放 CAS 引用 / GC。
- B. Room 再保留一段恢复期，例如 30 分钟，任何原参与者重新加入都可以恢复。
- C. Room 一直保留，直到房主重新上线或管理员手动清理。

我会偏向 ****A****，因为这最符合我们已经确定的“所有人都离开，会议就结束”的语义 (｡• ㉿•) ♡

#037 用户

直接解散 Room

#038 助手

好哒，这条也正式锁定～(｡• ㉿•) ♡

房主异常断线的完整规则现在是：

```
```text id="8v6es3"
Owner disconnect
 ↓
5 分钟 Grace Period
 |
 |—— Owner 重连
 | → 恢复 Owner 身份
 |
 |—— 5 分钟超时
 |
 |—— 有其他在线参与者
 | → Owner 自动转让给「当前连续在线时间最长」的人
 |
 |—— 无其他在线参与者
 | → Room 立即解散
 | → Share Link 失效
 | → Session / Membership 清除
 | → Workspace Revision 引用释放
 | → CAS 等待 GC
```
```

这里我建议“在线时间最长”定义成当前这一轮连续 Presence 的 `connected\_at`，而不是最早加入 Room 的 `joined\_at`。这样一个离开很久又刚重连的人，不会抢过一直留在会议里的参与者 (˘> ㉿ <˘)

下一问要进入整个 Open Tutti VM 最核心、也最容易踩坑的一层了：

****多台电脑同时修改同一个 Workspace 时，我们怎么处理并发写入？****

例如：

```
```text id="2xqmuo"
```

Server Revision: R100

Alice Replica	Bob Replica
修改 src/app.ts	修改 src/app.ts
提交 R101	提交 R102
```	

我看到三个方向：

\*\*A. 乐观并发 + 文件级冲突\*\*

每个客户端基于某个 `base\_revision` 提交变更：

```
```text id="ag8ldo"
commit {
 base_revision: R100
 changes: [...]
}
```
```

如果 Bob 提交时 Server 已经是 `R101`：

```
```text id="jua0e3"
不同文件
→ 自动 merge
→ R102
```

相同文件，但内容可以三方合并  
→ 自动 merge

无法三方合并  
→ 标记 Conflict  
```

比较像 Git，但同步粒度更实时。

\*\*B. 文件 Lease / Lock\*\*

某个 Agent 开始改 `src/app.ts` 时：

```
```text id="ubpp08"
src/app.ts
owner = Alice/device-1/agent-claude
lease = 30s
```
```

其他 Agent 暂时不能写这个文件。

实现安全，但 Agent 经常一次改几十个文件，锁管理很容易变得烦躁。

\*\*C. Workspace 单写者\*\*

整个 Room 同一时间只有一个 Agent / Device 获得写权限：

```
```text id="1nonq2"
Active Writer = Alice Claude
其他人 = Read Only
```
```

Agent 完成一轮任务后再释放。

最容易保证一致性，但会严重削弱 Tutti 那种“多 Agent 并行干活”的价值 (つ °Д °;) つ

我目前更倾向 \*\*A: Revision-based optimistic concurrency + 三方合并 + 显式 Conflict\*\*。

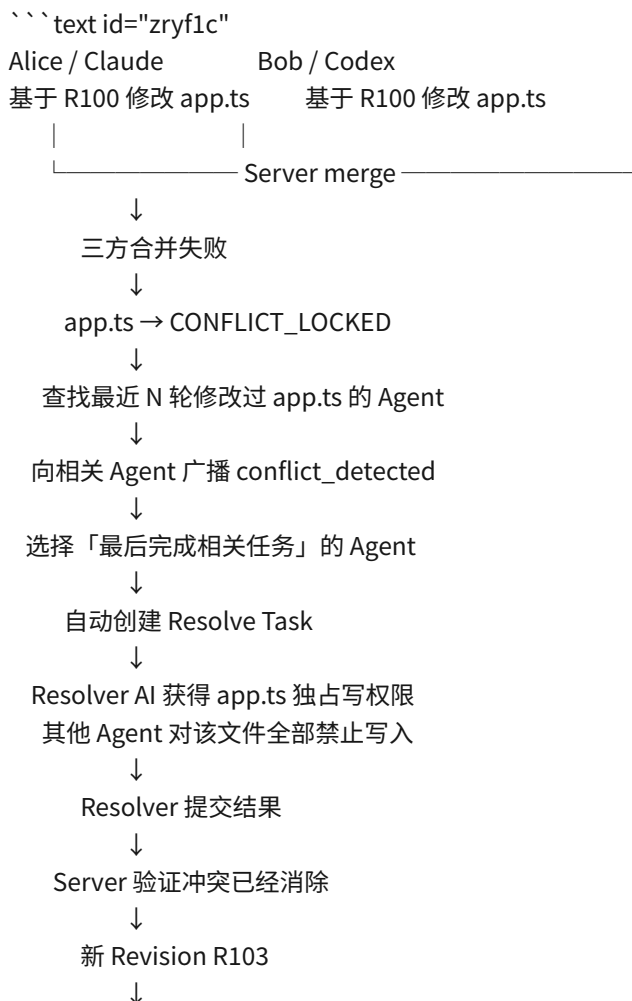
你希望选 \*\*A、B 还是 C\*\*？

#039 用户

乐观并发 + 文件级冲突，无法三方合并的话会标记 Conflict，通知最近几轮内对该文件进行修改的 AI，由于 user1 与 user2 同时修改该文件，该文件的合并出现冲突。系统将自动指派最后完成该任务的 AI 解决这部分冲突，期间该文件将被锁定无法更改。完成后再通知这些被告知文件已锁定的 AI，告诉他们可以修改了。

#040 助手

这一条可以定成一个明确的 \*\*Conflict Resolution 状态机\*\* 了 (๑ • ๒ •) 9 ✧



```

app.ts → UNLOCKED
↓
向之前被阻塞的 Agent 广播：
conflict_resolved / file_unlocked
↓
各 Agent 重新同步后继续
...

```

这里我会把“锁定”设计成 \*\*Server 强制的写入 fencing\*\*，而不能只是 UI 上告诉 Agent “别改”。也就是每次 Workspace commit 都必须带：

```

```json id="bn2ivl"
{
 "base_revision": "R102",
 "writer": {
 "device_id": "...",
 "agent_id": "..."
 }
}
```
...

```

Server 发现 `app.ts` 处于 `CONFLICT\_LOCKED` 时，除指定 `resolver\_agent\_id` 外，任何包含该文件的 commit 都直接拒绝。这样就算某个 Agent 没读通知、CLI 卡住或者继续运行，也突破不了锁 (・ω・)✧

同时锁应该只覆盖冲突文件，不冻结整个 Workspace：

```

```text id="smqm7u"
src/app.ts 🗝️ conflict
src/api.ts ✓ 可继续修改
README.md ✓ 可继续修改
tests/app.test.ts ✓ 可继续修改
```
...

```

被通知过锁定的 AI 在解锁后收到的事件，我建议包含最终 revision，而不只是简单说“可以继续了”：

```

```text id="j6ayxp"
workspace.file_unlocked
├── path
├── resolved_revision
├── resolver_agent
└── previous_conflict_revision
```
...

```

这样 Agent 可以先把 Replica 更新到 `resolved\_revision`，再继续之前被阻塞的任务，避免拿旧内容再次制造冲突 🐼(> ◡ <)🐼

下一问只处理这个机制最重要的故障情况：

\*\*被自动指派解决冲突的 AI，如果已经下线、任务执行失败，或者长时间没有解决冲突，怎么办？\*\*

- 按“最近完成相关任务”的顺序自动换下一个 AI。
- 自动重试一次，然后交给房东选择一个在线 AI。
- 设置例如 **\*\*2 分钟 Resolve Lease\*\***；超时就自动指派给下一个最近修改该文件、且当前在线可用的 AI，直到候选耗尽；

候选耗尽才通知房主人工指定。

我比较倾向 \*\*C\*\* ~这样不会因为一个卡死 Agent 永久锁住文件 (｡• ̯ •)✧

#041 用户

返回错误或者超时响应（指的是接取任务超时）则按“最近完成相关任务”的顺序自动换下一个 AI，知道问题被解决

#042 助手

那冲突 Resolver 的故障转移规则就锁定成这个啦~(｡• ̯ •)✧

```
```text id="abb58s"
检测到文件冲突
↓
生成 Conflict Resolution Task
↓
候选 AI 按「最近完成相关任务」排序
↓
Candidate #1
├—— 接取任务 → 成为 Resolver，文件继续保持锁定
├—— 明确返回 Error → Candidate #2
└—— 在接取期限内未响应 → Candidate #2
 ↓
 Candidate #3
 ↓
 ...
 ↓
 直到冲突被解决
...```
```

这里有个重要区别我也记下来：

```
```text id="y55v37"
接取任务超时 ≠ 解决任务执行超时
...```
```

也就是说，超时只限制 AI 是否及时接受这次 conflict-resolution assignment。AI 一旦正式接取，我们目前不会因为“解决得比较久”就自动抢走任务；否则两个 Resolver 同时工作反而容易产生新的竞争 (｡• ̯ •)✧

如果 Resolver 在解决过程中明确返回失败/Error，则锁继续存在，并自动交给候选队列里的下一位 AI：

```
```text id="mo9i84"
AI A 接取
→ resolve
→ ERROR
→ AI A 失去 Resolver 权限
→ AI B 接取
→ ...
→ SUCCESS
→ commit resolved revision
→ unlock
→ 通知所有此前被阻塞的 AI```
```

候选排序我暂时记成：

`最近成功完成过涉及该文件任务的 AI → 更早的相关 AI → .....`

下一问正好把这里的 ** “最近几轮” ** 定义清楚：

**冲突发生时，哪些 AI 应该进入 Resolver 候选队列？**

例如是：

A. 最近 **3 个曾修改该文件的 AI**

B. 最近 **5 个曾修改该文件的 AI**

C. 不设固定数量，**当前 Room 生命周期内所有曾成功修改过该文件、且目前在线可用的 AI** 都进入队列，按最近完成时间排序

你希望是哪一种呀？(≥▽≤)♡

#### #043 用户

C. 如果没有的话，改为随机选择目前所有在线可用的 AI，还是没有的话，弹窗暂停协作，告诉大家这个问题解决不了，且所有人的 AI 都没有办法重试。只能重试，或者将更改回退。

我感觉我处理的不好，那么 Tutti VM 是怎么处理的？

#### #044 助手

这个问题值得先对照一下官方实现再决定，因为“文件级自动合并 + 指派 AI 解冲突”如果不是 Tutti 本身的模型，我们没必要把第一版做得过重。我先查官方公开资料里有没有明确讲 Workspace 同步、并发写入和冲突处理 (。・_・)✧

#### #045 助手

查到了一个很关键的官方表述：**Tutti VM 公开描述的思路和我们刚才设计的“冲突发生后再找 AI 修”确实不太一样**。它更偏向“在写入发生的过程中就协调”，尽量不让传统 merge conflict 真正形成 (∩・_・)∩

#### #046 助手

你这个处理并不是“错”，而是更像 **Git/分布式版本控制的后置冲突修复**。如果目标是尽量复刻 Tutti VM，确实应该把这部分重新设计一下 (。・_・)✧

我查到 Tutti 官方现在对这件事的描述相当明确：Tutti VM 强调的是“基于底层文件系统的实时协作引擎”，不同设备上的 Agent 能实时看到彼此的文件变化，并且“自动避免和解决协作冲突”，而不是等各自完成一轮修改之后再 merge。官方甚至明确把它类比成 Google Docs / Figma。([tutti.sh](https://tutti.sh/zh))

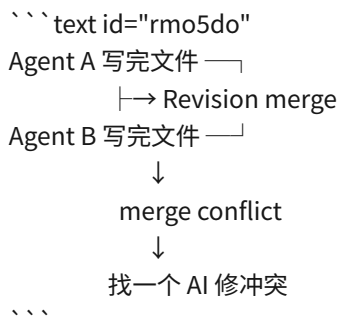
官方给出的具体行为描述是：一个 Agent 改了哪一行，其他 Agent 会毫秒级感知；如果两个 Agent 同时尝试修改同一个区域，系统会自动协调；双方看到的文件和 preview 会持续实时变化。官方还特意说，这种模型不是“各自做完再提交”，而是让多个 Agent 同时基于同一个工作状态工作，从而在过程中避免冲突。([tutti.sh](https://tutti.sh/zh/resources/usecases/multi-agent-collaboration))

不过这里有一个很重要的边界：**Tutti 并没有公开具体算法**。官方只说用了“复杂的协同编辑技术”，类似 Google Docs，并没有公开确认到底是 OT、CRDT、某种自研 operation log，还是几种机制混合。因此我们不能说“Tutti 就是用 CRDT”或者“Tutti 就是这样选 Resolver AI”。公开仓库目前也主要展示 Tutti Local 的实现和共享协议边界，没有公开 VM 文件系统协作引擎的具体实现。([tutti.sh](https://tutti.sh/en/resources/blogs/tutti-vm-technical-path?))

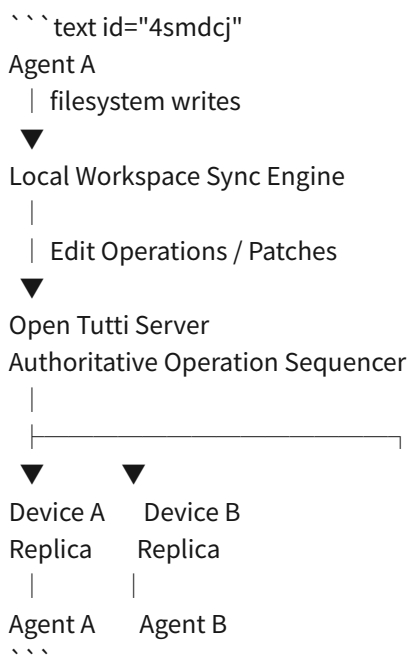


utm_source=chatgpt.com))

所以我觉得我们刚刚那个模型应该从：

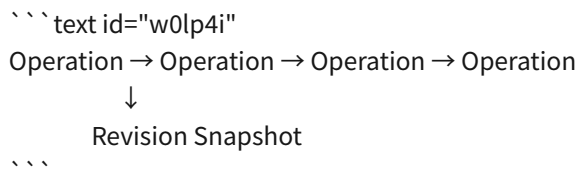


改成更接近 Tutti 的：



也就是说我们之前说的 `Revision` 仍然保留，但它不再是**协作的最小单位**。

它应该变成：



Revision 更多用于恢复、CAS、导出、历史状态和重连，而日常多人协作走实时 operation stream。

例如 Alice 的 Claude：

```
```text id="c3m41p"
R100

app.ts

```

```
line 20:
const port = 3000
``,`
```

Claude 把它改成：

```
``,`text id="acp8kj"
const port = 8080
``,`
```

Local Sync Engine 不应该直接上传“新的整个 app.ts”，而应该计算：

```
``,`text id="rqsd2y"
FileOperation {
  path: "app.ts"
  base_seq: 18291
  delete: range(...)
  insert: "const port = 8080"
}
``,`
```

Server 给 operation 排序、transform/rebase，再立刻推给其他 Replica。

Bob 的 Codex 很快就看到：

```
``,`text id="5nxyga"
const port = 8080
``,`
```

所以它下一次读 `app.ts` 的时候已经是最新状态了。

这其实就是 Tutti 官方宣传里那句很关键的话：Agent 是在\*\*实时感知对方正在做什么\*\*，然后自己判断哪些任务并行、哪些等待，而不是两个 Agent 长时间在两个版本的文件上独立工作。([tutti.sh](https://tutti.sh/zh/resources/usecases/multi-agent-collaboration))

这样一来，我们刚才设计的：

```
``,`text id="xmsbrd"
最近修改过文件的 AI
→ Resolver Queue
→ 随机 AI
→ 弹窗暂停整个协作
``,`
```

就不应该成为正常路径。

它最多只能是非常罕见的 \*\*Last Resort Recovery\*\*。

正常路径应该是：

```
``,`text id="pt7b8u"
① 不重叠的 edit
→ 自动合并
```

② 重叠但可以 transform 的 edit

→ 自动 transform

→ 自动合并

③ 同一区域存在语义冲突

→ Conflict Barrier

→ 暂时阻止该区域/文件进一步写入

→ 尝试自动 reconciliation

④ reconciliation 确实无法确定

→ 才进入人工/Agent Recovery

```\n

而且我现在不建议“随机挑一个在线 AI”。

原因很简单：随机 AI 可能完全不知道这个文件刚刚为什么被修改 (つ °Д °;)つ

比如：

```text id="ip2j6d"\n

Claude

任务：把 auth 改成 OAuth

Codex

任务：修 login race condition

随机挑中的 Hermes

任务：生成 README

```\n

让 Hermes 突然解决 `auth.ts` 的语义冲突，虽然它技术上可能能读代码，但上下文质量明显最差。

更接近 Tutti 思路的设计应该是：**尽可能根本不进入这个阶段。**

而我们有一个很大的架构优势：你已经选择了

```text id="8y5vzp"\n

Server = Source of Truth

```\n

所以相比真正去实现完全去中心化 CRDT，我们其实可以做得简单不少 ฅ(~> ౧ <~)ฅ

我目前会考虑这样的第一版核心：

```text id="ky2ll"\n

open-tutti-server

Room

└── Workspace

└── authoritative sequence

|

| 18291

```

| 18292
| 18293
| ...
|
|—— file operation log
|—— CAS
|—— revision snapshots
...

```

客户端：

```

```text id="gryy6g"
tuttid
|—— DockerDesktopRuntime
| |—— open-tutti-vm-<roomId>
| |—— workspace container
| |
| |—— /workspace
| |
| |—— Workspace Sync Engine
...

```

而真正比较有意思的技术难点就是：

****Agent 并不是在 Monaco Editor 里一个字一个字输入。****

Claude Code、Codex 可能：

```

```bash id="u454f1"
cat > file.ts
...

```

可能调用 `apply\_patch`，可能整个覆盖文件，也可能 rename/temp-write。

所以我们不能直接照搬普通 Google Docs CRDT。

Open Tutti 真正需要的是类似：

```

```text id="k1ry8i"
Filesystem change
↓
old content + new content
↓
diff
↓
canonical edit operations
↓
server transform / sequence
↓
apply to every replica
...

```

甚至以后可以进一步在 Docker Workspace 上做一个 ****FUSE Workspace FS****，让文件操作真正经过 Tutti 的协作层：

```

```text id="talwbs"
Agent
↓
/workspace
↓
Open Tutti FUSE FS
↓
Local Workspace Engine
↓
Server
```

```

这才是真正意义上的“基于底层文件系统的协作引擎”，也和 Tutti 官方目前公开描述的技术方向非常接近。([tutti.sh](https://tutti.sh/zh))

所以我建议我们把前面的设计修正为：

> **AI Resolver 不再是主冲突机制。主机改制成 Server-authoritative 的实时 filesystem operation collaboration engine；Revision/CAS 负责持久化和恢复。只有协作引擎无法确定结果时，才进入 Recovery。 **

下一问就只决定这一件事啦 (๑ • ๕ • ) ♡：

**我们是否正式推翻前面的“Revision 三方合并为主要协作方式”，改成 Tutti 风格的「实时 File Operation Stream 为主、Revision/CAS 为快照与恢复层」？ **

#### #047 用户

是

#### #048 助手

好哒～那这一点正式推翻并重定为：

```

`实时 File Operation Stream = 主协作机制`
`Revision + CAS = 快照 / 重连 / 恢复 / 导出机制` ๑(´□`*)๑

```

下一问就决定客户端最核心的实现方式：

**第一版怎么捕获 Docker Workspace 里的文件变化？ **

A. **Sync Sidecar + fsnotify/inotify + diff**

```

```text id="kpdkak"
open-tutti-vm-<roomId>
├── workspace
├── sync
│   └── mount 同一个 workspace volume
...
```

```

`sync` 容器监听文件系统变化，读取 old/new content，计算 canonical edit operations，再通过 `tuttid` 或直接 HTTPS/WS 发给 Server。

远端 operation 到达后，同一个 `sync` sidecar 写回 Volume。

优点是 MVP 很现实，不需要搞自定义文件系统。

B. **从第一版就做 FUSE Workspace**

```
```text id="hmtrqc"
```

Agent

↓

/workspace

↓

open-tutti-fs

↓

operation engine

↓

server

```
```
```

每次 `open/write/rename/unlink` 都能被精确拦截，理论上更接近真正的协作文件系统，但 Docker Desktop + FUSE + 权限 + 性能会让第一版难度大很多。

C. **A 起步，但接口按未来 FUSE 设计**

第一版实现：

```
`FsWatcherBackend`
```

以后补：

```
`FuseBackend`
```

上层统一输出：

```
```text id="zi14tt"
```

```
FileOperation {
```

```
  path
```

```
  base_seq
```

```
  kind
```

```
  range
```

```
  content
```

```
  previous_hash
```

```
  operation_id
```

```
}
```

```
```
```

我会明显倾向 **C**：第一版用 sidecar + inotify 做起来，但绝不把上层协议绑定死在 watcher 实现上 (๑ • ॢ •)₹

你希望选 **A、B 还是 C**？

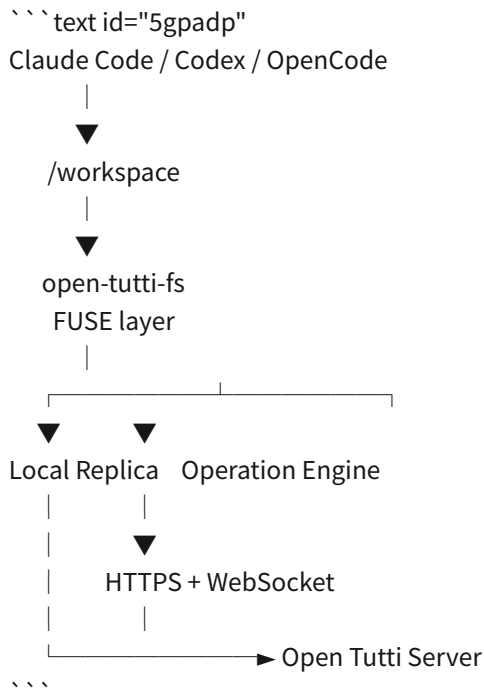
#049 用户

B

#050 助手

锁定为 **B**：第一版就做 FUSE Workspace^{~(๑ • ॢ • )}

这意味着 `/workspace` 不再是普通 Docker Volume 直接暴露给 Agent，而是 Open Tutti 自己控制的协作文件系统：



因此我们之前定的：

``open-tutti-vm-<roomId>-workspace``

仍然保留，但它更准确地应该成为 **FUSE 后端的本地 Replica/Cache Volume**，而不是 Agent 直接操作的 ``/workspace``。

这样 ``open-tutti-fs`` 可以直接捕获：

``create / open / write / truncate / rename / unlink / mkdir / chmod ...``

而不需要靠 inotify 在文件已经被改完以后再猜发生了什么。对于 Tutti 这种 Agent 会频繁 ``apply_patch``、整文件覆盖、`rename-temp-file` 的场景，这个方向确实更干净 (๑ • ̎)。

不过这里马上出现一个很重要的安全设计问题：**绝对不能为了 FUSE 给 Agent 本身 ``CAP_SYS_ADMIN`` 或直接开放 ``/dev/fuse``。**否则我们辛苦做的 Docker 安全边界会被明显削弱。

所以下一问：

第一版 ``open-tutti-fs`` 你希望放在哪里？

A. **和 Agent 在同一个 workspace container**

容器启动阶段由 root 挂载 FUSE，然后 Agent 进程降权到普通用户运行。实现最简单。

B. **独立的 ``open-tutti-fs`` sidecar**

只有这个 sidecar 拿 ``/dev/fuse`` 和必要 capability，Agent workspace container 完全无特权；但跨容器共享 FUSE mount 在 Docker Desktop 下会复杂不少。

C. **一个容器，但严格做双进程权限隔离**

`open-tutti-fs` daemon 以最小特权用户/能力运行，Agent 永远以独立 UID、无 capability 运行；Compose 层仍然只有一个 `workspace` service。

我目前更倾向 **C**：第一版能避开跨容器 FUSE mount propagation 的复杂坑，同时又不把 FUSE 权限直接交给 Agent (・ω・)✧

你选 **A、B 还是 C**？

#### #051 用户

C

#### #052 助手

锁定 **C**：单个 `workspace` 容器、双进程权限隔离**~FUSE daemon 负责挂载与同步，Agent 进程始终以无 capability 的普通 UID 运行；这样第一版能把隔离边界和实现复杂度控制在比较合理的位置 (๑・ㅍ・)✧

我顺便核一下 Docker Desktop 下 FUSE 的现实限制，再决定 `open-tutti-fs` 最适合用 Go 还是 Rust。

#### #053 助手

好哒，**C 正式锁定**~也就是说 `workspace` 容器内部存在两个完全不同的权限域：

```
```text id="imv86u"
workspace container
|
|—— open-tutti-fs
|   |—— 持有挂载 FUSE 所需的最小权限
|   |—— 管理 Replica / Operation Stream
|   |—— mount → /workspace
|
|—— agent-runtime
|   |—— UID: tutti
|   |—— no capabilities
|   |—— no /dev/fuse
|   |—— no docker.sock
|   |—— no host mounts
|   |—— Claude / Codex / OpenCode
|   ↓
|   /workspace
...`
```

也就是说即使“借用”某人的 Agent，远端指令最终能碰到的安全边界仍然只是这个 Room 的 `/workspace`，而不是 Docker Desktop、Host Home 或 FUSE 控制层 (๑・ㅍ・)✧

我刚核了一下目前适合我们的 FUSE 技术栈。Go 侧的 `hanwen/go-fuse/v2` 仍然活跃，提供 Node/Path API、底层 FUSE protocol、passthrough、IOCTL、ID-mapped mount 等能力，最近的 v2.10/v2.11 还增加了更底层的 protocol server 和 inflight 控制能力；对于我们这种“不只是做网盘，而是实现协作文件系统”的需求相当合适。([github.com](https://github.com/hanwen/go-fuse?utm\_source=chatgpt.com))

Rust 的 `fuser` 也很成熟，而且 2026 年 7 月刚发布 0.18.0，支持多 event loop、实验性 async API 和较完整的 Linux FUSE 能力。([docs.rs](https://docs.rs/crate/fuser/latest/source/CHANGELOG.md?utm\_source=chatgpt.com))

所以接下来只决定一个技术栈问题喵 (๑・ㇿ-)✧：

\*\*`open-tutti-fs` 本身用什么语言实现? \*\*

A. **\*\*Go + `hanwen/go-fuse/v2` \*\***

整个项目核心保持 Go:

`open-tutti-server`、`tuttid`、`open-tutti-fs` 都能共享 protocol、CAS、hash、operation model、认证代码。部署也只需要 Go 二进制。我目前最倾向这个。

B. **\*\*Rust + `fuser` \*\***

FUSE 层单独用 Rust，实现更偏系统编程，但会形成 Go ↔ Rust 两套代码和协议类型。

C. **\*\*Go Control Plane + Rust FS Engine\*\***

`open-tutti-fs` 的 FUSE hot path 用 Rust，网络同步/协议仍由 Go side process 负责。性能和隔离都漂亮，但第一版明显最复杂。

你希望选 **\*\*A、B 还是 C\*\***? 🐸(˃ ˂ ˆ)🐸

#054 用户

A

#055 助手

锁定 **\*\*A**: `open-tutti-fs` 使用 Go + `hanwen/go-fuse/v2` \*\*~(๑ • ๒ • ๓)✧

这样核心技术栈基本统一了:

```
```text id="ro19ia"
open-tutti-server Go
tuttid Go
open-tutti-fs Go + go-fuse/v2
Protocol / CAS Shared Go packages
Electron UI TypeScript / React
```
```

尤其重要的是，可以把真正复杂的协作逻辑抽到共享 Go package，而不是写死在 FUSE 里面:

```
```text id="ppufk5"
packages/
├── workspace-protocol
│ ├── FileOperation
│ ├── Sequence
│ ├── Snapshot
│ └── Conflict
├── workspace-cas
│ ├── Blob
│ ├── Tree
│ └── Revision
└── workspace-sync
 ├── Rebase
 └── Transform
```

└── Apply  
```

于是 `open-tutti-fs` 的职责尽可能简单：

```
``text id="ebemp9"
FUSE syscall
↓
转换成 canonical FileOperation
↓
workspace-sync
↓
Server authoritative sequence
↓
应用远端 operations
``
```

这样以后哪怕换掉 FUSE 实现，协作算法也不用一起重写 (。•̀-)-◇

下一问就是这个实时协作引擎最核心的算法选择：

\*\*我们要怎么表示和协调多人同时修改同一个文件？\*\*

A. **\*\*Server Sequenced OT / Patch Operations\*\***

Server 给每个 operation 一个全局递增序号：

```
``text id="fw7xqz"
seq 9001 Alice: insert(offset=100, "hello")
seq 9002 Bob: delete(offset=140, len=5)
seq 9003 Alice: replace(...)
``
```

客户端提交时携带自己看到的 `base\_seq`。如果落后，Server 把 operation 对中间发生的 operations 做 transform/rebase，再进入权威序列。

不需要完整 CRDT，因为我们的 Server 本来就是 Source of Truth。

B. **\*\*真正的 CRDT\*\***

每个文本文件建立 CRDT document，字符/块拥有稳定 ID，所有 Replica 最终一致。

理论上非常强，但 FUSE 面对的是普通 POSIX 文件：Agent 会 truncate、整文件覆盖、rename、生成临时文件。把这些行为映射到文本 CRDT 会复杂很多。

C. **\*\*Hybrid\*\***

```
``text id="hybozp"
UTF-8 text files
→ OT / operation stream
```

```
binary / large generated files
→ CAS blob replacement + optimistic version check
```

directory operations
→ Server sequenced filesystem operations
```

例如：

```text id="g4zofn"  
src/main.ts → 实时 OT
README.md → 实时 OT
package-lock.json → 可配置整文件 replacement
logo.png → CAS blob replacement
model.bin → CAS blob replacement
rename/mkdir → filesystem operation sequence
```

我目前最倾向 **C**，但其中“文本协作”仍然采用 **A** 的中央 Server Sequenced OT**，而不是引入真正的分布式 CRDT。因为我们已经明确 Server 是权威协调者，没必要承担 CRDT 最昂贵的复杂度 $\sim(\geq \nabla \leq)$

你希望选 **A、B 还是 C**？

#056 用户

C

#057 助手

锁定 **C：Hybrid 协作模型** $\sim(\otimes \cdot \boxplus)$ ◇

第一版可以把 Workspace 操作分成三类：

```text id="tgk0zr"  
UTF-8 文本文件
→ Server-sequenced OT / Patch Stream
→ 尽量实时协调

二进制 / 大文件 / 生成文件
→ CAS Blob Replacement
→ optimistic version check

目录与元数据操作
→ Server-sequenced FS Operations
→ mkdir / rename / unlink / chmod ...
```

这样 `open-tutti-fs` 不需要假装所有 POSIX 文件都是“协作文档”，同时又能让源码文件获得 Tutti VM 那种实时协作体验  $\mathfrak{A}(\rightsquigarrow \square \leftarrow \mathfrak{A})\mathfrak{A}$

下一问我觉得非常关键：

**哪些文件应该进入 Room 的权威 Workspace，同步到其他参与者；哪些文件应该明确保持设备本地？**

例如：

```text id="odceui"

应该同步？

src/

package.json

package-lock.json

.env.example

可能不应该同步？

.env

node\_modules/

.git/

.next/

dist/

\_\_pycache\_\_/

.venv/

target/

各种 Agent credential/config

```

我想到三种策略：

A. 完全遵循项目`.gitignore`，被 Git ignore 的文件默认不进入 Tutti Workspace。

B. Open Tutti 自己维护`.tuttiignore`，与`.gitignore`无关。

C. **双层规则**：内建安全/运行时排除规则 +`.tuttiignore`，并可选继承`.gitignore`。

比如：

```text id="mrnr10"

Built-in deny

→ credentials / sockets / device files / runtime secrets

.tuttiignore

→ Tutti 专属同步规则

.gitignore

→ 默认参与，但用户可关闭

```

我会倾向 **C**，因为`.gitignore`的语义是“不要提交 Git”，并不完全等于“不要协作同步”。例如某些项目可能希望共享`.env.local.example`或本地生成的设计产物，但绝对不能因为没写`.gitignore`就把敏感凭据同步出去 (。・_・)✧

你希望选 **A**、**B** 还是 **C**？

#### #058 用户

有些可以从 Tutti 的仓库看出端倪的，都不需要问我，尽可能模仿 Tutti

#### #059 助手

明白啦～从现在开始，**能从 Tutti 开源仓库、官方文档和 Tutti VM 宣传里推断出来的，我会直接按“尽量复刻 Tutti”的原则定下来，不再逐项问你**；只在公开资料无法判断、而且会明显改变架构的地方再问一个问题 🐼(“> ☐ <”)🐼

我先把 Workspace 同步排除规则、Room/Share、Agent 借用和本地运行边界再对一遍，然后继续往下收敛。

#### #060 助手

查完后，前面那个同步排除问题其实已经能直接定了。

Tutti 官方边界很清楚：**Room 里产生的工作状态属于共享内容；Agent 身份、Provider 凭据和宿主机其他内容属于本地私有内容。**官方明确说 Room 会共享文件、产物和运行状态，而 AI Provider 凭据仍以本地方式保存；技术文章也强调 MCP 配置、工具、Skills 和本机网络能力继续可用，但 Cloud 得到的是 Workspace 的执行权，而不是用户身份。([tutti.sh](https://tutti.sh/zh/privacy?utm_source=chatgpt.com))

所以 Open Tutti VM 不采用“`.gitignore` = 同步边界”。我会直接按 Tutti 的思路设计成两个世界：

```
```text id="zy9aw8"
Managed Local VM
|
|—— Shared Room State
|   |—— /workspace      ← FUSE
|   |—— source code
|   |—— configs created in Room
|   |—— documents / images
|   |—— app outputs
|   |—— project artifacts
|
|—— Device-Local Private State
|   |—— agent credentials
|   |—— agent provider state
|   |—— local MCP configuration
|   |—— local skills/tools
|   |—— sensitive runtime state
|   |—— live SQLite/WAL databases
|   |—— logs/cache/temp
|   |—— device identity
...
```
```

尤其是数据库这一点，上游自己已经有明确设计：live SQLite 数据库、WAL/SHM 和 indexes 放进独立`TUTTI\_APP\_DATABASE\_DIR`，目的就是避免同步/导出路径破坏文件锁和原子写语义。这个原则我们直接继承到 VM，不再询问。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/conventions/workspace-app-runtime.md?utm_source=chatgpt.com))

因此我会定义类似：

```
```text id="v9rfj1"
/workspace      Shared + FUSE

/.open-tutti/device/
|—— agents/      Local only
|—— credentials/ Local only
|—— mcp/         Local only
|—— skills/      Local only
```
```

```
|—— databases/ Local only
|—— runtime/ Local only
|—— cache/ Local only
...`
```

这些 private paths ****完全不进入 File Operation Stream，也不进入 CAS****。

`.gitignore` 仍然只是 Git 语义；我们也暂时不需要 `.tuttiignore` 作为安全措施。真正的安全边界应该由 mount/layout 强制实现，而不是指望一个 Agent 遵守 ignore 文件 (｡• ̳•｡)✧

还有一点也可以直接照 Tutti 定：第一版 ****不做 Workspace 内容 E2EE****。官方隐私说明明确写着 Tutti VM 会在云端处理和存储 Workspace 文件、产物和运行状态，并没有公开宣称端到端加密。我们的 Self Hosted Server 本来就是用户自己控制的，所以继续采用 Server-authoritative OT + CAS 会简单很多。([tutti.sh](https://tutti.sh/zh/privacy?utm_source=chatgpt.com))

接下来有一个公开 Tutti 资料无法替我们决定、而且对安全架构影响非常大的问题：

****Open Tutti 第一版要不要真的支持“借用别人的 Agent”？****

因为一旦支持，我们就必须解决一个比 Workspace 隔离更深的问题：

```
```text id="7u30se"
User B
  ↓ prompt
User A 的 Claude Code
  ↓
Claude Code 本地身份 / subscription / MCP / skills
...`
```

官方宣称这是 **\*\*“借使用能力，不借账号身份；Room-scoped、可随时撤销”\*\***。([tutti.sh](https://tutti.sh/en?utm\_source=chatgpt.com))

但要安全复刻它，我们不能只是把 User B 的 prompt 原样喂给 User A 的 CLI，否则 Agent 可能被诱导读取自身本地凭据或调用 User A 的私人 MCP。

所以我只问这一题：

\*\*第一版就实现 Agent Borrowing，还是先把多人实时 Workspace 做完整，Borrowing 放到第二阶段？\*\* (｡• ̳•｡)✧

#061 用户

第一版就实现

#062 助手

锁定：**\*\*第一版就支持 Agent Borrowing\*\***。我会把它当成核心能力，而不是后续插件；下一步我先对照 Tutti 官方对“借用 Agent、凭据留本机、Room-scoped 权限”的公开描述，把最小安全边界直接定下来，再只问你真正无法从官方推断的一点 (˘> ̫ <˘)ฅ

#063 助手

第一版就把 **\*\*Agent Borrowing 定义成一等公民能力\*\***，并直接照 Tutti 公开语义实现，不另造一套“远程 Agent”概念啦 ~ (｡• ̳•｡)✧

官方目前已经明确了几条，我们可以直接照搬：借用必须由 Agent 所有者在 Room 内主动开启；借用者可以像使用自己的 Agent 一样发指令、看实时执行过程；所有者随时可以撤销；借用产生的工作状态留在 Room；而且对方调好的 Skills、MCP、Tools 也会随 Agent 一起成为可用能力。官方强调这叫 capability delegation，不是账号转移。([tutti.sh](https://tutti.sh/en/resources/usecases/borrow-agents?utm\_source=chatgpt.com))

因此我们的数据模型可以直接收敛成：

```
```text id="5yn2gb"
Device
├── AgentInstance
│ ├── provider = claude-code / codex / ...
│ ├── owner_device_id
│ ├── status
│ ├── capabilities
│ │ ├── skills
│ │ ├── mcp
│ │ └── tools
│ ├── borrowing
│ │ ├── enabled
│ │ ├── room_id
│ │ └── lease_generation
...
```
```

借用流程则是：

```
```text id="k30uco"
User B
↓
选择 Alice / Claude Code
↓
open-tutti-server
↓ 创建 Room-scoped Borrow Task
Alice 的 tuttid
↓
本机 open-tutti-vm-<roomId>
↓
Alice 的 Claude Code
↓
/workspace
↓
结果 / Terminal / 文件变化 / Agent Stream
↓
Room 所有人实时看到
...
```
```

Server 永远不拿到 Claude/Codex 的登录凭据，也不代替 Agent 请求 Provider。Server 只知道“这个 Agent 当前愿意接受 Room 内的任务”。

而且我会把撤销设计成真正的 fencing，而不是 UI 开关：

```
```text id="n5f1qw"
borrowing_generation = 42
```
```

Owner 点击 Revoke

↓

generation = 43

↓

所有 generation=42 的新 command 立即失效

↓

当前借用 session 收到 revoked

↓

停止继续接受借用者的新输入

...

这与 Tutti “所有者可以随时收回” 的产品语义一致。([tutti.sh](https://tutti.sh/en/resources/usecases/borrow-agents?utm\_source=chatgpt.com))

不过安全上我们需要比宣传页多做一层。 \*\*Skills/MCP/Tools 可以借，但它们的配置和凭据不能借。 \*\*

也就是说：

```text id="zp056s"

Borrower 可以：

“用 Alice 的 GitHub MCP 查这个 issue”

Borrower 不可以：

cat Alice 的 MCP config

读取 OAuth token

读取 provider credential

查看 Host ~/.ssh

...

所以我会给每个 Agent Adapter 定一个 `BorrowSafe` 契约：

```text id="57k4mm"

AgentAdapter

|—— StartSession()

|—— SendTurn()

|—— Interrupt()

|—— StreamEvents()

|—— Capabilities()

|—— BorrowSafetyProfile()

...

只有满足以下条件的 Adapter 才允许打开 Borrowing：

```text id="v2pmtr"

filesystem

→ 只能看到 Room /workspace + 明确暴露的 capability

host filesystem

→ 不可见

provider credentials

→ 不作为普通文件暴露



Docker socket

→ 不可见

FUSE control daemon

→ 不可访问

privilege escalation

→ 禁止

network

→ 按 Agent / capability policy 控制

MCP secrets

→ 只能由 MCP runtime 使用

→ 不能作为 Room 文件同步

```\n

这不是多余设计。以 Codex 为例，OpenAI 当前明确把 filesystem sandbox、network policy 和 credentials 隔离视为 Agent 安全边界；其安全配置可以限制 writable roots、网络访问，并把 CLI/MCP OAuth 凭据放入安全凭据存储。
([openai.com](https://openai.com/index/running-codex-safely/?utm\_source=chatgpt.com))

所以我不会采用“把 `~/claude`、`~/codex` 直接 mount 到 workspace 容器里”这种偷懒方案 (ó\_\_ò。) 那样 Borrowing 基本等于把账号秘密交给不受信任的 Prompt。

这里终于有一个 \*\*Tutti 官方公开资料没告诉我们答案\*\* 的关键产品决策，需要你定：

\*\*如果某个 Agent Provider 第一版无法做到我们要求的 `BorrowSafe` 隔离，是不是允许它继续作为“本人可用 Agent”，但 UI 中禁止开启 Borrowing？\*\*

也就是宁可出现：

```\n

Claude Code

✓ 自己使用

✗ 不可借用：BorrowSafe isolation unavailable

```\n

也绝不为了“所有 Agent 都能借”而降低安全边界。你接受这个原则吗？(。・ヾ-)◇

#064 用户

可以挂载进去，通过权限控制来阻止访问

#065 助手

可以，但这里要把“挂载进容器”和“Agent 能读取”严格分开喵 (๑・ㅍ・)ゞ◇

我建议保留挂载，只是不要把私密配置放进 `~/workspace`，而是放进独立私有挂载点，并通过 UID/GID、mount namespace 和工具执行沙箱把它隔离开。

#066 助手

可以，这个方向反而更接近 Tutti 官方对 BYOS 的描述：Agent 所有者的订阅、MCP、Skills、Git/SSH 等本地能力都可以随 Agent 一起进入本地 VM，但身份本身仍留在本机。([tutti.sh](https://tutti.sh/en/resources/blogs/tutti-vm-technical-path?utm\_source=chatgpt.com))

不过我会把你说的“权限控制”定义得比单纯`chmod 600`更强一点，因为如果 Agent 主进程和它执行的 Shell 都是同一个 Linux UID，那么只要 Claude/Codex 自己能读凭据，Agent 执行的`cat`理论上也能读，传统 Unix 文件权限无法区分这两者 (。・△・。)

所以第一版我建议直接采用这种布局：

```
```text id="165mil"
open-tutti-vm-<roomId>
├── workspace container
│ │
│ ├── open-tutti-fs ← FUSE daemon
│ │
│ ├── open-tutti-agentd ← Agent supervisor
│ │
│ ├── /workspace ← Room 共享状态
│ │
│ └── /run/open-tutti/private/
│ ├── claude/
│ ├── codex/
│ ├── mcp/
│ ├── ssh/
│ └── git/
...
```
```

其中：

```
```text id="zt9b4h"
/workspace
→ Agent 可正常读写
→ 进入实时 File Operation Stream

/run/open-tutti/private/*
→ 挂载进容器
→ 永不进入 FUSE/CAS/Room
→ Room 其他成员不可读取
...
```
```

再做至少三层权限隔离：

```
```text id="db7god"
Linux DAC / UID/GID
+
Agent 自身 sandbox / permission profile
+
Open Tutti task execution policy
...
```
```

例如 Codex 当前的 permission profile 已经能指定命令允许读取/写入哪些 filesystem roots，并且官方也建议 Agent 主进程和它运行的命令采用最小权限策略。([github.com](https://github.com/openai/codex-action/blob/main/docs/security.md?utm\_source=chatgpt.com))

于是借用 Agent 时，我们可以给工具执行环境动态下发：

```

` `` text id="odeep6"
Borrowed Session
|—— read:
|   |—— /workspace/**
|   |—— 必要 runtime files
|
|—— write:
|   |—— /workspace/**
|
|—— deny read:
|   |—— /run/open-tutti/private/**
` ``

```

但是 Agent 本身仍能通过它原有的认证机制使用：

```
`` text id="ilis46"
Claude subscription
Codex subscription
Git identity
SSH capability
MCP
Skills
Tools
``
```

这正好对应 Tutti 宣传里的 \*\* “借的是 capability，不是 account” \*\*。官方甚至明确说借来的 Agent 会带上原主人的 MCP、Skills 和 Tools。([tutti.sh](https://tutti.sh/en/resources/usecases/borrow-agents?utm\_source=chatgpt.com))

因此我收回前面“不能直接挂载凭据”的限制，改成：

> \*\*允许所有者的 Agent 环境和凭据挂载进入本机 Room VM，但它们属于 Private Capability Plane，不属于 Shared Workspace Plane；借用者可以调用这些能力，但不能通过文件系统直接读取其底层秘密。\*\*

这个划分我觉得非常关键，可以直接成为 Open Tutti VM 的安全模型：

```
`text id="7uk9bh"
```

Shared Workspace Plane |

/workspace |

code / docs / outputs |

↕

Agent Runtime

↕

Private Capability Plane |

credentials / MCP / SSH |

skills / provider identity |

```
`
```

下一题是官方资料没有明确说明、但会直接影响 Borrowing 调度的：

\*\*同一个被借出的 Agent，是否允许两个人同时给它下任务？\*\*

也就是 Alice 共享一个 Claude Code 后，Bob 和 Carol 能否同时启动两个 Claude session；还是同一个 `AgentInstance` 同一时刻只能被一个借用者占用，其他人排队？(｡•̤̥-)✧

#067 用户

可以，在这个 compose 内多启动几个容器就行了，这样就隔离了

#068 助手

可以，这样更合理~我们把“一个 Room = 一个 Compose Project”保留，但把 **\*\*AgentInstance\*\*** 和 **\*\*AgentSession 容器\*\*** 分开：同一个被共享的 Agent 能同时派生多个隔离 Session，每个借用者/任务各自跑在独立容器里 (｡•̤̥-)✧

#069 助手

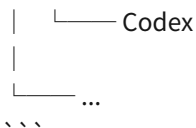
对，这样并发借用会更自然~而且我会把你刚才选的“单容器双进程隔离”理解为**\*\*每个 Agent Session 容器内部\*\***仍然这么做，而不是整个 Room 永远只能有一个容器 (｡•̤̥-)✧

#070 助手

对，这样更适合 Tutti 的“多人、多 Agent 并行”模型。容器不是 Agent 本身，而是一次隔离的 **\*\*Agent Session Runtime\*\***。官方也强调借来的 Agent 能继承 Room 上下文、像自己的 Agent 一样继续工作，所以我会让 Session 跟对话/上下文生命周期绑定，而不是“一任务一个容器、任务结束立即销毁”。([tutti.sh](https://tutti.sh/zh/resources/usecases/borrow-agents?utm\_source=chatgpt.com))

我建议把本机一个 Room 的 Compose Project 收敛成这样：

```
```text id="g718rw"
open-tutti-vm-<roomId>
|
|—— room-sync
| |—— 连接 open-tutti-server
| |—— authoritative operation client
| |—— 本机 Replica / CAS cache
| |—— mount:
| open-tutti-vm-<roomId>-workspace
|
|—— agent-session-01
| |—— open-tutti-fs
| | |—— /workspace (FUSE)
| |—— open-tutti-agentd
| |—— Claude Code
|
|—— agent-session-02
| |—— open-tutti-fs
| | |—— /workspace (FUSE)
| |—— open-tutti-agentd
| |—— Claude Code
|
|—— agent-session-03
| |—— open-tutti-fs
| |—— open-tutti-agentd
```



这里我会增加一个很重要的约束：**Agent Session 容器不能直接 mount `open-tutti-vm-<roomId>-workspace` Named Volume。**

否则 Agent 可以绕过 FUSE:

```

```text id="27ctue"
/workspace      ← FUSE, 受协作协议控制
/private-replica ← 如果也能访问这里
                Agent 就能直接改 backing files
                ✗ 绕过 OT
                ✗ 绕过 operation sequence
                ✗ 绕过 conflict control
...

```

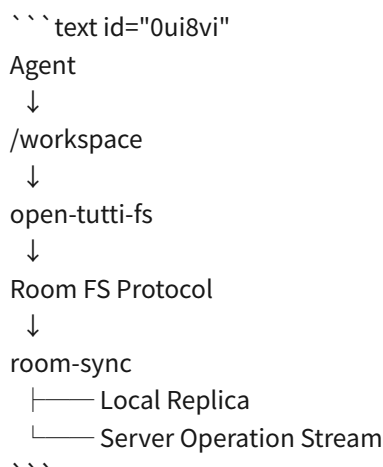
所以只有 `room-sync` 能碰:

```

```text id="cejecg"
open-tutti-vm-<roomId>-workspace
...

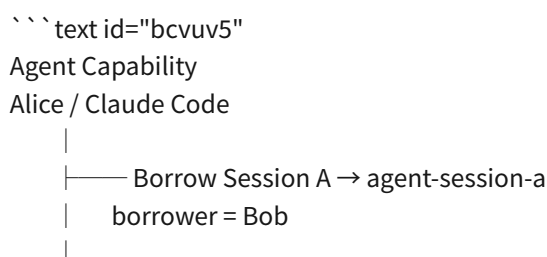
```

Agent Session 中的 `open-tutti-fs` 通过本机 Compose private network / Unix socket 风格协议连接 `room-sync`:



这样同一个设备启动 10 个 Agent 容器，它们看到的仍然是**同一个逻辑 Workspace**，而不是 10 份各自为政的 Replica  
 啊(↗ □ <↖)啊

借用 Alice 的 Claude 两次就变成:



└── Borrow Session B → agent-session-b

borrower = Carol

...

两者共享：

```text id="i67x6r"

✓ Claude subscription capability

✓ Alice 配好的 Skills

✓ Alice 配好的 MCP

✓ Room Workspace

...

但隔离：

```text id="vq46yl"

✓ PID namespace

✓ mount namespace

✓ /tmp

✓ Agent session state

✓ conversation runtime

✓ shell processes

✓ localhost ports

✓ task execution

...

其中 credentials / MCP secrets 等 Private Capability Plane 可以按你说的挂进去，但必须由 Agent runtime 权限体系控制；Room Workspace 和私密挂载完全分开。官方的公开模型本身也是“Agent 与凭据留在自己的机器，本地 VM 管理，只共享工作状态”。([tutti.sh](https://tutti.sh/zh?utm_source=chatgpt.com))

Container 也不用静态写死几十个 Compose service。我倾向预定义一个 `agent-session` 模板，由 `tuttid` 动态创建实例，并全部打上：

```text id="f02srr"

com.docker.compose.project=open-tutti-vm-<roomId>

open-tutti.room-id=<roomId>

open-tutti.session-id=<sessionId>

open-tutti.agent-id=<agentId>

open-tutti.owner-device-id=<deviceId>

open-tutti.borrower-id=<participantId>

...

Room 解散时，就能按 project label 一次性清理整个 Runtime。

这里还有一个必须由我们自己决定、官方公开资料没有给出答案的问题：

\*\*一个用户分享自己的 Claude/Codex 后，要不要允许他设置“最大同时借用数”？\*\*

例如 Alice 可以设置：

```text id="1qgmgd"

Claude Code

Borrowing: ON  
Max concurrent sessions: 3  
...

达到 3 个后，第 4 个借用请求进入排队，而不是继续无限创建容器。还是你希望第一版**不设人为并发上限，只受到 Docker Desktop 实际 CPU / RAM 和 Provider 自身限制**？ (｡•̯•)✧

#### #071 用户

不设人为并发上限先，默认能在一起开发的都是具有一定信任基础的

#### #072 助手

锁定：第一版 **不设置人为并发上限**，只受 Docker Desktop 实际资源、各 Provider 自身并发/额度限制，以及用户主动停止 Session 的影响。默认 Room 参与者之间已有一定信任基础，所以我们先避免做复杂的配额和排队系统 (｡•̯•)✧

我再对照一下 Tutti 对 Agent runtime / sandbox / localhost preview 的公开实现边界，下一题只问公开资料无法替我们决定的部分。

#### #073 助手

锁定：**第一版不设 Open Tutti 自己的 Agent 并发额度/排队规则**。一个人愿意共享 Claude/Codex，就可以在本机为同一个 Room 派生多个 Session 容器；资源不足、Provider 限流或订阅限制直接作为运行时状态反馈，不由 Server 人为限制 (｡•̯•)✧

另外我对照上游后，会顺手把两个设计直接定下来，不再问你：

第一，继续复用 Tutti 现有的 **provider-neutral Agent Host** 思路。上游已经把 Agent lifecycle application core 放在 `packages/agent/host`，Session、Turn、resume/send/cancel/recovery 都有明确的 Host Boundary，所以我们不应该重新为 VM 发明 Claude/Codex 专属调度核心。VM 层应该只是给既有 Agent Host 增加一个 `ContainerSessionRuntime`。([github.com](https://github.com/tutti-os/tutti/blob/main/AGENTS.md?utm_source=chatgpt.com))

第二，每个并发 Agent Session 都获得自己的 FUSE mount，但连接同一个逻辑 Room Workspace：

```
```text id="tte08n"
open-tutti-vm-<roomId>
|
|—— room-sync
|   └── local replica/cache
|
|—— claude-session-a
|   ├── open-tutti-fs → /workspace
|   └── Claude Code
|
|—— claude-session-b
|   ├── open-tutti-fs → /workspace
|   └── Claude Code
|
└── codex-session-c
    ├── open-tutti-fs → /workspace
    └── Codex
...
```
```

三个 `/workspace` 是三个独立 FUSE mount，但背后都连接 `room-sync → open-tutti-server`，所以逻辑上仍是“同一台电脑”。这也符合 Tutti VM 官方所描述的：不同人的 Agent 仍在本机运行，但处于同一个实时共享工作状态里。  
([github.com](https://github.com/tutti-os/tutti?utm_source=chatgpt.com))

接下来有一件官方只说明了“能做到”，没有公开说明具体实现，所以需要我们自己定：

****Room 里的 localhost Preview，第一版是否允许所有参与者直接访问任意 Agent Session 启动的 TCP 服务？****

例如 Alice 的 Codex 在自己的 Session 容器里启动：

```
```text id="gp805w"
npm run dev
→ 0.0.0.0:3000
```
```

Bob 在 Tutti Room 里就能直接打开这个 Preview。

这里我想确认的是安全语义：****默认 Room 内完全信任，任何监听端口都可以被发布给 Room；还是必须由 Tutti 检测到端口后，经启动该服务的人点击一次“Share Preview”才允许别人访问？**** (。・ヾ-)◇

#### #074 用户

默认 Room 内完全信任，任何监听端口都可以被发布给 Room

#### #075 助手

锁定：Room 内默认互信，****任何 Agent Session 中监听的 TCP 端口都可以自动暴露为 Room Preview****，不再要求逐个点击授权 (๑・ิ・ิ)◇

我接下来先按 Tutti 现有的 device-link / relay 思路把 Preview 数据路径对齐，只把公开资料无法判断的那一处留给你决定。

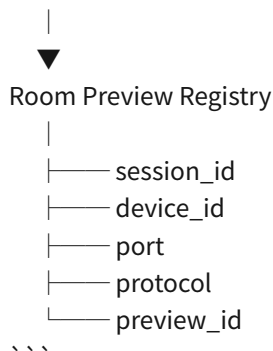
#### #076 助手

锁定：Room 内默认互信，Agent Session 监听到的 TCP 服务可以自动成为 Room Preview，不需要额外点“Share Preview” ~ (๑・ิ・ิ)◇

这里我查到的上游代码已经能给我们很强的实现暗示，所以我直接按 Tutti 的方向定几件事：上游 `packages/device-link` 已经负责 ICE、QUIC、authenticated bidirectional streams，以及 Relay/WebSocket-yamux owner tunnel；最近的 Agent Session 故障排查文档也明确区分了 direct、rendezvous、relay 等连接路径。与此同时 Browser Workbench 已经预留 `preview proxy` 边界，只是要求具体产品 Host 自己实现 route resolution。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/architecture/project-structure.md?utm_source=chatgpt.com))

所以 Preview 在产品模型上应该是：

```
```text id="d7zni6"
Agent Session Container
|
|—— :3000
|—— :5173
|—— :8080
|
▼
open-tutti-agentd
Port Discovery
```

Room 里的 Browser Node 再把：

```

```text id="fh50bd"
http://localhost:5173
```

```

解析成一个 Room-scoped Preview，而不是让 Bob 的浏览器真的访问 Bob 自己的 `localhost:5173`。

这也符合 Tutti VM 官方所描述的“本机 localhost 不部署即可让同 Room 的其他人实时打开”。([github.com](https://github.com/tutti-os/tutti?utm\_source=chatgpt.com))

不过这里碰到了我们之前一个选择和“尽量模仿 Tutti”之间的冲突：

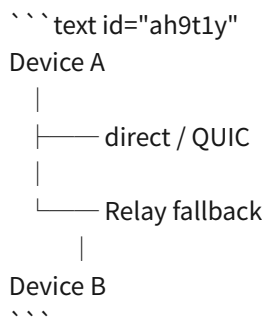
你之前选择了：

```

```text id="9ukzij"
tuttid ↔ Server
HTTP/OpenAPI + WebSocket
```

```

而 Tutti 上游现在明显已经在往：



这种 DeviceLink 数据通道走，尤其 Shared Agent 已经能看到这套 direct / relay connection machinery。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/architecture/project-structure.md?utm\_source=chatgpt.com))

对于我们来说，最理想其实是把两者区分：

```

```text id="jmkoer"
Control Plane
tuttid ↔ open-tutti-server
→ HTTPS + WebSocket
→ Room / Presence / Tasks / metadata
```

```

Data Plane
Device ↔ Device
→ DeviceLink
→ Preview TCP streams
→ Agent interactive streams
→ 大数据传输

无法直连
→ open-tutti-server Relay
` ` `

这样 Server 平时真的主要做调度，Preview 流量不必：

` ` `text id="44m2ex"
Alice → Server → Bob
` ` `

而可以：

` ` `text id="207ttq"
Alice ————— direct ————— Bob
` ` `

只有 NAT / 防火墙导致直连失败时才：

` ` `text id="ueq9fj"
Alice → Self Hosted Relay → Bob
` ` `

这会明显更接近 Tutti 当前仓库已经透露出来的架构。

所以这一题需要你重新定一下，因为它会修改我们之前的 A：

\*\*我们是否把之前的“A: HTTP/OpenAPI + WebSocket”限定为 Control Plane，并正式加入 Tutti 风格的`DeviceLink P2P + Relay fallback`作为 Data Plane? \*\* (｡•̀~•́)✧

#077 用户

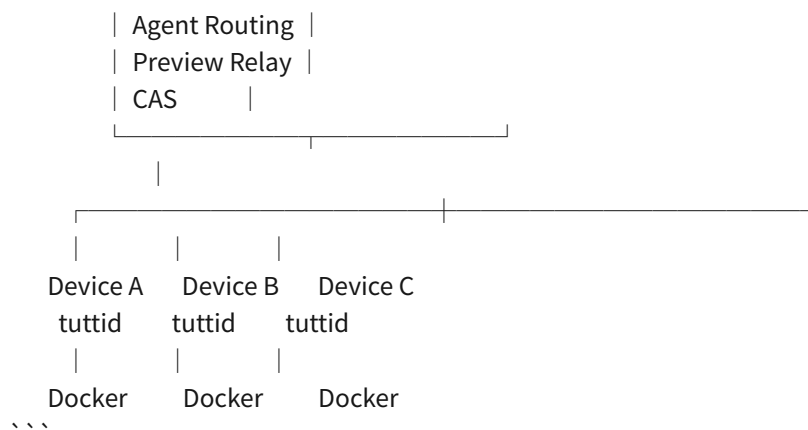
不要，保留主服务器调度的方式

#078 助手

锁定: \*\*不做 P2P，所有协作与数据流都由主服务器统一调度和中转\*\*~(๑•̀ㅂ•́)✧

因此整体网络模型就固定成：

` ` `text id="v5f6n2"
open-tutti-server
┌──────────────────┐
| Room Scheduler |
| Realtime WS |
| FS Op Stream |



这意味着我们之前定的 **HTTP/OpenAPI + WebSocket** 不需要修改：

```

```text id="1itqt7"
HTTP/OpenAPI
→ Room CRUD
→ CAS
→ Revision/Snapshot
→ Share / Join
→ metadata

WebSocket
→ Presence
→ File Operations
→ Agent Events
→ Borrow Commands
→ Terminal Streams
→ Preview Streams
```

```

不过我不会把 Preview 的字节流和普通 JSON business event 混在一个 socket 里。更合理的是仍然通过主服务器，但建立专门的 **multiplexed tunnel WebSocket**：

```

```text id="32fyxl"
Device A
|
| wss://server/tunnel
▼
open-tutti-server
|
| logical stream #184
▼
Device B
```

```

上层可以直接复用 Tutti 已经存在的 WebSocket/yamux 思路，只去掉 P2P/ICE/QUIC 那部分：

```

```text id="fnd89n"
一个 Device Tunnel
|—— stream 1 → Agent terminal
|—— stream 2 → localhost:3000

```

```
|—— stream 3 → localhost:5173
|—— stream 4 → borrowed agent interactive IO
|—— ...
...`
```

这样即使一个 Room 同时有几十个 Agent Session 和很多 Preview，也不用给每个服务建立一套新的 WebSocket (｡•̀-)-✧

而 localhost Preview 我也会尽量模仿 Tutti: **Desktop 内部继续显示 `http://localhost:<port>` 这种语义，由 Browser Workbench/Preview Proxy 把请求透明解析到 Room 中实际提供该端口的 Session，再经主服务器 Tunnel 转过去。 **

也就是 Bob 点：

```
```text id="rxg6g3"
http://localhost:3000
```
```

实际数据路径：

```
```text id="9jd6kp"
Bob Tutti Browser
  ↓
Bob tuttid preview proxy
  ↓
open-tutti-server
  ↓
Alice tuttid tunnel
  ↓
agent-session-123:3000
```
```

因此 Self Hosted 第一版甚至不需要 wildcard DNS，也不用要求用户配：

```
```text id="noxai8"
*.preview.example.com
```
```

部署仍然可以保持得很干净：

```
```text id="1l66fp"
open-tutti-server
.env
SQLite
CAS directory
HTTPS/WSS
```
```

下一处是 Tutti 公开资料无法替我们完全决定的：

**同一个 Room 里，如果两个不同 Agent Session 都监听 `localhost:3000`，用户输入 `http://localhost:3000` 时应该如何选择目标？ **

我想到比较符合 Tutti UI 的方式是： **端口并不全局唯一，Preview 实际绑定 `session\_id + port`； UI 里每个 Agent 的运行结果各自出现一个 Preview，点击哪个就访问哪个。 ** 只有用户手动输入裸 `localhost:3000` 时，才弹一个很轻的目标

选择器。

这个行为你接受吗? (˘˘˘)♡

#### #079 用户

我还是偏向，用户电脑名称.tutti 作为 host，例如 Anna 和 Leo 的电脑上都有 :3000，我访问 AnnasMacBookPro:3000 就行了

#### #080 助手

这个方向我觉得更干净~(๑•૩•) ♡ 我们就把 **Device hostname** 做成 Room 内的一等公民，而不是把 `localhost` 做一堆隐式映射。

例如 Room 里：

```
```text id="lhxgk0"
Anna's MacBook Pro
→ annasmacbookpro.tutti
```

```
Leo's PC
→ leospc.tutti
```
```

那么：

```
```text id="1qwg7"
http://annasmacbookpro.tutti:3000
http://leospc.tutti:3000
```
```

分别明确访问两台设备上的 `:3000`。如果 Anna 同时跑：

```
```text id="vknq8y"
:3000 → frontend
:5432 → postgres
:8000 → api
```
```

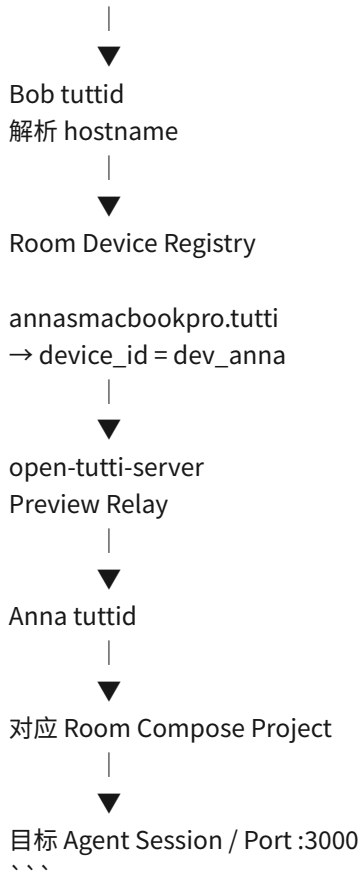
Room 内就统一表现为：

```
```text id="jg8bcz"
annasmacbookpro.tutti:3000
annasmacbookpro.tutti:5432
annasmacbookpro.tutti:8000
```
```

而它**不是公网 DNS**。因为我们已经确定所有流量经过主服务器，所以 `tutti` 只需要成为 Tutti 内部的 Virtual Host Namespace：

```
```text id="v80dwx"
Bob 的 Browser Workbench
```

```
GET http://annasmacbookpro.tutti:3000
```



所以完全不需要真的配置 DNS:

```
```text id="t81i20"
*.tutti
```
```

也不需要让系统 DNS 能解析它。Electron/Tutti Browser 在发出网络请求之前就由 `tuttid` 的 Preview Proxy 接管。

我还会把设备 hostname 注册成 Room metadata:

```

    ``text id="u2at0i"
RoomDevice {
    device_id
    display_name
    hostname
    owner
    connected_at
}
``

```

然后所有 Agent 都能获得类似的 Room 网络信息：

```
```text id="zacmss"
Available room devices:

- annasmacbookpro.tutti
- leospc.tutti
```

```
- johnsdesktop.tutti
```
```

这样 Agent 自己甚至可以执行：

```
```bash id="40iffk"
curl http://leospc.tutti:8000/api/health
```
```

当然实际网络访问仍然由我们的 Runtime 网络层拦截并通过 Server Relay，而不是让 Docker 去真实 DNS 查询 `tutti` ~ (。
• ̂-)◇

这也顺带解决了之前“两个 Agent 都监听 3000 怎么办”的问题：\*\*端口属于设备网络命名空间，而不是整个 Room 的全局命名空间。\*\*

不过还有一个二级冲突：

```
```text id="gwxzs1"
Anna 有两个 Agent Session
session A → :3000
session B → :3000
```
```

它们都在 \*\*同一台 Anna 电脑\*\* 上。

所以我下一问只需要确定这一点：

你希望同一设备上的多个 Agent Session \*\*共享一个虚拟网络命名空间\*\*，因此同一台电脑不能同时占用相同端口；还是允许各 Session 都有自己的 `:3000`，然后再给 hostname 增加 Session 层级？例如：

```
```text id="8kagfn"
claude-1.annasmacbookpro.tutti:3000
codex-1.annasmacbookpro.tutti:3000
```
```

哪一种更符合你想要的 Tutti VM? ($\geq \nabla \leq$)♡

#081 用户

允许各 Session 都有自己的 `:3000`，然后再给 hostname 增加 Session 层级，如果只有一个占用 3000 则允许缩写如 annasmacbookpro.tutti:3000，否则会弹出选择器

#082 助手

这个规则很顺：\*\*完整地址始终唯一，设备级短地址只是无歧义时的便利别名\*\*。我会把它作为 Preview/Room Network 的正式寻址规则，不再让端口冲突影响并发 Session (๑ • ̂•)₉◇

#083 助手

锁定啦～这个寻址模型我会定义成「\*\*Session 地址永远唯一，Device 地址是条件式别名\*\*」(๑ • ̂•)₉◇

例如 Anna 机器上：

```
```text id="al0nt2"
```

Claude Session A → claude-a.annasmacbookpro.tutti:3000  
Codex Session B → codex-b.annasmacbookpro.tutti:3000  
Claude Session C → claude-c.annasmacbookpro.tutti:5173  
```

完整地址始终可以访问：

```text id="3rs2hy"  
http://claude-a.annasmacbookpro.tutti:3000  
http://codex-b.annasmacbookpro.tutti:3000  
http://claude-c.annasmacbookpro.tutti:5173  
```

设备级地址则由 Server 的 `Room Preview Registry` 动态解析：

```text id="l1auuh"  
annasmacbookpro.tutti:5173  
→ 只有 claude-c 占用  
→ 直接路由到 claude-c

annasmacbookpro.tutti:3000  
→ claude-a + codex-b 都占用  
→ Browser Workbench 弹出 Session 选择器  
```

Canonical route 我会让 Server 实际维护：

```text id="aixif6"  
(room_id, device_id, session_id, port)  
```

`tutti` hostname 只是人类友好的虚拟 DNS 表示，所以设备改名、Session UI 名称改变都不会影响底层路由标识。

另外 Tutti 官方本身明确把「正在运行的进程和端口」「localhost 产物」「实时 Preview」作为 Room 共享工作状态的一部分，所以我们这种把端口注册进 Room、通过统一 Server Relay 访问的设计与它的产品语义是一致的。([tutti.sh](https://tutti.sh/zh?utm\_source=chatgpt.com))

这里还有一个必须处理的边界：\*\*Browser Workbench 可以弹选择器，但 `curl`、Agent、测试程序之类的非交互客户端不能。\*\*

例如：

```bash id="64kwb9"  
curl http://annasmacbookpro.tutti:3000  
```

而 Anna 恰好有两个 Session 都占用了 `3000`。

我建议此时\*\*绝不随机选择\*\*，而是返回一个明确的 `ambiguous host` 错误，并列出可用的 canonical hostname：

```text id="gn7tit"  
Ambiguous .tutti route: annasmacbookpro.tutti:3000



Available targets:

- claude-a.annasmacbookpro.tutti:3000
- codex-b.annasmacbookpro.tutti:3000

...

Browser UI 才弹选择器。

****你接受这种规则吗：交互式 Browser 遇到歧义弹选择器；CLI/Agent 遇到歧义直接报错，必须改用完整 Session hostname? **** 🐼(↗️👉🏻)🐼

#### #084 用户

不是，直接会解析出来 H5 单页选择器不就好了，Agent 看得懂的

#### #085 助手

对，这个更优雅～(๑ • ॢ •)₯✧ ****歧义本身就可以成为一个可访问的资源****，没必要先报错。

例如：

```
```text id="cg23xi"
GET http://annasmacbookpro.tutti:3000
```
```

Server 发现：

```
```text id="cdrr62"
Anna / Claude A → :3000
Anna / Codex B → :3000
```
```

就返回一个很轻的 H5 单页：

```
```text id="vw8hv1"
annasmacbookpro.tutti:3000
```

请选择目标 Session：

Claude Code • feature/auth
→ claude-a.annasmacbookpro.tutti:3000

Codex • fix-dashboard
→ codex-b.annasmacbookpro.tutti:3000

...

浏览器里用户直接点；Agent 用 `curl` 请求时也会看到结构清晰的 HTML 和完整 canonical URL，所以它自己就能选择目标 (≥▽≤)♡

我会再给页面增加机器友好的语义，比如：

```
```html id="ztahsk"
<a
href="http://claude-a.annasmacbookpro.tutti:3000"
```

```
data-session-id="..."
data-agent="claude-code"
>
Claude Code • feature/auth

```
```

这样不用专门为 Agent 发明另一套寻址协议。

不过这里有一个协议层边界需要你决定：

我们之前定的是\*\*任何 TCP 监听端口都可以发布\*\*。H5 选择器只对 HTTP/HTTPS 有意义，例如：

```
```text id="ivtr1n"
annasmacbookpro.tutti:3000 → HTTP ✓
```
```

但如果两个 Session 都监听 PostgreSQL：

```
```text id="vpjnj4"
annasmacbookpro.tutti:5432
```
```

PostgreSQL 客户端不可能显示 H5。

所以我建议：\*\*设备级缩写的歧义选择器只支持 HTTP/HTTPS；任意 Raw TCP 如果存在同端口歧义，就必须使用完整 Session hostname，例如 `codex-b.annasmacbookpro.tutti:5432`。 \*\*

这一条你接受吗？ (。・ヾ-)◇

#086 用户

是的

#087 助手

锁定这一条：

- HTTP/HTTPS：设备级短地址发生歧义时，返回 H5 Session 选择器。
- Raw TCP：发生歧义时必须使用完整 Session hostname。
- 完整地址始终确定，例如 `codex-b.annasmacbookpro.tutti:5432`。
- Agent 访问 HTTP 歧义地址时也会拿到同一份结构化 H5，因此可以自行读取链接并选择目标 $\mathcal{A}(\rightarrow \square \leftarrow) \mathcal{A}$

`tutti` 的实现我也会直接定为两层，不再问：Agent Session 容器通过 Room 内部 DNS 解析 `tutti`；Tutti 自己的 Browser Workbench 则由 `tuttid` 拦截并解析，因此都不依赖公网 DNS。

下一处确实会改变系统权限模型，需要你决定：

\*\*你希望 `\*.tutti` 只能在 Tutti Browser + Open Tutti 的 Agent 容器里访问，还是也希望用户直接打开系统 Chrome/Safari，输入 `http://annasmacbookpro.tutti:3000` 就能访问？ \*\* (。・ヾ-)◇

后者意味着 `tuttid` 还需要接管宿主机的 `tutti` DNS/代理配置。

#088 用户

只能在 Tutti Browser + Open Tutti 的 Agent 容器里访问

#089 助手

锁定：`.tutti` 是 \*\*Open Tutti 的私有虚拟网络域名\*\*，绝不改宿主机 DNS、`/etc/hosts` 或系统代理 $\mathfrak{A}(\text{~>}\square<\text{~})\mathfrak{A}$

也就是说：

```
```text id="vnny7e"
```

Tutti Browser

```
└── *.tutti ✓
```

open-tutti-vm-<roomId> 内的 Agent Session

```
└── *.tutti ✓
```

系统 Chrome / Safari

```
└── *.tutti ✗
```

宿主机 Terminal

```
└── *.tutti ✗
```

```
```
```

这样 `tuttid` 和 Room Runtime 自己维护解析逻辑即可，卸载/退出 Tutti 时不会给系统留下网络配置，也不会把 Room 内服务意外暴露给宿主机其他程序 $(\bullet\cdot\mathfrak{H}\cdot\mathfrak{Y})_9\diamond$

下一处会显著影响第一版工程量，而且不能单纯从 Tutti 架构替你决定：

\*\*Open Tutti VM 第一版的桌面平台范围是什么？\*\*

我建议在下面三种里确定一个：

\*\*A. macOS only\*\*：先把 Docker Desktop、FUSE、Host Workspace Import/Apply、Local CA、Agent Borrowing 全部打磨通。

\*\*B. macOS + Windows\*\*：更接近完整桌面产品，但 Host filesystem、路径语义、Docker Desktop、证书和权限都要同时处理。

\*\*C. macOS + Windows + Linux Desktop\*\*：第一版直接跨三平台，工程量最大。

你希望第一版覆盖哪一个？ $(\circ\cdot\mathfrak{S})\diamond$

#090 助手

下一处我需要你决定的是 **\*\*Room 创建时 Workspace 从哪里来\*\***：

如果 Anna 创建 Room，第一版是否允许她直接选择宿主机上的一个现有项目，例如：

```
```text id="eur55c"
```

```
~/Projects/my-app
```

```
```
```

然后 Open Tutti **\*\*复制/导入一份快照\*\***到：

```
```text id="n3o0p6"
open-tutti-vm-<roomId>-workspace
```
```

从这一刻开始两边完全脱离，不再做 bind mount 或双向同步；会议结束时房主再必须 Export 回某个目录。

也就是：

```
```text id="jbvoej"
Host Project
|
| Import snapshot
▼
Room Workspace
|
| 实时多人协作
▼
Export snapshot
|
▼
Host Folder
```
```

还是你希望第一版创建 Room 时\*\*只能新建空 Workspace / Git Clone\*\*，完全不提供“从本机目录导入”？(。・\_・)◇

#091 用户

允许她直接选择宿主机上的一个现有项目，结束后的导出改为\*\*\*应用到 Workspace\*

#092 助手

锁定啦～创建 Room 时可以直接选宿主机现有项目导入；会议结束不再叫“导出”，而是\*\*「应用到 Workspace」\*\*，把 Room 里的最终结果回写到最初选中的宿主机项目(๑・\_・)ゞ◇ 我先对照一下 Tutti 公开资料里有没有类似“apply back / sync back”的现成语义，再把这条收敛成不会误覆盖本地文件的流程。

#093 助手

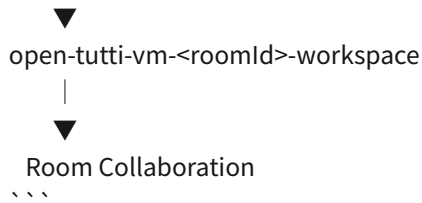
对，这样语义会比“Export”准确很多～(๑・\_・)ゞ◇ 我会把它正式定义成\*\*Import → Detached Room Workspace → Apply to Workspace\*\*。

Tutti 公开资料强调 Room 是共享边界，Room 内创建和修改的工作状态被共享，而 Room 外的本地工作保持私有；但公开资料里没有看到官方披露“会议结束后如何把 VM Workspace 合回原始本地目录”的具体算法，所以这一段我们需要自己设计。([github.com])([https://github.com/tutti-os/tutti?utm\\_source=chatgpt.com](https://github.com/tutti-os/tutti?utm_source=chatgpt.com)))

创建 Room 时：

```
```text id="2ms12c"
Anna Host

~/Projects/my-app
|
| Import to Room
| 记录 baseline
```



这里绝不 bind mount。导入完成后，两边立即脱离：

```
```text id="8s7mn9"
Host Workspace      Room Workspace
~/Projects/my-app   FUSE /workspace

可以被 Host 正常使用    所有人实时协作
不自动同步      ✗      Server Source of Truth
```
```

而创建 Room 的设备本地保存一份不会上传 Server 的关联：

```
```text id="yr15ay"
WorkspaceOrigin {
  room_id
  host_path
  baseline_revision
  baseline_manifest_hash
}
```
```

最后房主退出时，UI 不显示：

```
```text id="gwqc41"
Export Workspace
```
```

而显示：

```
```text id="5duarv"
Apply to Workspace
```
```

语义就是：

```
```text id="cg8oc1"
Room Final Revision
↓
计算相对于 Import Baseline 的变化
↓
应用到 ~/Projects/my-app
```
```

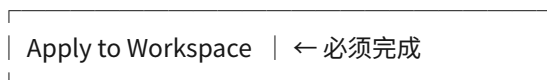
普通参与者如果他们本身也是从自己的某个 Workspace 加入，我认为也可以使用同样的「应用到 Workspace」动作；否则可以选择放弃本地 Room Replica。

这样房主退出流程就变成：

```text id="4t1so2"

Leave Room

房主：



Transfer Ownership

或

Disband Room



Leave

```

而不是产生一个 ZIP 或额外目录，很符合“我只是进去和别人一起开发了一会儿，现在把成果应用回原工程”的直觉 ( $\geq \nabla \leq$ )♥

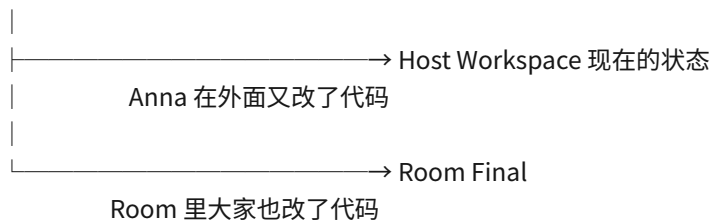
这里随之出现一个必须确定的冲突场景：

Anna 在创建 Room 后，**又用 VS Code 或其他程序修改了原来的 `~/Projects/my-app`**。

此时会出现三个状态：

```text id="bl7eek"

Import Baseline



```

那么「应用到 Workspace」时，我倾向做真正的 **三方合并**：

```text id="yp2rlm"

Baseline

+ Host Current

+ Room Final



无冲突 → 自动应用

有冲突 → 进入 Apply Conflict UI

```

而不是粗暴用 Room Final 覆盖 Host。

**你希望就采用这种三方合并语义吗？** 🤖(“> □ <”)🤖

直接覆盖即可，第一版简单点

#### #095 助手

锁定：第一版「应用到 Workspace」就走**直接覆盖**，不做三方合并、不做 Host 侧并发变更保护 (๑ • ๒ • ๓) ♡  
我先按 Tutti 公开实现习惯确认一下`.git` / 项目元数据应该怎么处理，再只问一个真正需要你定的边界。

#### #096 助手

锁定：第一版 **Apply to Workspace = 直接覆盖，不做三方合并**。这样实现会简单很多，也符合我们现在优先把 Tutti VM 核心协作跑通的目标 (๑ • ๒ • ๓) ♡

我会先把流程定成：

```
```text id="lo5ej5"
创建 Room
Host Workspace
  ↓ Import Snapshot
Room Workspace
  ↓ 多人协作
Room Final State
  ↓ Apply to Workspace
Host Workspace 被 Room 最终状态覆盖
```
```

第一版不判断用户在会议期间有没有用 VS Code 改过原目录；点击「应用到 Workspace」时明确提示这是破坏性操作即可。

这里有一个不能替你擅自定的关键语义：

**“直接覆盖”是否意味着最终 Host Workspace 与 Room 完全镜像？**

例如 Host 原本还有：

```
```text id="h43aak"
old-file.ts
```
```

但 Room 里已经删除了它。

应用后是：

```
```text id="cqjmm0"
A. 真正镜像
Room 没有 old-file.ts
→ Host 也删除 old-file.ts

B. 只覆盖/新增
Room 没有 old-file.ts
→ Host 仍保留 old-file.ts
```
```

我会倾向 **A**：最终目录严格等于 Room Final State，否则 Room 里的删除操作无法真正应用回来 (。・ 3- ) 你要 A 还是 B？

#097 用户

终目录严格等于 Room Final State

#098 助手

锁定：**Apply to Workspace 后，宿主目标目录必须严格镜像 Room Final State**。Room 中新增的文件会创建、修改的会覆盖、删除的也会从宿主删除，不保留 Room 最终状态里不存在的旧文件 (。・ 3- )

我接着把 `.git`、Git 状态和项目元数据这一层按 Tutti 的公开实现习惯对齐，能直接判断的就不再问你。

#099 助手

锁定：**「应用到 Workspace」是镜像式覆盖**。Room 中不存在的普通工作文件，应用后宿主也会删除；第一版不做三方合并或宿主侧变更保护 (。・ 3- )

这里我根据 Tutti 的公开边界直接补一条：**Git 凭据仍然必须是设备私有的**。官方明确说明 SSH keys、Git credentials 不会跨设备或上传云端；共享的是代码、文件变化、任务和产物这些 work state。([tutti.sh](https://tutti.sh/en/resources/blogs/tutti-vm-technical-path?utm_source=chatgpt.com))

但 `.git` 本身比较特殊：Agent 通常需要 `git status/diff/commit`，而把整个宿主 `.git` 原样共享又可能带入带 Token 的 remote URL、hooks 等内容。

所以这是下一处真正需要你决定的边界：

**第一版 Room 是否共享 `.git` 仓库元数据？**

我建议采用：导入项目时 **复制 `.git`**，但先做安全清洗（例如去掉可能带凭据的 remote URL、credential 配置），之后 Room 内 `.git` 作为 Workspace 的一部分同步；最终「应用到 Workspace」时也用 Room 的 `.git` 覆盖原目录。这样所有 Agent 看到同一套 branch/index/commit 状态，也最像“同一台协作电脑” (。・ 3- )

你希望这样做，还是 **只同步工作树，不同步 `.git`**？

#100 用户

复制 `.git`，但先做安全清洗

#101 助手

锁定：导入现有项目时 **复制 `.git`**，但先做安全清洗；Room 内继续共享 Git 状态，这样所有 Agent 看到一致的 branch / index / commit 视图，同时避免把宿主凭据和高风险 hooks 原样带进去 (。・ 3- ) 我再对一下 Tutti 仓库里有没有现成的 Git/workspace 处理约定，能复用的直接沿用。

#102 助手

锁定：**导入 Workspace 时复制 `.git`**，但做安全清洗后再进入 Room (。・ 3- )

我会直接把第一版清洗规则定成：

`text id="pku2c5"`

保留



- └── HEAD / refs / objects
- └── index
- └── branches / tags / commit history
- └── remote 名称
- └── 安全的 remote URL
- └── 普通 Git repository metadata

清除 / 重写

- └── .git/hooks/* ← 禁止把宿主机可执行 hook 带入 VM
- └── credential.* ← 凭据相关配置
- └── http*.extraHeader ← 很可能包含 Authorization
- └── URL 中的 username/password/token
- └── core.sshCommand ← 可能指向宿主机私钥/脚本
- └── core.hooksPath ← 避免跳到外部目录
- └── include / includelf ← 避免读取宿主机 Git 配置
- └── 指向 Workspace 外部的路径型配置

...

Git 凭据本身则继续走我们已经确定的 ****Private Capability Plane****，由 Agent Session 挂载和权限体系提供，而不是跟 ``.git`` 一起同步。Tutti 的安全范围本身也明确把“未经用户意图暴露本地文件、凭据、Agent Session 数据”视为安全问题，所以这个边界应该 fail-closed。([github.com](https://github.com/tutti-os/tutti/security?utm_source=chatgpt.com))

还有一个容易踩坑的情况我会直接处理：如果用户选中的项目是 ****Git linked worktree****，它的 ``.git`` 往往只是一个指向外部 gitdir 的文件。Open Tutti Import 不能把这个外部路径原样带进 Docker，而应把它 materialize 成 Room 内一个完整、独立的 Git repository。Tutti 自己的开发仓库也大量考虑 linked worktree 工作流，因此这个场景值得第一版就兼容。([github.com](https://github.com/tutti-os/tutti/blob/main/CONTRIBUTING.md?utm_source=chatgpt.com))

最终「应用到 Workspace」时，清洗后的 Room ``.git`` 也属于 Room Final State，因此会一起镜像回宿主机目录。原宿主机 Git credentials 仍然不来自 Room；需要认证时继续使用本机私有能力层。

接下来有一个会直接影响数据安全、而且 Tutti 公开资料不足以替我们决定的问题：

****如果 `open-tutti-server` 在会议过程中重启/崩溃，Room 要不要恢复？****

我建议 ****必须恢复****：Server 重启不应该被解释成“所有人退出会议”。SQLite 中保存 Room lifecycle、membership、权威 operation sequence 和 CAS 引用；重启后 Room 进入 ``.RECOVERING``，给原设备例如 5 分钟重新连接，重新建立 Workspace Replica / Agent Presence；5 分钟内一个人都没回来才解散并 GC。

你希望采用这个恢复模型吗？(。•̀-)-◇

### #103 用户

不用，所有人断开，断开时其他用户可选择导出，房主可选择应用到工作区

### #104 助手

锁定：****Server 不做 Room 恢复****。服务器正常重启、崩溃或不可达，都视为本次会议中断并结束，不尝试恢复原 Room  
⌂(↗> □ <↖)⌂

此时客户端进入本地收尾模式：

```
```text id="13ytc2"
open-tutti-server 断开
```

↓

所有参与者 Room Session 结束

↓

不再接受新的协作操作

普通参与者

└── 导出本机最后同步到的 Room Workspace

└── 放弃

房主

└── 应用到 Workspace

└── 暂不应用，保留本地副本供后续处理

...

这样也意味着我们必须保证 `room-sync` 始终维护一份\*\*足以脱离服务器独立 materialize 的本地 Replica\*\*：

```text id="ml6bef"

open-tutti-vm-<roomId>-workspace

```

不能把它仅仅做成依赖 Server 才能读取的稀疏缓存。Server 掉了以后，FUSE 应该切成只读/收尾状态，但现有 Workspace 内容仍可完整导出或应用。

Room 本身则无需恢复：

```text id="wtsr88"

Server 再次启动

→ 原 Room ID 无效

→ 原 Share URL 无效

→ SQLite 不恢复 Room lifecycle

→ 用户需要重新创建 Room

...

不过这里有一个无法完全消除的边界：Server 突然崩溃时，各设备最后收到的 operation sequence 可能略有不同。

例如：

```text id="ij40mm"

Server seq = 1052

Anna 最后同步到 1052

Leo 最后同步到 1051

↓ Server 突然崩溃

...

我们又已经明确不要 P2P 恢复，所以 Leo 不可能在断线后从 Anna 补回 `1052`。

因此下一问只需要确定这个行为：

\*\*Server 故障结束会议时，是否接受每个人的「导出 / 应用到 Workspace」都基于该设备最后成功同步到本地的状态？\*\*

也就是说极端情况下，不同参与者导出的最终内容可能差最后一两个尚未来得及广播到自己设备的操作；第一版不再尝试跨设备找“最完整副本”。(◦•̂̃)-◇

#105 用户

接受。或者其他人不用给导出功能，`room-sync`就不用维护一份\*\*足以脱离服务器独立 materialize 的本地 Replica 了\*\*

#106 助手

对，这样第一版会更干净~(◦•̂̃)-◇ 我建议就采用你这个收敛：\*\*只有房主拥有最终成果落地责任，普通参与者不提供“导出”能力。\*\*

这样我们可以把 Replica 分成两种角色：

```text id="wegen4"

Owner Device

- └── room-sync
  - └── open-tutti-vm-<roomId>-workspace
    - Full Replica
    - 始终保证完整
    - Server 崩溃后仍可 Apply to Workspace

Participant Device

- └── room-sync
  - └── open-tutti-vm-<roomId>-workspace
    - Working Cache / Partial Replica
    - 只保证当前 Agent 工作所需内容
    - 不保证断网后能独立 materialize 整个 Workspace

```

这样普通参与者的 FUSE 就可以真正采用 lazy materialization：

```text id="gwgoqx"

Agent

↓

/workspace

↓

open-tutti-fs

↓

本地已有？

- └── Yes → cache
- └── No → room-sync → Server CAS / current state

```

远端发生变化时，也不一定需要把所有内容立即写进参与者的 Named Volume，只维护必要 metadata、打开/近期使用文件以及运行时需要的数据即可。

而房主不同：

```text id="2b0apw"

Server Authoritative Workspace

↓

持续同步完整状态

↓  
Owner Full Replica  
↓  
Server 故障仍存在  
↓  
Apply to Workspace  
...

这样也重新定义退出 UX:

```text id="bi10ec"  
普通参与者退出
→ Leave
→ 销毁本机 Room cache / Session containers

房主正常退出
→ 必须 Apply to Workspace
→ 然后:
|—— Transfer Owner
|—— Disband

Server 故障
→ 所有人断开
→ 普通参与者直接结束
→ 房主可用最后完整 Replica 执行 Apply to Workspace
...

这还有一个很好的副作用: \*\*Server Source of Truth 和“房主灾难恢复副本”职责分开了。房主 Replica 不是第二个权威节点;会议正常运行时永远不能反向决定 Server 状态,它只是在本地完整镜像。

我建议 Named Volume 名字仍保持你定的:

`open-tutti-vm-<roomId>-workspace`

只是`room-sync`根据设备角色决定`full`或`lazy`策略,不需要两套命名 ٩(̄>̄<)٩

下一问只有一个关键边界: \*\*房主转让给 Leo 时,是否要求 Leo 的`room-sync`先从 lazy cache 升级成 Full Replica,并确认完整同步完成之后,才能真正完成 Owner Transfer? \*\*

我会倾向“是”,否则刚转让完服务器就挂了,新房主可能没有完整 Workspace 可以应用回来~(。・̀・)◇

#107 用户

必须 Apply to Workspace 直接自动化就好了,无需二次询问

是的,

#108 助手

锁定啦~房主的 \*\*Apply to Workspace 改成自动动作\*\*,不再弹第二次确认 (๑•̀•́)۶◇

也就是房主主动退出、解散房间,或因为 Server 故障导致会议结束时,`tuttid`都会先自动把当前可用的 Room Final

State 镜像覆盖回最初的 Host Workspace；只有 Apply 本身失败时才阻止正常收尾并显示错误。房主转让时，新房主必须先把自己的 `room-sync` 从 lazy cache 升级成 **Full Replica**，确认完整后才允许 Owner Transfer 完成。

#109 助手

锁定～另外根据 Tutti 官方公开技术路线，有一条网络行为我们也可以直接照搬，不需要再问：**Agent Session 的普通互联网/内网请求继续从 Agent 所有者自己的电脑网络出口发出**。官方明确说这样公司 IP allowlist、SSO 和内网服务都能继续工作。([tutti.sh](https://tutti.sh/en/resources/blogs/tutti-vm-technical-path?utm\_source=chatgpt.com))

因此我们的网络会分成两条：

```
```text id="uptspi"
```

普通网络

Agent Container

→ Docker Desktop NAT

→ Owner Host Network

→ Internet / 公司内网 / MCP 等

.tutti 虚拟网络

Agent Container

→ Open Tutti Virtual Network

→ room-sync

→ open-tutti-server

→ Remote Device

→ Remote Agent Session :port

```
```
```

而且我发现 `.tutti` 这里最好不要做成普通“所有域名都解析到一个 Proxy IP”，因为我们还要支持 **任意 Raw TCP**。更合适的是给每个 Room Device / Agent Session 分配一个只在 Open Tutti VM 内存在的 **Synthetic Virtual IP**：

```
```text id="r0j2kd"
```

annasmacbookpro.tutti

→ 100.96.0.10 // Device Virtual IP

claude-a.annasmacbookpro.tutti

→ 100.96.1.21 // Session Virtual IP

codex-b.annasmacbookpro.tutti

→ 100.96.1.22

```
```
```

这些地址不会真正出现在物理网络上。Agent Session 的路由表把 Open Tutti 保留网段全部送到 `room-sync`：

```
```text id="t1csfq"
```

100.96.0.0/12

↓

room-sync virtual network gateway

↓

(device/session, port)

↓

Server Tunnel

↓

目标 Session Container

```

这样 Raw TCP 就天然成立：

```
```text id="hk3gmq"
psql -h codex-b.annasmacbookpro.tutti -p 5432
redis-cli -h claudex-a.annasmacbookpro.tutti -p 6379
curl http://claudex-a.annasmacbookpro.tutti:3000
```
```

设备缩写：

```
```text id="e8zx2b"
annasmacbookpro.tutti:3000
```
```

则先路由到 Device Virtual IP，`room-sync` 根据：

```
```text id="9pz08s"
device_id + port
```
```

查询 Preview/Port Registry。如果只有一个 Session 占用，透明转发；如果多个 Session 占用且检测到 HTTP，则返回我们刚刚确定的 H5 选择器。Raw TCP 歧义则不能选择，要求完整 Session hostname。

这样 `.tutti` 网络会很像一个由主服务器控制的虚拟 Overlay Network，但\*\*不存在 Device-to-Device P2P\*\*，数据仍然严格：

```
```text id="b43lgp"
Device A → open-tutti-server → Device B
```
```

这也和我们已经确定的“服务器负责调度和中转”完全一致 (ゝ> ◡ <ゝ) ฅ

现在只剩一个 `.tutti` 网络里比较大的技术选择：

\*\*HTTPS 的歧义 H5 选择器要不要做本地 CA？\*\*

因为：

```
```text id="u9ucgr"
https://annasmacbookpro.tutti:3000
```
```

如果有两个 Session，TLS 已经在 HTTP 之前发生，我们没法直接插入 H5 页面。

第一版可以让 Open Tutti 为每个安装生成自己的本地 CA，并且\*\*只在 Tutti Browser 和 Agent Session Container 内信任\*\*，不安装进 macOS/Windows 系统证书库。这样 `room-sync` 就可以为 `.tutti` 动态签证书、终止 TLS，然后实现 HTTPS 的 H5 选择器；宿主机 Chrome/Safari 仍然完全不信任 `.tutti`。

\*\*你希望这样做吗？\*\* (。・ヾ-)◇

可以

#111 助手

锁定：第一版加入 **\*\*Open Tutti Local CA\*\***，但信任范围只存在于 Tutti 自己的运行环境里，不污染宿主机系统证书库 (๑ • ˆ ٠ • ๑)

结构就定成：

```
```text id="bw5p2p"
tuttid
├── Local CA
│ ├── CA private key → 设备本地私有存储
│ ├── CA certificate
│ └── 动态签发 *.tutti / session host certificate
```

```
Tutti Browser
├── Trust Open Tutti Local CA
```

```
Agent Session Containers
├── Trust Open Tutti Local CA
```

```
System Chrome / Safari / Host OS
├── 不信任
```
```

这样：

```
```text id="806c0h"
https://claude-a.annasmacbookpro.tutti:3000
```
```

可以透明代理；

而：

```
```text id="w2kvis"
https://annasmacbookpro.tutti:3000
```
```

如果多个 Session 冲突，`room-sync` 可以终止 `.tutti` TLS，返回 H5 选择器，再跳转到具体 Session 地址。Raw TLS/TCP 协议则仍然遵循之前的规则：不尝试做 H5 选择，必须使用完整 Session hostname (ˆ > ˆ < ˆ) ˆ

下一处会显著影响第一版工程量，而且不能单纯从 Tutti 架构替你决定：

\*\*Open Tutti VM 第一版的桌面平台范围是什么？\*\*

我建议在下面三种里确定一个：

\*\*A. macOS only\*\*：先把 Docker Desktop、FUSE、Host Workspace Import/Apply、Local CA、Agent Borrowing 全部打磨通。

\*\*B. macOS + Windows\*\*: 更接近完整桌面产品，但 Host filesystem、路径语义、Docker Desktop、证书和权限都要同时处理。

\*\*C. macOS + Windows + Linux Desktop\*\*: 第一版直接跨三平台，工程量最大。

你希望第一版覆盖哪一个？(。・ヾ-)◇

#112 助手

接下来只需要你定第一版平台范围啦~(。・ヾ-)◇

\*\*A. macOS only\*\*

\*\*B. macOS + Windows\*\*

\*\*C. macOS + Windows + Linux Desktop\*\*

我目前会更推荐 **\*\*B\*\***: Docker Desktop 的主要目标平台都覆盖到, 同时先不处理 Linux Desktop 各发行版差异。

#113 助手

你选 **\*\*A、B 还是 C\*\*** 呢? ($\geq \nabla \leq$)♡

#114 用户

B

#115 助手

锁定\*\*B: 第一版覆盖 macOS + Windows, Linux Desktop 暂缓\*\*~(๑ • ๒)₃ ✧

这样 Desktop、Docker Desktop Runtime、Host Workspace Import/Apply、Local CA、`.tutti` 虚拟网络都必须从一开始按双平台实现；我先核一下 Docker Desktop 在这两边对 `/dev/fuse` / Linux 容器能力的现实边界，再决定 FUSE 是否需要额外兼容层。

#116 助手

锁定：第一版 **\*\*macOS + Windows\*\***。需要注意的是，上游 Tutti Local 公开仓库目前仍写着 macOS 可下载、Windows “coming soon”，所以 Windows 端会是我们 Fork 里需要自己补齐的一块，而不是简单复用现成功能。([github.com](https://github.com/tutti-os/tutti?utm\_source=chatgpt.com))

Docker Desktop 在两个平台上都会把 Linux 容器跑在其 Linux VM 里，因此我们前面确定的 Go + `go-fuse`、Linux Agent Session image、`open-tutti-fs` 这套核心不用拆成 macOS/Windows 两份。Docker 官方目前 Windows 默认 backend 是 WSL2，同时也提供 Hyper-V；容器 root 也仍被 Docker Desktop 的 Linux VM 与 Windows Host 隔开。([docs.docker.com](https://docs.docker.com/desktop/setup/install/windows-install/?utm\_source=chatgpt.com))

我还会直接加一个启动前 `Runtime Preflight`：

```
```text id="tr6cc1"
```

tuttid

↓

## Docker Desktop available?

↓

## Linux containers?

↓

/dev/fuse + required mount capability available?

↓



Open Tutti workspace image compatible?

↓

PASS → 可以创建 / 加入 VM Room

FAIL → 前端明确显示具体缺失能力

````

这样不会等到 Agent 启动之后才发现 FUSE 根本挂不起来 (๑ • ๒ • ๓) ♡

下一处是 Windows 第一版真正需要你定的：

\*\*Windows 是否只支持 Docker Desktop 的 WSL2 backend? \*\*

我建议第一版直接要求 **\*\*WSL2 only\*\***，暂时不支持 Hyper-V backend。这样 Host Workspace Import / Apply、路径映射、Docker Runtime 探测、网络和 FUSE 只需要验证一个 Windows 环境；以后再补 Hyper-V。Docker Desktop 当前本来也默认使用 WSL2。([docs.docker.com](https://docs.docker.com/desktop/setup/install/windows-install/?utm\_source=chatgpt.com))

也就是：

```text id="pyo8dl"

macOS

→ Docker Desktop Linux VM ✓

Windows

→ Docker Desktop + WSL2 ✓

→ Hyper-V backend 暂不支持

````

你接受第一版 Windows **\*\*WSL2 only\*\*** 吗? (๑ • ๒ • ๓) ♡

#117 用户

Windows 已经可下载了，你的信息过时了。目前需要兼容 Windows x86\_64 macOS Intel macOS Apple Silicon 三个版本。

Docker VMM 已经开启 Windows Beta，我想默认 Docker VMM

#118 助手

你说得对，这两处需要按 2026 年现状更新，我刚才的信息确实过时了 (๑ • ๒ • ๓) ♡

我会把目标平台改成 **\*\*Windows x86\_64、macOS Intel、macOS Apple Silicon\*\*** 三个发行版本，并核对 Docker VMM 在三者上的实际可用性后，把 Runtime backend 策略直接定下来。

#119 助手

对，这里按当前状态修正。你指出得没错：Tutti 最新稳定版已经提供 **\*\*Windows x64\*\***，同时提供 **\*\*macOS Intel x64\*\*** 与 **\*\*macOS Apple Silicon arm64\*\***；GitHub 首页里 “Windows coming soon” 已经落后于实际 Release 状态。当前 v0.2.29 的发布资产明确列出了这三类构建。([github.com](https://github.com/tutti-os/tutti/releases?utm\_source=chatgpt.com))

Docker Runtime 我们也改成 **\*\*Docker VMM 优先\*\***。Docker Desktop 4.86 起，Docker VMM 已进入 Beta，而且现在明确支持 Windows；Docker 官方将它描述为与 WSL2 相比具有真实 VM 边界的替代方案。([docs.docker.com](https://docs.docker.com/desktop/features/vmm/))

但有一个平台差异必须保留：Docker VMM 当前选择界面只列 **Mac Apple Silicon + Windows**，并不支持 Intel Mac。Intel Mac 官方推荐 Apple Virtualization Framework，HyperKit 已 deprecated。([docs.docker.com](https://docs.docker.com/desktop/features/vmm/))

所以 Runtime Policy 我直接定成：

```
```text id="v99tdh"
Windows x86_64
├── Docker Desktop >= 4.86
└── Docker VMM (Beta) ← 默认 / 推荐

macOS Apple Silicon
├── Docker Desktop >= 4.86
└── Docker VMM (Beta) ← 默认 / 推荐

macOS Intel
├── Docker Desktop >= 4.86
└── Apple Virtualization Framework ← 默认
 HyperKit 不支持 / 不推荐
```
```

前端不用让用户理解这些细节。`tuttid` 的 Preflight 自动检测：

```
```text id="75mnzd"
RuntimePreflight
├── Docker Desktop version >= 4.86
├── CPU architecture
├── active VMM/backend
├── Linux containers
├── /dev/fuse
└── required capabilities
```
```

如果 Windows / Apple Silicon 没开 Docker VMM，就提示：

```
> Open Tutti VM 推荐使用 Docker VMM
> [打开 Docker Desktop 设置]
```

但第一版我建议仍允许用户继续使用现有 backend，只把 Docker VMM 标为推荐，而不是硬阻止 Room 创建。这样 Beta 本身出兼容问题时还能 fallback。

还有一个必须因此直接修改的地方：我们的 VM 镜像必须原生 **multi-arch**：

```
```text id="yar0f8"
open-tutti/workspace
├── linux/amd64
└── linux/arm64

open-tutti/room-sync
├── linux/amd64
└── linux/arm64
```
```

尤其 Apple Silicon 上不能偷懒跑 amd64 镜像，因为 Docker VMM 当前明确不支持 Rosetta，amd64 emulation 会很慢。
([docs.docker.com])(https://docs.docker.com/desktop/features/vmm/))

所以现在正式支持矩阵就是：

```
```text id="rybzy7"
 Desktop VM image
Windows x86_64 win-x64 linux/amd64
macOS Intel mac-x64 linux/amd64
macOS Apple Silicon mac-arm64 linux/arm64
```
```

下一题是目前 Tutti 公开资料不能替我们决定的，而且会影响部署体验：

``open-tutti-server` 第一版也要提供 Docker Compose 一键部署吗？\*\*

即用户只需要：

```
```bash id="vela1h"
cp .env.example .env
docker compose up -d
```
```

就启动 ``open-tutti-server + SQLite + Local CAS`，还是 Server 第一版主要以单 Go 二进制发布？(≥▽≤)♡

#120 用户

如果用户已经装了 Docker 就不要偷偷把设置修改为 Docker VMM 了。

提供

#121 助手

锁定这两点~(๑•۞•)✧

Docker Desktop 这边改成“只检测、不改设置”：

```
```text id="2ss3dh"
检测当前 backend
|—— Docker VMM
| |—— ✓ 推荐配置
|
|—— Apple Virtualization Framework / WSL2 / 其他可用 backend
| |—— ✓ 允许正常使用
| UI 可以提示 “Docker VMM 可获得更好的隔离/性能”
|
|—— backend 不满足 Open Tutti 所需能力
| |—— ✗ Preflight 报错，并告诉用户如何自行修改
```
```

也就是说 ``tuttid` \*\*绝不自动修改 Docker Desktop 配置、绝不偷偷重启 Docker Desktop\*\*。用户已经有自己的 Docker 工作环境，我们不能为了 Open Tutti 把它改掉 (｡• ̮ •)✧

Server 第一版则同时提供 Go 二进制和 Docker Compose，一键部署作为主要推荐方式：

```
```text id="ayequn"
open-tutti/
├── docker-compose.yml
├── .env.example
└── data/
```
```

大致就是：

```
```yaml id="f2lw77"
services:
 open-tutti-server:
 image: ghcr.io/open-tutti/open-tutti-server:latest
 restart: unless-stopped
 env_file:
 - .env
 ports:
 - "${OPEN_TUTTI_PORT:-8080}:8080"
 volumes:
 - ./data:/var/lib/open-tutti
```
```

`.data` 里同时放：

```
```text id="u5we40"
data/
├── open-tutti.db
├── objects/
└── server-state/
```
```

所以迁移/备份第一版甚至可以做到“停服务 → 复制整个 `.data/` → 恢复”。

服务器镜像也做：

```
```text id="0c5ma3"
linux/amd64
linux/arm64
```
```

虽然 Desktop 目前三个发行版本里 Windows/Intel Mac 都用 amd64，Apple Silicon 用 arm64，但这样 Self Hosted Server 也可以直接部署在 ARM NAS、小主机之类的环境里 🐧(> ◡ <)🐧

下一处需要你决定的是公网入口：

\*\*第一版 Docker Compose 是否自带 HTTPS 反向代理？\*\*

我倾向于：

```
```text id="e7yzbv"
默认：
```

open-tutti-server:8080

→ 用户自己接 Nginx / Caddy / Traefik / Cloudflare Tunnel

可选 compose profile:

Caddy

→ 自动 HTTPS

→ 用户只填 OPEN_TUTTI_DOMAIN=tutti.example.com

...

这样既不强迫已经有反代环境的人再套一层，又能让新用户做到接近一键部署。

你希望 **只提供裸 Server**，还是 **裸 Server + 可选 Caddy 自动 HTTPS**？(≥▽≤)♡

## #122 用户

裸 Server

## #123 助手

锁定：Server 第一版就是**裸 Go Server**，不内置 Caddy / Nginx / Traefik；生产环境由用户自己负责反向代理和 HTTPS。`docker compose up -d` 只启动 `open-tutti-server` 本身，部署面保持最小 (๑ • ๒ • ๓) ♡

接下来我先按 Tutti 的 Agent/runtime 公开边界确认一下“Agent CLI 到底装在哪一层”，这个会直接决定我们 VM 镜像怎么拆。

## #124 助手

从公开资料看，这一层其实可以直接按 Tutti 的思路定，不需要你再选啦～(๑ • ๒ • ๓) ♡

Tutti 明确把 Agent 放在 **managed local VM** 里运行，同时继续使用用户本机已有的登录状态、配置、MCP、Tools、Skills；而开源仓库现有 Runtime 体系已经采用 daemon-managed Node/Python runtime，并且 Agent provider installer 可以把 adapter 安装到 daemon 自己管理的 per-agent prefix，而不是依赖宿主机全局环境。([tutti.sh](https://tutti.sh/en/resources/blogs/tutti-vm-technical-path?utm_source=chatgpt.com))

所以 Open Tutti VM 第一版我会直接定成：

```text id="1l135o"

Host OS

├── 用户现有 Agent credentials/config

├── MCP / Skills / SSH / Git credentials

└── 不直接提供 Agent executable

|

| private mounts / capability bridge



Agent Session Container

├── Open Tutti managed runtime

├── provider adapter

├── Claude Code / Codex / ... CLI

├── open-tutti-agentd

└── open-tutti-fs → /workspace

...

也就是说：

\*\*CLI / adapter / Node/Python runtime 由 Open Tutti 管理；身份和私人配置来自宿主机。\*\*

不做这种事情：

```
```text id="vnh6tn"
/Users/anna/.local/bin/claude
 ↓ bind mount
container
```
```

这样 Windows、Intel Mac、Apple Silicon 三个平台不会因为宿主机安装路径、Node 版本、shell 环境完全不同而把 VM Runtime 搞碎。

更具体一点，我会把镜像拆成：

```
```text id="x09cy9"
ghcr.io/open-tutti/workspace-base
|—— linux/amd64
|—— linux/arm64
```
```

包含：

```
|—— open-tutti-agentd
|—— open-tutti-fs
|—— managed bash/core tools
|—— Git
|—— CA bundle + Open Tutti Local CA injection
|—— runtime bootstrap
```
```

Agent Provider 再由 `tuttid` 根据上游已有 adapter 动态准备：

```
```text id="bpwymz"
SessionCreate
  ↓
Provider = codex
  ↓
prepare provider runtime
  ↓
启动 session container
  ↓
挂载 device-private Codex state
  ↓
启动 Codex
```
```

这样我们不需要为：

```
```text id="vcxlru"
ClaudeContainer
CodexContainer
HermesContainer
OpenCodeContainer
```
```

维护四套几乎一样的镜像。

Provider 支持范围也直接跟着 Tutti 的 `Agent Host + Provider Adapter` 走，不在 VM 里重新维护一份名单。这样上游以后加一个 Provider，我们主要补它的 `ContainerSessionRuntime` 适配，而不是重新造 Agent 生命周期。Tutti 当前公开产品也仍然强调复用用户已有 Agent、配置、MCP、Tools 和 Skills，而不是替换 Agent 本身。([tutti.sh](https://tutti.sh/en?utm_source=chatgpt.com))

接下来有一个公开资料里我还没看到明确答案、而且会明显影响 Docker 生命周期的问题：

****同一台电脑第一版是否允许同时在线加入多个 Room? ****

比如 Anna 同时打开：

```
```text id="zly5yq"
Room A
→ open-tutti-vm-roomA
→ 3 个 Agent Sessions

Room B
→ open-tutti-vm-roomB
→ 2 个 Agent Sessions
```
```

还是第一版限制 ****一台设备同一时间只能加入一个 VM Room****? (。・_・)◇

#### #125 用户

同一台电脑第一版允许同时在线加入多个 Room

#### #126 助手

锁定: ****同一台设备可以同时加入多个 Room****~(๑・_・)ゞ◇

运行时就天然按 Compose Project 隔离：

```
```text id="sjvoox"
Anna 的电脑

open-tutti-vm-roomA
├── room-sync
├── claude-session-a
└── codex-session-b

open-tutti-vm-roomB
├── room-sync
├── claude-session-c
├── claude-session-d
└── codex-session-e
```
```

每个 Room 都有独立的 Workspace Volume、FUSE operation stream、`.tutti` 虚拟网络、Local CA 路由上下文和 Session 容器。Agent Provider 的私有能力可以同时被多个 Room 的 Session 使用，我们不额外做人为并发限制。

这里随之出现一个必须定的边界：

****同一个宿主机 Workspace，是否允许同时被导入到两个仍在进行中的 Room？****

例如：

```
```text id="a3tbaf"
~/Projects/my-app
|—— Room A 正在开发
|—— Room B 也正在开发
```
```

最后两个 Room 都会执行「Apply to Workspace」，由于我们已经确定第一版是****镜像式直接覆盖****，就会出现明显的 last-write-wins。

我倾向第一版直接禁止：****一个 Host Workspace Path 同一时间只能作为一个活跃 Room 的 Origin****；如果已经被 Room A 使用，创建 Room B 时提示该 Workspace 正在另一个 Room 中协作。

你希望这样限制吗？(｡•̯•)◇

#127 用户

是的

#128 助手

锁定：****同一个 Host Workspace Path 同一时间只能绑定一个活跃 Room****。`tuttid` 会用规范化后的真实路径做本地 Workspace Lease；Room 结束并成功 Apply 后才释放，避免两个会议最后互相镜像覆盖 (｡•̯•)◇

接下来我会继续优先照 Tutti 的 Room/Share 行为走，先核一下官方目前分享链接和加入流程的公开表现。

#129 助手

锁定：同一个 Host Workspace 不能同时作为多个活跃 Room 的 Origin；这个限制由本机 `tuttid` 管理，不依赖 Server。

我没有查到 Tutti 公开资料对 ****Share Link 如何唤起 Desktop**** 的具体协议，因此这一处需要我们自己定 (｡•̯•)◇

下一问：

用户收到：

`https://tutti.example.com/share/<share-id>`

之后，你希望它如何加入 Room？

****A. Share URL 本身提供一个极简 H5 加入页****：输入房间密码 → 点击「Open in Open Tutti」→ 通过 `open-tutti://join/...` 唤起桌面客户端；没安装则提示下载。

****B. Share URL 直接尝试唤起 Open Tutti****：密码完全在 Desktop 内输入，网页只负责 Deep Link 跳转。

我更倾向 ****A****：链接在微信/Slack/浏览器里打开都有明确反馈，而且密码仍然不进入 URL；Server 本身是 Go，可以顺手提供这一张静态 H5，无需额外 Web 服务。你选 A 还是 B？(≥▽≤)♡



### #130 用户

A

### #131 助手

锁定 **A: Share URL 提供极简 H5 加入页**~密码在网页里输入，验证通过后再通过 `open-tutti://...` 唤起桌面端；密码不进入 URL，也不写进 deep link (๑ • ๒ • ๓)✧

我顺便查一下 Tutti 开源仓库现有的 deep-link / protocol handling，能复用的话就直接沿用。

### #132 助手

锁定 **A**。我会把加入链路做成“网页验证 + 一次性 Join Ticket”，而不是把房间密码塞进 `open-tutti://` 链接里~(๑ • ๒ • ๓)✧

流程直接定为：

```
```text id="1p1591"
https://my-server/share/<share-id>
↓
极简 H5
↓
输入 Room Password
↓
POST /api/share/<share-id>/join-ticket
↓
Server 验证密码
↓
签发一次性短期 Join Ticket
↓
open-tutti://join?server=...&ticket=...
↓
Open Tutti Desktop
↓
tuttid redeem ticket
↓
加入 Room
```
```

`Join Ticket` 我会直接设成：

```
```text id="vdc4vg"
• 单次使用
• 例如 60 秒有效
• 绑定 server_id
• 绑定 room_id
• 绑定 share_id
• redeem 后立即失效
• SQLite 只记录 hash / redemption state
```
```

这样即使 Windows 的 custom protocol 参数可能经过进程启动参数，也不会暴露真正的 Room Password。

实现位置也可以直接遵循 Tutti 开源仓库现有分层：Electron `main` 负责 Deep Link 这种 OS 集成能力，收到 `open-tutti://` 后把解析结果交给 `tuttid`；真正的 join-ticket redemption、Room membership 等业务逻辑仍归 `tuttid`，不把 Electron 变成第二个业务核心。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/architecture/desktop-backend-access.md?utm_source=chatgpt.com)) Windows 和 macOS 的 Deep Link 入口不同，这部分也由 Electron main 层统一适配即可。([github.com](https://github.com/electron/electron/blob/main/docs/tutorial/launch-app-from-url-in-another-app.md?utm_source=chatgpt.com))

另外这和 Tutti 目前公开的权限语义也很接近：**创建 Room 需要邀请码，但加入别人的 Room 不需要服务器邀请码**；Room 邀请链接本身是共享入口。([tutti.sh](https://tutti.sh/en?utm_source=chatgpt.com))

下一处是我们“无账号 Self Hosted”与官方 Tutti 有明显差异、无法直接照搬的地方：

****Room 里的用户身份怎么表示？****

我比较建议每个 Open Tutti Desktop 有一个长期的本地 Profile：

```
```text id="unw1n7"
LocalProfile
├── display_name = "Anna"
├── avatar      = optional
└── DeviceIdentity = Ed25519
```
```

然后同一个人可以有：

```
```text id="oceh97"
Anna
├── annasmacbookpro.tutti
└── annaswindowsdesktop.tutti
```
```

Server 不建立传统 User Account，只在加入 Room 时接收 `display\_name + device public key`。名字可以在 Desktop 设置里改，存本机；不同 Self Hosted Server 默认复用这个名字。

****你希望采用这种「本地 Profile + 多 Device」身份模型，还是直接把每台设备都当成一个独立用户？**** ( $\geq \nabla \leq$ )♡

### #133 用户

直接把每台设备都当成一个独立用户，Display Name 可以在设置里设置

### #134 助手

锁定：****每台设备就是一个独立用户身份****，不再额外抽象 Person/User Account 这一层~(๑ • ॢ •)ゞ

数据模型可以直接简化成：

```
```text id="90o5nm"
DeviceIdentity
├── device_id
├── ed25519_public_key
├── display_name    // 用户在设置中修改
├── machine_name    // 系统设备名
└── created_at
```

```

例如:

```
```text id="dbl9c1"
device_id: dev_01...
display_name: Anna
machine_name: AnnasMacBookPro
```

Room 显示:

Anna

```
.tutti hostname:
annasmacbookpro.tutti
```
```

所以即使 Anna 同时有 Mac 和 Windows, 它们也被视为两个独立参与者:

```
```text id="o9vrjg"
Anna • Mac
Anna • Windows
```
```

各自拥有独立的 Agent、凭据、Docker Runtime、Presence 和房主资格。

Ed25519 私钥只存在设备本地, Server 保存公钥; 换电脑或者重装导致 Device Identity 丢失, 就视为一台新的设备。这样第一版完全不需要账号绑定、设备同步、账号找回这些复杂系统 🐼(“> □ <”)🐼

下一处 Tutti 官方中心化服务无法替 Self Hosted 直接回答的问题是:

****Open Tutti Desktop 是否要支持同时保存多个 Self Hosted Server? ****

例如设置里可以有:

```
```text id="abeegt"
Servers
```

✓ 公司
<https://tutti.company.com>

家里
<https://tutti.home.example>

朋友的服务器
<https://tutti.example.org>

```

创建 Room 时选择一个 Server; 收到 Share Link 时则自动识别并临时/永久添加对应 Server。

还是第一版规定 Desktop 只能配置一个 Server? (｡• ̯ •)💎

## #136 助手

锁定: **Open Tutti Desktop 第一版支持同时保存多个 Self Hosted Server**~(๑ • ॢ •)๑✧

我会把它设计成设备本地的 `ServerRegistry`:

```
```text id="i661gt"
Servers
├── 公司
│   ├── https://tutti.company.com
├── 家里
│   ├── https://tutti.home.example
└── 朋友
    ├── https://tutti.friend.example
```
```

创建 Room 时选择目标 Server; 不同 Server 的 Room 可以同时在线, 因此一台电脑可以同时存在:

```
```text id="9uokjd"
open-tutti-vm-roomA → company server
open-tutti-vm-roomB → home server
open-tutti-vm-roomC → friend server
```
```

收到:

```
```text id="l7o3rf"
https://new-server.example/share/abc...
```
```

时, Desktop 根据 Share URL 自动识别 Server, 不要求用户提前手动添加。Server Registry 只保存地址、显示名称和设备对该 Server 的本地状态, 不引入跨服务器账号体系 ฅ(˘> ౧ <˘)ฅ

下一处需要你决定的是 Self Hosted 很现实的 TLS 问题:

****第一版是否允许使用自签名 HTTPS 证书的 Open Tutti Server? ****

例如公司内网部署:

```
```text id="y8guin"
https://tutti.internal
→ 企业 CA / 自签名 CA
```
```

我建议支持, 但****不能提供“忽略所有 TLS 错误”****。首次连接时显示证书指纹, 让用户显式信任并对这个 Server 做 certificate pinning; 之后证书异常变化就阻止连接。

也就是支持:

```
`系统可信证书✓`
`用户显式信任的自签名证书✓`
```

`-k / 忽略 TLS 验证 ✕`

你希望这样做吗? (｡• ̯ •)✧

#137 用户

可以

#138 助手

锁定: **支持系统可信证书, 也支持用户显式信任的自签名/企业 CA 证书; 绝不提供全局跳过 TLS 验证**。首次信任后按 Server 做证书/CA pinning, 后续指纹异常变化就阻断并要求重新确认 (｡• ̯ •)✧

这一层可以直接实现, 不需要再问更多细节。

#139 助手

这条也锁定啦~自签名/企业 CA 支持到这里就够了, 不再往 TLS 细节上继续发散 (｡• ̯ •)✧

接下来我会转去补 **Tutti 最核心但我们还没系统设计的那一块: Room 内的对话、@Agent、任务路由与借用上下文**。这部分官方公开资料能看出不少端倪, 我先按官方行为收敛, 只有无法判断的地方再问你。

#140 助手

我对照了官方最新说明后, 这块其实可以直接定: **第一版不能只做“共享文件 + 借 Agent”, 还应该把 Room Group Chat 和 Big @ 一起带上**。官方把 @ 明确定义成 Room 的核心上下文总线, 可以引用对话、文件、产物、任务, 也可以调用 Agent / App; 借来的 Agent 也正是靠这套 Room 上下文做到“接着干”。

#141 助手

这部分我会直接按 Tutti 的公开模型定下来, 不再逐项问你~(｡• ̯ •)✧

第一版加入 **Group Chat + Big @ + Room Context Graph**。官方现在明确把 `@` 定义为 Tutti VM 的核心交互: 可以引用 Agent 历史对话、群聊、文件、任务、App 产物, 也可以直接调用 Agent 和 App; 而借来的 Agent 同样能读取这些 Room 上下文继续工作。([tutti.sh](https://tutti.sh/en/resources/usecases/at-in-tutti-vm?utm_source=chatgpt.com))

Server 侧因此不只保存文件 Revision, 还维护 Room 生命周期内的上下文对象索引:

```
```text id="crdyry"
RoomContextObject
├── Conversation
├── Message
├── Task
├── File
├── Artifact
├── AgentSession
├── App
└── GroupChatEntry
```
```

`@` 不通过“复制一坨文字”实现, 而是引用稳定对象:

```
```text id="zbz0as"
@claude-session-abc
```

```
@task-123
@file:src/auth.ts
@group-chat-message-456
@artifact-789
```
```

例如 Bob 调用 Anna 借出的 Claude:

```
```text id="4i6amr"
Bob
|
| "@Claude 继续 @task-123,
| 参考 @codex-session-xyz"
▼
```

```
open-tutti-server
|
|—— resolve context refs
|—— 创建 Task
|—— route 到 Anna Device
↓
Anna / Agent Session Container
↓
Claude Code
↓
共享 /workspace
```
```

Conversation、Task 和执行过程属于 **Room Work State**，所以服务器保存索引/metadata；较大的历史内容、附件和产物继续进 CAS。Room 解散后整体释放并 GC。这也符合 Tutti 当前公开的“Room 内的对话、任务、文件和产物都是下一步可以继续引用的上下文”模型。([tutti.sh](https://tutti.sh/zh/resources/usecases/at-in-tutti-vm?utm_source=chatgpt.com))

App 体系也没必要重造。上游已经有 Workspace App Catalog、`tutti.app.json`、`bootstrap.sh`、App package、App Center 和远程 Catalog 机制；我们应该直接让 App 跑进 Room VM，App 产物自然进入 Workspace + Context Graph。([github.com](https://github.com/tutti-os/tutti-apps/blob/main/docs/architecture/tutti-packaging.md?utm_source=chatgpt.com))

现在只有一个 Self Hosted 特有的问题需要你决定：

****Open Tutti 第一版默认要不要继续连接 Tutti 官方的远程 App Catalog? ****

也就是：

```
```text id="fvqe0f"
A. 默认继续使用官方 Tutti App Catalog
+ 允许配置自己的 Catalog URL
+ 允许完全关闭远程 Catalog

B. Self Hosted 默认完全不请求 Tutti 官方服务
  只加载内置 App / 用户自己配置的 Catalog
```
```

如果强调真正 Self Hosted，我会比较倾向 ****B**** 🙄(˃ ˂ )🙄

默认继续使用官方 Tutti App Catalog

## #143 助手

锁定~第一版默认继续使用 ****Tutti 官方 App Catalog****，Self Hosted 只替换 VM/Room/协作后端，不主动 Fork App 生态 (๑•૭•)₯💎

我会把 Catalog 设计成客户端能力，而不是让 `open-tutti-server` 代理：

```text id="ptsj3s"

Open Tutti Desktop

默认 Catalog → Tutti Official Catalog

——可追加 Custom Catalog

——可关闭某个 Catalog

open-tutti-server

—— 不负责转发 App Catalog 流量

...

因此服务器继续符合我们之前的原则：**\*\*尽量只做 Room 调度、Workspace 权威状态、CAS、Agent/Preview Relay\*\***。官方 Catalog 挂掉也不应该导致已有 Room 无法工作；已经安装/缓存的 App 继续可用。

配置上我会默认：

```
```text id="rlu1pe"
```

Official Tutti Catalog ✓ Enabled

Custom Catalog      Optional

### Offline / Disable Remote Optional

...

而 App 真正启动后仍运行在当前 Room 的 Docker Compose Project 内，其 Workspace 文件和产物遵循我们已经确定的 FUSE/Room 协作规则，不因为 App 来自官方 Catalog 就绕过隔离边界 ( $\rightarrow \square \leftarrow$ )

下一处是“会议式 Room”体验里还没有定的核心细节：

****房间密码你希望怎么产生？****

我比较建议模仿视频会议：创建 Room 时自动生成一个 ****6 位数字密码****，房主也可以手动修改，例如：

```text id="v3dty3"

分享链接:

<https://tutti.example.com/share/xK9pQ...>

房间密码:

482731

...

Server Invite Code 负责“能否创建房间”，Room Password 负责“能否加入这个房间”，两者完全独立。

你希望采用这种 **\*\*自动生成 6 位数字 + 允许房主修改\*\*** 的方式吗? ($\geq \nabla \leq$)

#144 用户

可以

#145 助手

锁定: Room 创建时自动生成 \*\*6 位数字密码\*\*，房主可修改；Server Invite Code 与 Room Password 完全分离 (˘> ˘<) ฅ

接下来我不继续问小 UX 细节了，我会把焦点移到 \*\*Fork 后的仓库结构与代码边界\*\*。这部分 Tutti 开源仓库本身已经给了很多答案，我先按上游结构直接收敛。

#146 助手

这一块可以直接照 Tutti 的仓库哲学来，不需要再拆成另一个完全独立项目啦~(๑ • ๓ •)๑

我建议继续维持 \*\*单 Monorepo Fork\*\*。上游现在明确把 `apps/` 用于用户入口、`services/` 用于具有独立生命周期和状态所有权的长期后端、`packages/` 只放真实的多消费者边界；Electron Desktop 也继续只是 `tuttid` 的壳，业务逻辑不能搬回 Electron。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/architecture/project-structure.md?utm\_source=chatgpt.com))

因此我们新增的结构我会定成：

```
```text id="cx8aep"
```

```
tutti/
```

```
|—— apps/
```

```
| |—— cli
```

```
| |—— desktop
```

```
|
```

```
|—— services/
```

```
| |—— tuttid
```

```
| | |—— Room client
```

```
| | |—— DockerDesktopRuntime
```

```
| | |—— ServerRegistry
```

```
| | |—— Workspace Import / Apply
```

```
| | |—— Device Identity
```

```
| |
```

```
| |—— open-tutti-server
```

```
| | |—— HTTP/OpenAPI
```

```
| | |—— WebSocket
```

```
| | |—— Room scheduler
```

```
| | |—— Workspace sequencer
```

```
| | |—— CAS
```

```
| | |—— Preview / TCP relay
```

```
| | |—— SQLite repository
```

```
| |
```

```
| |—— open-tutti-room-sync
```

```
| | |—— Full/Lazy replica
```

```
| | |—— Operation client
```

```
| | |—— CAS cache
```

```
| | |—— .tutti virtual network gateway
```

```
| |
```

```
| |—— open-tutti-fs
```

```
| |—— go-fuse/v2
```



```

| |—— POSIX → canonical operations
| |—— /workspace mount
|
|—— packages/
| |—— events/... ← 尽量复用上游
| |—— workspace/... ← 扩展真实共享边界
| |—— clients/
| |—— open-tutti-server ← generated Go client
|
|—— docs/
| |—— architecture/
| |—— conventions/
...

```

“open-tutti-server” 作为新的独立部署后端放 “services/” 是完全符合上游规则的；“room-sync” 和 “open-tutti-fs” 也是有真实 runtime 生命周期的进程，所以先放 “services/”，而不是为了“代码看起来整齐”提前创建大量 “packages/common”。([github.com])([https://github.com/tutti-os/tutti/blob/main/docs/architecture/project-structure.md?utm_source=chatgpt.com](https://github.com/tutti-os/tutti/blob/main/docs/architecture/project-structure.md?utm_source=chatgpt.com))

同时我会把我们前面决定的协作协议整理成真正的共享 seam：

```
```text id="cj7bt8"
```

Workspace Protocol

FileOperation

```

|—— TextPatch
|—— BlobReplace
|—— Create
|—— Remove
|—— Rename
|—— Mkdir
|—— MetadataChange

```

OperationEnvelope

```

|—— room_id
|—— operation_id
|—— author_device_id
|—— agent_session_id
|—— base_seq
|—— server_seq
|—— timestamp

```

WorkspaceSnapshot

```

|—— sequence
|—— root_tree_hash
|—— CAS references
...

```

文本 OT 第一版我也会直接选 “UTF-8 byte range + old/new hash + Server authoritative sequence”，因为 FUSE/POSIX 面对的是 byte stream，而不是 Monaco 的 UTF-16 document model；这样 Go、Git diff、文件 hash 和 CAS 全部保持一种坐标系，不需要来回转换 (。・\_・)◇

现在有一个非常大的产品/工程边界还没定，而且 Tutti 上游无法替我们决定：

\*\*这个 Fork 第一版要不要保留原版 Tutti Local 的“非 Room 本地 Workspace”模式？\*\*

也就是 Desktop 首页同时存在：

```text id="c3ynmd"

Local Workspace

→ 原 Tutti Local

→ HostRuntime 代码路径

→ 不可 Borrow

Open Tutti VM Room

→ DockerDesktopRuntime

→ 可多人协作 / Borrow

```

还是第一版 UI \*\*彻底隐藏原本的 Local Workspace 产品入口，只保留 Open Tutti VM Room\*\*，但 HostRuntime 代码仍然留在仓库里？(。・ˇ-)◇

#147 用户

只保留 Open Tutti VM Room

#148 助手

锁定：\*\*第一版 UI 彻底移除原 Tutti Local Workspace 产品入口，只保留 Open Tutti VM Room\*\*~(๑・ㅂ・)9◇

`HostRuntime` 仍然保留在代码里，方便跟上游合并和未来调试，但：

```text id="i9kwqr"

HostRuntime

|—— 代码保留

|—— 不删除测试

|—— 不作为用户可选 Runtime

|—— Desktop 前端不可达

|—— 不允许创建 Host Workspace

|—— 不允许 Agent Borrowing

```

Desktop 首页可以直接收敛成：

```text id="visr2m"

Open Tutti

Create Room

Join Room

Servers

└── Company  
└── Home  
└── Friend

#### Settings

└── Display Name  
└── Servers  
└── Docker Runtime  
└── Agent Providers  
...

这样整个 Fork 的产品心智非常干净: ****Open Tutti = Self Hosted Tutti VM Client****, 不会同时存在两套 Workspace 模型 (。  
• ~-)◇

不过现在出现一个很关键、之前还没有解决的问题:

假设 Anna 是房主, 她创建 Room 时绑定:

```
```text id="9umtd8"
Anna
~/Projects/my-app
  ↑
Room Workspace
```
```

后来 Anna 把房主转让给 Leo。

我们已经规定 Leo 必须先:

```
```text id="4hi0tx"
Lazy Replica
→ Full Replica
→ 完整同步成功
```
```

但 Leo 此时****没有一个 Host Workspace 可以在最终退出时执行自动 `Apply to Workspace`****。

所以转让房主时, 我认为还必须要求 Leo 选择一个本机 Workspace 目标, 例如:

```
```text id="9zpxku"
Become Room Owner

选择 Workspace:
C:\Projects\my-app
```
```

然后立即把当前 Room State 镜像到这个目录, 并建立新的:

```
```text id="hof1vv"
WorkspaceOrigin {
  room_id
  host_path = "C:\\Projects\\my-app"
  owner_device_id = Leo
}
```

```
}  
```
```

之后 Leo 才正式成为 Owner。

也就是说 **Owner Transfer 是一个三阶段事务**：

```
```text id="7qsj11"
```

1. Leo Full Replica ready
 2. Leo 选择并初始化 Host Workspace
 3. Server commit ownership transfer
- ```
```
```

任一步失败，Anna 仍然是房主。

你接受这种规则吗？(≥▽≤)♡

#149 用户

可以

#150 助手

锁定～房主转让就按三阶段事务执行：\*\*新房主先补齐 Full Replica，再选择并初始化自己的 Host Workspace，最后 Server 才提交 Owner Transfer\*\*；中途任何一步失败，原房主身份不变 (๑ • ๒ • ๓) ♡

我接着把“Room 创建 / 加入后 Agent 如何自动准备运行环境”这一块对照 Tutti 现有 runtime 约定收敛一下，尽量不再把仓库里已经有答案的问题丢给你。

#151 助手

房主转让这一块就完整闭环了～(๑ • ๒ • ๓) ♡

我对照 Tutti 最新公开说明后，还发现一个我们接下来必须明确的“大边界”：官方现在把 Tutti VM 描述成不仅共享文件，还共享“配置、运行中的服务、数据、依赖”的同一个最新工作状态；同时 Agent 本身仍运行在各自本机的受管 Linux VM。([tutti.sh](https://tutti.sh/en/resources/blogs/tutti-vm-technical-path?utm\_source=chatgpt.com))

我们已经决定不在主服务器上真正运行 Workspace，因此我会把其中能直接复刻的部分定成：

```
```text id="bekgcd"
```

共享并进入 Room Source of Truth

- ├── 源码 / 文档 / 产物
- ├── Git 状态
- ├── 项目配置
- ├── Agent 对话 / Task / Work State
- ├── 端口与运行服务的 Registry
- └── 可重建运行环境的声明

设备本地

- ├── Agent credentials
- ├── MCP secrets
- ├── SSH / Git credentials
- ├── Provider identity
- └── architecture-specific cache

└── 实际运行中的进程

、、、

尤其 **``node\_modules``、``.venv``** 这类宿主机依赖目录不能直接 Import 后共享**``**。Windows/macOS 上生成的依赖可能包含本机二进制，而我们的 Session 是 Linux，并且还同时有 `linux/amd64` 与 `linux/arm64`。所以上游现有“由 Tutti 管理 Node/Python runtime、不要依赖宿主机全局 runtime”的方向值得直接继承。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/conventions/workspace-app-runtime.md?utm_source=chatgpt.com))

因此下一题是真正需要你决定的：

**``Room 的项目开发环境第一版怎么定义？``**

我最建议做成：

```
```text id="ag1nlv"
Open Tutti Managed Linux Base
+
项目自己的环境声明
```
```

优先识别类似：

```
```text id="1la14f"
.devcontainer/devcontainer.json
Dockerfile
.tutti/runtime.yaml
```
```

没有声明时就使用 Open Tutti 默认 Ubuntu 开发镜像，内置 Git、Node、Python、常用 shell/build 工具；  
`package.json`、`lockfile`、`requirements.txt` 等同步，但依赖在每台设备对应架构的 Session 环境里重新安装。

也就是说允许项目**``自定义 Agent Session 镜像/开发环境``**，而不是所有项目永远只能跑一个固定的 `workspace-base`。

第一版你希望支持这种 **``项目级自定义开发环境``**，还是先只提供 Open Tutti 固定的 Managed Linux Base？ (。・ヾ-)◇

#### #152 用户

识别：

```
.opentuttivm/Dockerfile
.devcontainer/devcontainer.json
```

都没有：

```
ubuntu:latest
```

#### #153 助手

锁定～项目运行环境解析顺序就按你给的来 (๑・ㅂ・)ゞ◇

```
```text id="tnr58l"
1. .opentuttivm/Dockerfile
↓ 不存在
2. .devcontainer/devcontainer.json
```

↓ 不存在
3. ubuntu:latest
```

这里我会把“项目镜像”和“Open Tutti Runtime”分开。即使用户自己提供 Dockerfile，也不能要求他们把 `open-tutti-fs`、`open-tutti-agentd` 等组件写进去：

```
```text id="nq3x00"
Project Base Image
  ↓
Open Tutti Runtime Injection
  ↓
最终 Agent Session Image
```

注入：

```
|—— open-tutti-fs
|—— open-tutti-agentd
|—— Open Tutti CA
|—— .tutti virtual-network runtime
|—— provider runtime bootstrap
|—— 必要的 entrypoint/supervisor
```
```

例如：

```
```text id="ipyas3"
.opentuttivm/Dockerfile
→ open-tutti-project:<roomId>
```

然后派生

```
→ open-tutti-session:<roomId>-<arch>
```
```

`devcontainer.json` 也尽量按 Dev Container 标准解析其中的 `image` / `build.dockerfile` / `features` / `environment` 等，而不是自己再发明一套配置格式。

并且所有构建都发生在**各参与者自己的 Docker Desktop** 上：

```
```text id="l8lxgk"
Apple Silicon → linux/arm64
Intel Mac    → linux/amd64
Windows x64  → linux/amd64
```
```

Server 不构建、不保存 OCI Image，只保存 Workspace/配置本身。这样仍然符合我们“Server 尽量只调度”的原则  

下一处是很大的运行环境语义：

假设 `.opentuttivm/Dockerfile` 或 `devcontainer.json` 在 Room 里被 Agent 修改了。

你希望 **Open Tutti 自动检测环境定义变化，然后自动 rebuild 所有参与者本机的 Room Runtime**，还是显示一个类似：

`Environment changed — Rebuild`

由每台参与者主动触发重建? (｡•̀｡)✧

#### #154 用户

显示一个类似:

`Environment changed — Rebuild`

由每台参与者主动触发重建

#### #155 助手

锁定: 环境定义变化只广播状态, 不自动重建任何人的本地 Runtime~(｡•̀｡)✧

当 Room 中以下文件发生变化时:

```
```text id="17pj2d"
.opentuttim/Dockerfile
.devcontainer/devcontainer.json
```
```

Server 广播:

```
```text id="dl9tro"
room.environment.changed
|—— revision
|—— changed_files
|—— changed_by
```
```

每台设备独立进入:

```
```text id="tx9itb"
Environment changed — Rebuild
```
```

但现有 Agent Session 继续运行, 不会被别人修改 Dockerfile 后突然杀掉。

用户点击 `Rebuild` 后, 该设备才执行:

```
```text id="e4csi0"
当前 Workspace Revision
↓
重新 build / resolve project image
↓
注入 Open Tutti Runtime
↓
生成新的 local environment generation
↓
重启该 Room 在本机的 Agent Session Runtime
↓
```

重新挂载同一个逻辑 /workspace
```

Workspace 本身不受 rebuild 影响；FUSE/CAS/Room Operation Stream 继续以 Server 的 Workspace 状态为准。

我还会给 Runtime 增加：

```
```text id="xryhft"
environment_generation
environment_source_hash
```
```

于是 Room 里可能暂时出现：

```
```text id="wwzhqr"
Anna  generation 12 ✓ current
Leo   generation 11 ⚠ rebuild available
Bob   generation 12 ✓ current
```
```

这是允许的。**环境版本不一致不会阻止继续协作**，只是 UI 明确提示，避免某台机器因为旧依赖产生奇怪结果 (ノ>□<")

下一处比较关键，而且不是 Tutti 公开资料能直接替我们决定的：

如果用户点击 `Rebuild` 时，该设备上当前有几个 Agent Session 正在执行任务，应该：

**直接等待这些任务自然完成后再 rebuild**，还是**立即中断该设备所有正在运行的 Agent Session，然后 rebuild**？

我会倾向前者：进入 `Rebuild Pending`，不再启动新的本地 Session，等正在运行的任务结束后自动重建。(。・ヾ-)◇

#### #156 用户

立即中断该设备所有正在运行的 Agent Session，然后 rebuild，然后对所有因此中断的 Agent 发送「continue」

#### #157 助手

锁定：**Rebuild 是强制中断式**。用户点击后，当前设备这个 Room 里的所有 Agent Session 立即停止；环境重建完成后，系统重新恢复这些 Session，并对每个因 Rebuild 被中断的 Agent 自动发送一次 `continue`，让它们基于新的环境继续未完成工作 (●・彡)◇

这里会牵涉到 Session 恢复状态放在哪里，我先对照一下 Tutti 上游现有的 resume/recovery 边界，尽量直接复用。

#### #158 助手

这个行为可以直接落到 Tutti 上游现有的 Agent Host 生命周期里，不需要我们另造一套“重建后恢复协议”啦～(●・彡)◇

上游现在已经明确规定：Session / Turn 的创建、取消、Resume、Recovery 都由 `packages/agent/host` 统一拥有；而且现有实现本身就依赖持久化的 provider session identity 和 runtime snapshot 来完成 daemon 重启后的 Session 恢复。  
([github.com](https://github.com/tutti-os/tutti/blob/main/AGENTS.md?utm_source=chatgpt.com))

因此我们把 Rebuild 定成下面这个事务：



用户点击 Environment changed — Rebuild



↓

↓

```
+ provider_session_id
```

+ agent_target_id

+ 当前 Turn / Goal

- + runtime snapshot



↓

↓

↓

↓

↓

```
send("continue")
```



...

例如：

Claude A 正在 coding → interrupt → resume → "continue"

Codex B 正在跑测试 → interrupt → resume → "continue"

...

而且 `continue` 必须是:

## Resume Provider Session



↓

↓

...

而不是启动一个全新的 Agent Conversation。上游目前对取消后的正确恢复也正是强调下一次真实 prompt 应继续使用原 `providerSessionId`，而不是把 Session 当新会话重新启动。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/conventions/troubleshooting/agent-session-lifecycle.md?utm_source=chatgpt.com))

Agent Session 的可恢复状态因此不能存在于 disposable container filesystem 里，而应该在设备私有的持久化区域：

```
```text id="o0he02"
```

Host / Device Private State

Open Tutti Agent State

- |—— session metadata
- |—— provider\_session\_id
- |—— runtime snapshot
- |—— agent\_target\_id
- |—— conversation projection
- |—— provider-private session state
 - | private mount



new Agent Session Container

```
```
```

这也意味着：

```
```text id="peiqt"
```

```
docker rm agent-session-xxx
```

```
```
```

****不能等于删除 Agent Conversation。 ****

Container 是 Runtime；Session 是 Tutti 的持久业务对象。这个边界与上游当前架构是一致的。([github.com](https://github.com/tutti-os/tutti/blob/main/AGENTS.md?utm_source=chatgpt.com))

如果某 Provider 在重建后 `Resume` 失败，则这一只 Agent 标成：

```
```text id="461i4i"
```

```
Resume failed after environment rebuild
```

```
```
```

其他 Agent 不受影响，也不能为了恢复它把整个 Room 回滚。上游现在也明确要求单个 runtime recovery 错误不能拖垮整个 daemon/runtime。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/conventions/troubleshooting/agent-session-lifecycle.md?utm_source=chatgpt.com))

下一处我觉得是 ****Agent Borrowing 最关键的交互权限问题****：

Bob 正在使用 Anna 借出的 Claude，Claude 突然产生一个诸如：

```
```text id="1md3tc"
```

```
Allow this command?
```

```
Allow MCP action?
```

```
Allow network access?
```

```
```
```

这样的权限确认。

这个 Approval 应该交给 **Bob（当前借用者）** 决定，还是必须交给 **Anna（Agent 所有者）** 决定？(｡•̀~)✧

#### #159 用户

交给 **Bob（当前借用者）** 决定，兼容 Claude Code / Codex 的 Auto Mode，还要额外兼容 OpenCode

#### #160 助手

锁定：借用 Session 的所有 Approval 都由**当前借用者**决定，不再回传给 Agent 所有者；同时第一版必须把 Claude Code、Codex 的自动审批/Auto Mode，以及 OpenCode 的权限模型都纳入统一 Adapter 设计里(｡•̀~)✧  
我先把三家的当前权限/自动模式接口对齐一下，然后直接给出统一抽象，只在公开资料无法决定的地方再问你。

#### #161 助手

锁定：第一版的 Agent Adapter 必须把 **Claude Code、Codex、OpenCode 三套权限体系统一接进 Open Tutti，但不把它们硬压成完全相同的行为**~(｡•̀~)✧

截至现在，Claude Code 已有原生 `auto / manual / acceptEdits / plan` 等权限模式，而且 2026 年 8 月已经开始把 Auto Mode 作为 Pro/Max/Team 新 Session 的默认模式；Auto Mode 会在工具调用前做安全审查，真正需要人工处理时才继续产生 approval。([support.claude.com](https://support.claude.com/en/articles/14554922-claude-code-user-faq?utm_source=chatgpt.com))

Codex 当前的 app-server 协议本身就很适合我们做 Tutti Adapter：它会向客户端发出 server-initiated approval request，并等待客户端返回 decision；同时现在支持 `approvals\_reviewer = auto\_review`，也就是 UI 中的类似“Approve for me”模式。([github.com](https://github.com/openai/codex/blob/main/codex-rs/app-server/README.md?utm_source=chatgpt.com))

OpenCode 也已经有非常清晰的 permission 层：`allow / ask / deny`，并提供 `--auto`，自动批准所有没有被显式 `deny` 的权限请求；人工 Ask 支持 `once / always / reject`。([opencode.ai](https://opencode.ai/docs/permissions/?utm_source=chatgpt.com))

所以我会把上层抽象定成：

```
```text id="okmn0c"
AgentPermissionMode
|—— PROVIDER_DEFAULT
|—— MANUAL
|—— AUTO
|—— PROVIDER_SPECIFIC
```
```

而不是试图造：

```
```text id="htna1i"
Claude.auto == Codex.auto == OpenCode.auto
```
```

因为三者实际上不是完全相同的安全机制。

Adapter 负责映射：

```
```text id="mw8b5o"
```

Claude Code

AUTO

→ native auto mode

Codex

AUTO

→ approval\_policy = on-request

→ approvals\_reviewer = auto\_review

OpenCode

AUTO

→ --auto

```
```
```

人工审批统一进入一个 Open Tutti 事件模型：

```
```text id="xrz387"
```

AgentApprovalRequest

|—— approval\_id

|—— room\_id

|—— session\_id

|—— borrower\_device\_id

|—— provider

|—— kind

| |—— shell

| |—— filesystem

| |—— network

| |—— mcp

| |—— skill

| |—— other

|—— description

|—— command?

|—— paths?

|—— target?

|—— native\_payload

|—— available\_decisions

```
```
```

数据路径则固定为：

```
```text id="mcciml"
```

Anna Device

Borrowed Claude

↓

Claude asks permission

↓

open-tutti-agentd

↓

AgentAdapter

↓

open-tutti-server

↓

```
Bob Desktop
↓
Bob chooses
↓
open-tutti-server
↓
Anna Device
↓
native approval response
...

```

\*\*Anna 不收到 Approval 弹窗。 \*\* Anna 的身份是 capability owner；Bob 才是当前 Session 的 operator。

Auto Mode 则更简单：

```
```text id="9totni"
Provider Native Auto Mode
↓
能自己决定
→ Open Tutti 完全不打扰 Bob

```

```
Provider 判断必须人工审批
↓
AgentApprovalRequest
↓
Bob
...

```

OpenCode 还有一个细节值得我们直接兼容：它的 `always` 可以产生 Session/Project 范围的 permission rule，因此 Adapter 不能只支持简单的 Yes/No，而应该保留 provider-native decision，例如：([opencode.ai](https://opencode.ai/docs/permissions/?utm_source=chatgpt.com))

```
```text id="h1ywb5"
ApprovalDecision
|—— ALLOW_ONCE
|—— ALLOW_SESSION
|—— ALLOW_PERSISTENT // provider 支持时
|—— DENY
|—— NATIVE(...)
...

```

但 **\*\*Open Tutti 不自己永久保存这些 Provider 权限规则\*\***。Provider 要写自己的 `.claude/settings.json`、Codex config 或 OpenCode permission state 时，它仍属于我们之前定义的 Private Capability Plane，不进入 Room CAS。

而 Container 隔离仍然是最外层硬边界：

```
```text id="t87m6w"
Provider Auto / Full Access
≠ Host Full Access

```

它最多是：

Full Access inside Agent Session Container

...

无论 Bob 在 Claude/Codex/OpenCode 中开启多激进的自动模式，都不能因此获得宿主机任意文件权限。这个区分非常重要 (〰> □ <〰)〰

下一题只剩一个真正的产品权限选择：

****是否允许借用者 Bob 在自己借用的 Session 中，自由切换 Agent 的权限模式？****

例如把 Anna 分享出来的 Claude 从 `manual` 改成 `auto`，或者 Codex 从 `Ask for approval` 改成 `Auto-review`。我倾向 **允许，而且这个设置只作用于 Bob 的这一条借用 Session，不修改 Anna 自己的全局 Provider 默认设置**。

#### #162 用户

是

#### #163 助手

锁定：**借用者可以在自己的 Borrowed Session 内自由切换权限模式**；这个选择只绑定 `room\_id + session\_id`，不会写回 Agent 所有者的全局 Claude/Codex/OpenCode 配置 (〰 • 〰 • 〰)〰

同时我会顺带把私有能力挂载做成“全局配置只读 + Session 状态可写 overlay”，避免借用者通过切换 Auto Mode 意外永久改掉 Anna 本机的 Provider 配置。

#### #164 助手

锁定：**权限模式是 Session 级配置**。Claude Code / Codex / OpenCode 的 Adapter 优先通过启动参数、API 或 Session overlay 实现 `manual ↔ auto`，不会改写 Agent 所有者的全局配置；Provider 凭据本身如果需要刷新 Token，仍可在 Private Capability Plane 内由 Provider 进程正常维护，不进入 Room Workspace (〰> □ <〰)〰

我也核了一下 Tutti 官方公开资料：它明确说借用 Agent 是 Room-scoped、所有者可随时收回，同时 Agent 的工作过程与上下文属于 Room；但**没有公开说明“借用者突然断线/离开时，正在跑的借用 Agent 怎么办”**，所以这里确实要我们自己定。([tutti.sh](https://tutti.sh/en/resources/usecases/borrow-agents?utm_source=chatgpt.com))

下一问：

Bob 正在借用 Anna 的 Claude 跑一个长任务，此时 **Bob 主动退出 Room 或突然掉线**，这个 Session 怎么处理？

我比较倾向区分两种情况：

```text id="zjdssg"

Bob 主动 Leave Room

→ 立即 Interrupt 他正在操作的 Borrowed Sessions

→ Session 工作记录仍保留在 Room Context

→ Anna 的 Agent sharing 本身不受影响

Bob 意外断线

→ 给 Bob 5 分钟 Operator Grace Period

→ Agent 可以继续执行不需要人工 Approval 的部分

→ 如果产生 Approval，则暂停等待

→ Bob 5 分钟内回来：继续

→ 5 分钟仍未回来：Interrupt Session

```

你希望采用这个规则吗? (｡•̮̮-)✧

#165 用户

可以

#166 助手

锁定: 借用者主动退出就立即中断其 Borrowed Sessions; 意外断线则给 **5 分钟 Operator Grace Period**, 期间 Agent 可继续无需审批的工作, 需要 Approval 时暂停等待, 超时仍未回来再 Interrupt~(｡•̮̮-)✧

#167 助手

下一处会直接影响第一版工程量: ``.devcontainer/devcontainer.json`` 的兼容范围。

Dev Container 规范很大, 里面可能包含 ``features``、``mounts``、``forwardPorts``、``postCreateCommand``、``runArgs``、``privileged``、Docker-in-Docker 等能力。我们又有自己的 FUSE、``.tutti`` 网络和安全边界, 所以不适合无条件照单全收。

我建议第一版做 **安全子集兼容**:

````text id="yljsa2"`

支持

- |—— image
- |—— build.dockerfile / build.context / build.args
- |—— features
- |—— containerEnv
- |—— remoteEnv
- |—— workingDir / workspaceFolder
- |—— postCreateCommand
- |—— postStartCommand
- |—— 基础 user 配置

忽略或拒绝

- |—— privileged
- |—— 任意 host bind mounts
- |—— docker.sock
- |—— 自定义 workspaceMount
- |—— 会绕过 Open Tutti FUSE 的 mounts
- |—— 破坏 Room 网络/权限边界的 runArgs

`````

如果遇到不支持字段, 就明确显示:

``Dev Container contains unsupported Open Tutti VM options``

而不是静默执行。

你希望第一版按这个 **安全子集** 做, 还是目标就是尽量完整兼容 Dev Container Spec? (｡•̮̮-)✧

#168 用户

只看 image 与 build.dockerfile / build.context / build.args，其他不兼容

#### #169 助手

锁定啦～`.devcontainer/devcontainer.json` 第一版只把它当作**镜像来源描述文件**，不实现 Dev Container Runtime 语义 (๑ • ๕ •)๑

只解析：

```
```text id="ank9yr"
image
```

```
build.dockerfile
build.context
build.args
```
```

其他字段即使存在，也**不参与 Open Tutti VM Runtime 构建或启动**，例如 `features`、`mounts`、`runArgs`、`forwardPorts`、`postCreateCommand`、`containerEnv`、`privileged`、`workspaceMount` 等都不执行。UI 可以提示“该字段暂不受 Open Tutti VM 支持”，但不会因为这些字段存在就拒绝整个项目。

优先级继续保持：

```
```text id="f5jman"
.opentuttivm/Dockerfile
  ↓ 不存在
.devcontainer/devcontainer.json
  ↓ 没有 image/build
ubuntu:latest
```
```

如果 `.devcontainer` 同时有 `image` 和 `build`，我会按 Dev Container 常规语义优先使用 `build`，因为它描述的是最终要构建的开发镜像。

下一题是自定义镜像兼容范围：**第一版是否要求用户提供的 `.opentuttivm/Dockerfile` / devcontainer image 必须是 Linux glibc 系（例如 Ubuntu/Debian/Fedora），不支持 Alpine/musl、distroless、scratch？**

这样我们才能稳定注入 `open-tutti-fs`、`open-tutti-agentd`、CA、shell 和 Provider runtime，而不用第一版就维护多套 libc/runtime 兼容层～(｡ • ̎-)๑

#### #170 用户

是的

#### #171 助手

锁定：第一版自定义开发镜像只支持**Linux + glibc 用户态**；Alpine/musl、distroless、scratch 等直接标记为不兼容，不尝试偷偷 fallback (๑ • ๕ •)๑

我会顺手把构建安全边界定死：

```
```text id="8lf9cj"
.opentuttivm/Dockerfile
.devcontainer build.context
```


- build context 必须位于 Room Workspace 内
- 禁止 .././ 跳出 Workspace
- 不允许把宿主机目录作为 build context
- 不注入 Host Docker socket
- Open Tutti Runtime 在项目镜像构建完成后再注入
- ...

这样即使 Room 里的 Agent 修改了 Dockerfile，也只能影响这个 Room 自己的 Docker Runtime。

下一处比较大的设计问题是 **Room 内终端**：

Tutti VM 里除了 Agent，用户自己也会需要 Terminal。第一版你希望用户点击「New Terminal」时，也像 Agent 一样**每个 Terminal 创建独立 Session Container**；还是 Terminal 默认进入某个已有 Agent Session 容器？

我更倾向前者：

```
```text id="eewo7d"
Terminal Session
→ 独立 container
→ 同一个 FUSE /workspace
→ 同一个 .tutti 网络
→ 同一个 Project Image
→ 不继承任何 Agent Provider 凭据
```
```

这样用户 Terminal 和借出的 Claude/Codex/OpenCode 权限不会混在一起～(。・\_・)◇

#172 用户

每个 Terminal 创建独立 Session Container

#173 助手

锁定：**每个 Terminal 都创建独立 Session Container**，与 Agent Session 一样挂载同一个逻辑 `/workspace`、接入 `.tutti` 虚拟网络，但不继承任何 Agent Provider 私有凭据；Terminal 只是纯开发 Shell Runtime (๑ • ๒ • ๓)◇

我再对照一下 Tutti 公开资料里 Terminal/Workbench 是否有“共享给他人操作”的语义，能照搬的就直接照搬。

#174 助手

锁定：**每个 Terminal = 独立 Session Container**。它使用当前 Room 的 Project Image、同一个逻辑 `/workspace` 与 `.tutti` 网络，但不挂载 Claude/Codex/OpenCode 的 Provider 私有凭据。这样 Terminal 和 Agent Session 的生命周期、PID/临时文件/端口都互相隔离 (๑ • ๒ • ๓)◇

上游已经有独立的 `packages/workspace/terminal`，而且 Desktop 的架构明确把 terminal stream 归为 `renderer` → `tuttid` 的业务流；不过我没有查到 Tutti VM 公开资料明确说明“别人的 Terminal 能否远程接管操作”。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/architecture/project-structure.md?utm\_source=chatgpt.com))

所以这里只问一个官方资料无法替我们决定的问题：

Room 里的 Terminal 要不要允许其他参与者共享操作？

例如 Anna 创建了：

```
```text id="7bf53p"
terminal-a.annasmacbookpro.tutti
```
```

Bob 可以看到终端实时输出，并直接往同一个 PTY 输入命令；也就是 Terminal 像 Agent 一样成为 Room 内可共享的 Session。

还是 \*\*Terminal 只允许创建它的设备本人输入，其他成员只能看到其产生的文件/服务，而看不到也不能操作 Terminal 本身\*\*？ (｡•̎̎)✧

#175 用户

不要，其他成员只能看到其产生的文件/服务，而看不到也不能操作 Terminal 本身

#176 助手

锁定： \*\*Terminal Session 完全私有\*\*~(｡•̎̎)✧

```
```text id="eb5imh"
Terminal Session
|—— 创建者本人：可看、可输入、可关闭
|—— 其他 Room 成员：不可看 Terminal UI
|—— 其他 Room 成员：不可读取 PTY 输出
|—— 其他 Room 成员：不可输入命令
|
|—— Terminal 产生的共享结果：
|—— /workspace 文件变化 ✓
|—— .tutti 发布的监听服务 ✓
|—— Room Artifact / App 产物 ✓
```
```

Server 因此也不需要把 Terminal PTY 流广播给整个 Room，只在 `device ↔ open-tutti-server ↔ 当前设备的 Tutti Desktop` 这条私有链路上处理。Terminal 本身不会进入 Big @ 的可引用会话对象；但它产生的文件、服务、产物仍然可以被引用 (｡•̎̎)✧

下一处需要你决定的是 \*\*Room 邀请权限\*\*：

已经加入 Room 的普通参与者，是否也可以看到并复制：

`https://server/share/...` + `6 位房间密码`

然后继续邀请其他人加入？

还是只有 \*\*房主\*\* 可以看到/分享邀请信息，普通参与者不能继续拉人进来？ (≥▽≤)♡

#177 用户

只有 \*\*房主\*\* 可以看到/分享邀请信息，普通参与者不能继续拉人进来

#178 助手

锁定: \*\*只有房主能查看分享链接与 6 位房间密码, 也只有房主能继续邀请新成员\*\*; 普通参与者加入后不获得传播入口 (｡• ̳•) ♡

我再对照一下 Tutti 当前对 Room Owner / participant 权限的公开描述, 把能直接模仿的管理权限一起收拢掉。

#179 助手

这条也和 Tutti 当前公开权限模型一致: 官方条款明确写着 \*\*Room creator/host 管理成员邀请和访问权限\*\*。所以我会直接把“查看邀请、轮换房间密码、撤销 Share Link、移除成员、转让房主、解散 Room”都归房主, 不再逐项问你 (˘> ˘<) ♡

我接下来查一个更大的兼容边界: Tutti 现在宣传里还包含内置的 \*\*Tutti Agent\*\*, 要确认开源仓库里它到底是可复用 Provider 还是依赖官方托管服务。

#180 助手

查到啦: 开源仓库现在已经把 \*\*Tutti Agent\*\* 当成正式 Provider 收口进统一 Agent 基建, 而且是“专属协议族”, 并不是 OpenCode/ACP 那种纯本地 Provider。Claude Code、Codex、OpenCode、Cursor、Tutti Agent 都已经通过同一套 descriptor / strategy / capability seam 接入, 所以我们应该保留这条 Provider seam, 而不是在 Fork 里删掉。

我再查一下它的服务端地址和认证是不是硬依赖 Tutti 官方托管, 这会决定 Self Hosted 默认行为。

#181 助手

这条锁定后, Room 权限可以直接收敛成: \*\*房主负责成员与访问控制, 普通参与者只负责自己的协作行为和 Agent 分享\*\*。这也符合 Tutti 当前公开条款里“Room creator/host manages member invitations and access permissions”的定义。([tutti.sh])(https://tutti.sh/en/terms?utm\_source=chatgpt.com))

另外刚查到一个值得直接保留的上游能力: 最新版仓库的 Agent 基建已经把 \*\*Tutti Agent\*\* 收口成正式 Provider, 而且 Claude Code、Codex、OpenCode、Cursor、Tutti Agent 等都经过统一的 Provider Descriptor / Strategy / Capability seam; 其中 Tutti Agent 使用自己的专属协议族。([github.com])(https://github.com/tutti-os/tutti/blob/main/AGENT\_GUI\_CONTROLLER\_REFACTOR\_HANDOFF.md?utm\_source=chatgpt.com)) 官方现在也仍然宣传: 没有其他 Agent 订阅时, 可以使用 Tutti Agent, Early Access 期间免费。([github.com])(https://github.com/tutti-os/tutti?utm\_source=chatgpt.com))

所以我们不应该为了 Self Hosted 把这套 Provider 支持从 Fork 里删掉。

下一问只决定这一点:

\*\*Open Tutti 第一版是否默认继续显示并允许使用官方 Tutti Agent? \*\*

我的建议是:

```text id="6xliuw"

Tutti Agent

→ 保留上游 Provider Adapter

→ 仍连接 Tutti 官方服务

→ 不经过 open-tutti-server 代理

→ open-tutti-server 不接触模型请求/API 凭据

→ 官方服务不可达时就显示 unavailable

```

也就是说和你刚才选的 \*\*官方 Tutti App Catalog\*\* 一样: Self Hosted 的是 \*\*Room / VM / 协作基础设施\*\*, 不是强制把 Tutti 自己提供的云端 Agent 服务也重做一遍~(｡• ̳•) ♡

你希望就这样保留官方 Tutti Agent 吗?

#182 用户

不保留，隐藏掉

#183 助手

锁定: \*\*Tutti Agent 第一版完全隐藏\*\*，不作为 Open Tutti VM 的可选 Agent Provider 🙄(˘> ౧ <˘)🙄

和 `HostRuntime` 一样，我建议采用“代码保留、产品不可达”的策略：

```
```text id="oc6x5q"
Tutti Agent
|—— 上游 Provider Adapter 代码保留
|—— 相关协议代码保留
|—— 尽量保留测试，方便同步上游
|—— Desktop Agent Picker 不显示
|—— 不自动连接 Tutti Agent 官方服务
|—— 不参与 Agent Borrowing
```
```

这样 Open Tutti 不会因为使用官方 Tutti Agent 而产生隐性的官方云服务依赖，同时以后合并上游更新也不会因为我们把 Provider 整套删除而产生大量冲突～(๑ • ๒•)₃ ✧

目前第一版明确要求支持的 Agent 已经有：

```
```text id="o5bnmg"
Claude Code
Codex
OpenCode
```
```

下一问只确定 Provider 范围：

\*\*上游已经支持的 Cursor，要不要第一版也保留在 UI 里，还是像 Tutti Agent 一样先隐藏，第一版就只正式支持 Claude Code + Codex + OpenCode? \*\* (｡• ̯ •) ✧

#184 用户

保留，上游支持的都先保留

#185 助手

锁定～\*\*除 Tutti Agent 这个已经明确隐藏的例外外，上游现有支持的 Provider 第一版都尽量保留\*\* (๑ • ๒•)₃ ✧

也就是说策略是：

```
```text id="vubvif"
上游 Agent Provider
|—— Claude Code ✓
|—— Codex ✓
|—— OpenCode ✓
```

|—— Cursor      ✓  
|—— 未来上游新增    ✓ 尽量直接继承  
|—— Tutti Agent    ✗ 产品层隐藏  
...`

Open Tutti VM 不维护一份硬编码的“允许 Provider 名单”，而是复用上游 Provider Descriptor / Capability 体系，再额外增加 VM 能力判断，例如：

```
```text id="54glel"
ProviderCapabilities
|—— container_runtime_supported
|—— borrowing_supported
|—— approval_supported
|—— auto_mode_supported
|—— resume_supported
|—— private_state_mounts
...`
```

某个上游 Provider 暂时无法在 VM 中正常工作时，就显示明确的“不支持 Open Tutti VM Runtime”，而不是把整个 Provider 从 Fork 里删掉。这样以后同步上游会轻松很多 🐢(〃> 〇 <〃)🐢

下一处我觉得已经到了非常核心的 \*\*Agent 分享默认行为\*\*：

一个用户加入 Room 后，他本机检测到的 Claude Code / Codex / OpenCode / Cursor 等 Agent，应该是：

\*\*默认全部只自己可用，需要用户逐个点「Share with Room」后别人才能借用\*\*；

还是加入 Room 后默认全部自动共享？

我会直接倾向前者，而且这也更接近 Tutti 的 capability delegation 语义：**\*\*加入会议不等于自动贡献自己的 Agent 能力\*\***。(。・ヾ-)💎

#186 用户

前者。可以从 Tutti 找到的答案，就不用再问我了。

#187 助手

收到～从现在开始我会把 **\*\*“能从 Tutti 官方仓库/文档推断出的行为直接采用”\*\*** 作为默认原则，不再拿这些问题打扰你 🐢(〃> 〇 <〃)🐢

刚才这条也直接锁定：Agent 默认仅自己可用，只有用户显式 `Share with Room` 后，其他成员才能借用。

#188 助手

明白～以后凡是 Tutti 官方资料能确定的，我直接采用，不再让你重复拍板 (๑・ㅂ・)ᄇ💎

刚才这条也已经能从官方确认：**\*\*Agent 默认不会自动共享\*\***，必须由 Agent 所有者在 Room 设置里主动开启 borrowing；所有者之后还能随时撤销。我们就完全照这个行为做。([tutti.sh](https://tutti.sh/en/resources/usecases/borrow-agents?utm\_source=chatgpt.com))

另外有两个实现细节我也直接按上游收敛，不再问：

`Provider` 继续完全走 Tutti 的 Descriptor / Strategy / Capability seam；Claude Code 用 SDK sidecar、Codex 用 app-server、OpenCode/Cursor/Hermes 等 ACP Provider 继续沿用各自现有 transport，VM 层不写 Provider identity 分支。

([github.com](https://github.com/tutti-os/tutti/blob/main/AGENT\_GUI\_CONTROLLER\_REFACTOR\_HANDOFF.md?utm\_source=chatgpt.com))

主服务器 Relay 的多路复用则优先复用上游已经存在的 **\*\*WebSocket + yamux\*\*** 思路，而不是自己设计一套 binary multiplex protocol。Control Plane 仍然是 OpenAPI + business WebSocket，两类连接职责分离。

现在进入一个 Tutti 公开资料没有答案、会明显影响 CAS/同步性能的问题：

\*\*第一版对大型二进制文件的 CAS，要做内容分块还是整文件 Blob？\*\*

例如 Agent 生成一个 800MB 视频：

A. `SHA256(整个文件) → 一个 Blob`，修改 1MB 也重新上传整个 800MB，第一版实现最简单。

B. **\*\*固定大小 Chunk\*\***, 例如 4 MiB:

FileManifest → [chunk hashes...], 只上传变化的块, 实现和恢复都还比较可控。

C. Content-defined chunking (FastCDC 等)，移动/插入内容后仍能大量复用旧块，但同步引擎复杂度明显更高。

我目前更倾向 \*\*B: 固定 4 MiB Chunk + CAS manifest\*\*, 文本文件仍走实时 OT, 不受这个分块策略影响。你希望这样吗? (。•̀-̎)✧

#189 用户

B

#190 助手

锁定 \*\*B: 固定 4 MiB Chunk + CAS File Manifest\*\*~(๑・ㅂ・)₹◇

第一版大型/二进制文件就按这个模型：

```

`` `text id="43pwlt"
FileManifest
├── size
├── mode
├── content_hash
└── chunks
    ├── sha256:aaa... 4 MiB
    ├── sha256:bbb... 4 MiB
    ├── sha256:ccc... 4 MiB
    └── sha256:ddd... remainder
`` `

```

客户端先询问 Server 缺哪些 Chunk，只上传缺失对象；Chunk 上传使用幂等的 content-addressed PUT。这样设备重连、重复上传和大文件局部修改都比较容易处理，同时第一版不用承担 FastCDC 的额外复杂度 $\mathcal{O}(n)$

文本文件仍然完全走我们已经确定的：

```
``text id="rhmiijl"
实时 OT / File Operation Stream
↓
周期性 Snapshot
```

↓
CAS
```

而不是每次敲几行代码就上传一组 4 MiB Chunk。

下一处 Tutti 公开资料无法替我们决定，而且对 Server 内存/SQLite/CAS 增长影响比较大：

****Workspace 的实时 Operation Log 要保留历史多久？****

我建议第一版采用 ****Checkpoint Compaction****：

`` `text id="t4hakf"  
实时 operations  
↓  
达到一定数量 / 时间  
↓  
生成 Workspace Snapshot  
↓  
确认所有在线 Replica 已追到 Snapshot  
↓  
删除 Snapshot 之前的旧 operation log  
```

也就是说 **\*\*不提供完整的“会议时间旅行/无限撤销历史”\*\***，Operation Log 只是同步机制；历史落到 Snapshot 后就可以压缩掉。Room 解散则全部释放。

你希望第一版就采用这种 **\*\*只保证当前状态、不长期保存完整操作历史\*\*** 的方式吗？(。•̀-)-◇

#191 用户

可以

#192 助手

锁定：Operation Log 只用于实时同步，不做永久时间旅行历史；通过 Snapshot/Checkpoint 定期压缩，确认在线 Replica 已经越过检查点后即可丢弃旧操作，Room 解散时再整体释放 ♪(↪ ▢ <~)♪

接下来我想把一个真正会影响一致性和性能的点定下来：**\*\*Server 对实时文本操作到底要不要“落盘后再 ACK”\*\***。因为我们已经明确 Server 崩溃就结束 Room，不恢复，所以这里可以换取不少性能。

#193 助手

下一问只定这个：

实时文本操作到达 `open-tutti-server` 后，你希望 ACK 策略是哪种？

- A. **\*\*每个 Operation 落 SQLite/WAL 后才 ACK\*\***：最稳，但高频 OT 会增加磁盘写入压力。
- B. **\*\*内存排序 + 小批量落盘\*\***：Server 先完成权威排序并广播，例如每 50ms / N 条批量写 WAL；性能和故障窗口比较平衡。
- C. **\*\*Operation Log 主要放内存，只在 Snapshot 时落盘\*\***：最简单最快；反正 Server 崩溃时 Room 本来就结束，最终兜底依赖房主的 Full Replica。

按我们目前的设计，我会倾向 \*\*C\*\*：SQLite 保存 Room/成员/元数据，实时 OT Sequencer 保持在内存，Snapshot 才进入 CAS/SQLite。这样 Server 真正保持轻量 (。・ヾ-)◇

你选 \*\*A、B 还是 C\*\*？

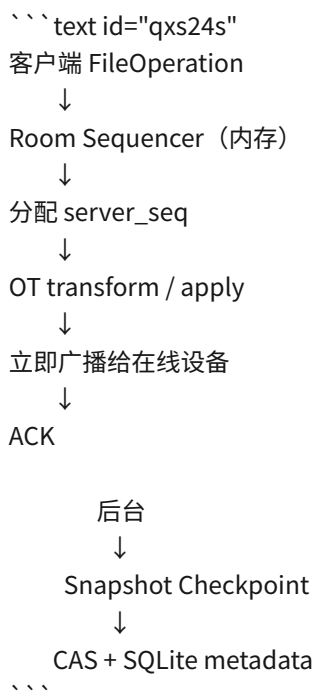
#194 用户

C

#195 助手

锁定 \*\*C\*\*：实时 Operation Log 主要驻留内存，Snapshot 才持久化\*\*~(๑・ㅂ・)ᵝ◇

因此 `open-tutti-server` 的 Workspace 热路径会非常轻：



也就是说我们明确接受：

```
```\text id="buy0bk"
Server crash
→ 未进入 Snapshot 的 Operation Log 丢失
→ Room 结束
→ 不恢复
→ 房主依赖 Full Replica 自动 Apply to Workspace
```\
```

SQLite 因此不承担高频 OT 写入，只保存这类相对低频的数据：

```
```\text id="v19zb6"
Room metadata
Membership
Share information
Device metadata
```



Snapshot metadata  
CAS references  
Context metadata  
Task / Conversation metadata  
```\n

而 `server\_seq`、OT transform window、近期操作、在线 Presence 等全部是 Room 级内存状态。

Snapshot 我会直接采用\*\*多条件触发\*\*，不需要你再为某个魔法数字拍板：

```text id="x1870n"  
定时 checkpoint  
或  
Operation 数量达到阈值  
或  
累计 dirty bytes 达到阈值  
或  
新设备加入，需要稳定 bootstrap point  
或  
Owner Transfer 前  
```\n

而房主的 Full Replica 始终比 Snapshot 更接近最新状态，所以它才是 Server 故障后的最终成果兜底～ㄟ(ˋ> ㄟ<)ㄟ

接下来有一个比较大的架构问题需要你决定：

\*\*Room 的 Group Chat / Agent Conversation / Task Context，也采用和 Workspace 一样的“Room 解散即全部销毁”语义吗？\*\*

也就是说服务器在会议结束后：

```text id="vtcfnx"  
Room  
├── Chat messages  
├── Agent conversations  
├── Tasks  
├── Big @ references  
├── Artifacts metadata  
└── Workspace CAS  
  
全部删除 / GC  
```\n

全部删除 / GC
```\n

最终只有房主通过 **Apply to Workspace** 留下文件成果，聊天和 Agent 历史本身不会永久保存到 Self Hosted Server。

还是你希望 Apply 时同时把这些协作历史以某种形式保存到宿主机 Workspace? (｡• ̯ •)✧

#### #196 用户

聊天和 Agent 会按照会话归类到仓库的 .opentutivm/historys/ 里，在协作开始时允许恢复历史数据或者开始全新

#### #197 助手

这条很关键，我会把历史从“Server 临时状态”改成 ****Workspace 的正式可移植数据****：Room 解散后服务器仍然全部销毁，但聊天与 Agent 会话已经被写入 `.opentuttivm/historys/`，因此会随着最终 Workspace 一起 Apply 回仓库 (๑ • ๓) ♡

我先对照 Tutti 当前对 Session/Conversation 持久化的公开边界，避免把 Provider 私有状态误写进共享仓库。

#### #198 助手

锁定：****聊天与 Agent 历史属于 Workspace 的持久成果****，统一写入仓库的：

```
```text id="clbr2k"
.opentuttivm/historys/
```
```

Room Server 本身仍然是临时的：会议结束后 Server 删除 Room、Chat、Task、Conversation、CAS 引用；真正需要跨会议保留的历史已经随着 Workspace 一起 Apply 回宿主主机啦~(๑ • ๓) ♡

这和 Tutti 上游现有思路也比较吻合：Tutti 已经有 durable Agent Session / message history，并区分“历史可查看”和“Provider 原生 Session 是否还能 Resume”；导入的历史即使不能原生 Resume，也应该保持可查看，并通过新的 continuation/handoff 继续，而不是假装原 Provider Session 仍然存在。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/conventions/troubleshooting/agent-session-lifecycle.md?utm_source=chatgpt.com))

我建议第一版直接定成这样的持久格式：

```
```text id="qn63kr"
.opentuttivm/
├── historys/
│   ├── <collaboration-session-id>/
│   │   ├── manifest.json
│   │   ├── group-chat.jsonl
│   │   ├── tasks.jsonl
│   │   └── agents/
│   │       ├── <agent-session-id>/
│   │       │   ├── session.json
│   │       │   └── messages.jsonl
│   │       └── ...
└── ...
```
```

其中一个 `collaboration-session-id` 就代表“一次会议/协作”。

`manifest.json` 可以记录：

```
```text id="r152he"
开始时间 / 结束时间
原房主设备名称
参与设备
Workspace revision
Agent session 列表
环境 generation
可选标题 / 摘要
```
```

Agent History 保存的是 ****Open Tutti 的 provider-neutral semantic history****：

```
```text id="u943g7"
```

User messages

Assistant messages

Turns

Tasks / Goals

Tool 调用的可共享投影

Artifacts / Workspace references

Big @ references

Provider / model metadata

```
```
```

但这些东西绝不能放进去：

```
```text id="h5bmd2"
```

Provider credentials

OAuth token

MCP secrets

Private Capability Plane 内容

Provider runtime private snapshot

设备本地路径

```
```
```

Provider-native 的 `provider\_session\_id` / runtime state 继续放设备 Private State。Tutti 上游本身也说明 Resume 依赖 provider-local runtime state，历史记录存在并不等于该 Provider Session 在另一台设备上还能真正 resume。  
([github.com](https://github.com/tutti-os/tutti/blob/main/docs/conventions/troubleshooting/agent-session-lifecycle.md?utm_source=chatgpt.com))

所以创建下一次 Room 时：

```
```text id="gils44"
```

选择 Host Workspace

↓

发现 .opentuttivm/historys/

↓

Restore History

或

Start Fresh

```
```
```

选择 **Start Fresh**：

```
```text id="fw0z4j"
```

旧 historys 仍保留在仓库

但不载入新 Room Context

```
```
```

不是删除历史。

选择 **Restore History**:

```
```text id="sx1u15"
.opentuttivm/history/<...>
```

↓

恢复 Group Chat
恢复历史 Agent Conversations
恢复 Tasks / Big @ Context

↓

成为新 Room 的初始 Context Graph
```

如果某个历史 Agent Session 恰好是在**同一台设备**创建，而且 Provider 私有 Resume State 仍然存在：

```
```text id="02egxk"
```

History

+

Device Private Provider State

↓

Native Resume ✓

```
```
```

否则：

```
```text id="ij5va2"
```

History

↓

显示完整历史

↓

需要继续时创建新的 Provider Session

↓

以历史 Context 做 handoff / continuation
```

这样 Anna 创建的 Claude 历史到了 Leo 电脑上仍然能读、能`@`、能交给 Leo 的 Codex 接着处理，但绝不会把 Anna 的 Claude 登录状态一起复制过去 🙅(˃ ˃ˊ)🙅

下一处 Tutti 无法替我们决定的是：**发现多个历史 collaboration session 时，Restore History 是只允许选择其中一个历史会议恢复，还是允许一次勾选多个历史会议一起载入 Context？** 我会偏向第一版**只选一个**，Context 会更可控。

#### #199 用户

第一个

#### #200 助手

锁定：**恢复历史时一次只允许选择一个历史协作会话**~(๑•ㅏ•)✧

例如：

```
```text id="mxwvzu"
```

```
.opentuttivm/historys/
|—— 2026-08-20-room-abc/
|—— 2026-08-25-room-def/
|—— 2026-08-27-room-xyz/
```
```

创建新 Room 时只能：

```
` ``text id="8vseen"
Restore History
→ 选择其中一个 collaboration session
` ``
```

其他历史仍留在仓库里，不删除、不合并进当前 Context。新一轮协作结束后再生成新的 history session，因此历史天然形成一系列独立会议记录 📄(↗️ □ <~)📄

下一处我觉得需要你定的是 **恢复历史时 Workspace 文件的语义**：

选择某个历史 Session 后，是否只是恢复它的 **Chat / Agent Conversations / Tasks / Big @ Context**，而 Workspace 文件仍然始终以创建 Room 时当前 Host Workspace 为准？

我倾向 **是**：历史记录只是上下文，不负责把代码回滚到当时的版本。否则“恢复一次旧会议”很容易意外覆盖当前代码 (｡• ̯ •)💎

## #201 用户

是

## #202 助手

锁定： **Restore History 只恢复上下文，不恢复/回滚 Workspace 文件** ~ (｡• ̯ •)💎

也就是说新 Room 的初始状态始终是：

```
` ``text id="fv9wsi"
当前 Host Workspace
+
选中的一个历史协作 Session
↓
新的 Room
` ``
```

历史只恢复：

```
` ``text id="8bd1rr"
Group Chat
Agent Conversations
Tasks
Big @ references
Artifacts metadata
历史参与者/Agent 信息
` ``
```

而源码、配置、`.git` 等始终以**当前宿主机 Workspace** 为准。这样即使恢复几个月前的会议，也只是告诉 Agent “之前发生过什么”，不会偷偷把代码退回旧版本 🙄👉👈🙄

下一处需要你决定，而且涉及隐私：

`.opentuttivm/historys/` **默认是否应该进入 Git? **

也就是：

**A. 正常文件**：默认可以被 `git add/commit`，团队可以把 AI/群聊历史跟仓库一起版本管理。

**B. 默认加入 `.gitignore`**：历史留在 Workspace 和本机，但默认不提交，因为里面可能包含聊天内容、Prompt、工具调用记录等敏感信息。

我会更倾向 **B**。

#### #203 用户

当保存 `.opentuttivm/historys/` 后，会询问房主，要不要进入 Git，如果选择不进入，就加一条

```

Chat History Ignored by OpenTuttiVM

```

#### #204 助手

锁定～保存 `.opentuttivm/historys/` 后，由房主决定是否纳入 Git (๑ • ๒ • ๓) ✧

如果选择“不进入 Git”，Open Tutti VM 自动在仓库 `.gitignore` 末尾追加：

```gitignore id="63dijm"

Chat History Ignored by OpenTuttiVM

.opentuttivm/historys/

```

如果仓库已经有这条规则，就不重复追加；如果选择“进入 Git”，则不修改 `.gitignore`。

同时这个询问只在**首次产生 historys 且当前仓库尚未明确配置该路径**时出现，避免每次会议结束都弹一次。之后用户仍可手动修改 `.gitignore` 改变行为 🙄👉👈🙄

下一问只定历史内容的体积边界：

Agent 的 Tool Call 如果产生很大的 stdout/stderr，例如测试输出几十 MB，写入 `.opentuttivm/historys/` 时是要**完整保存**，还是只保存摘要/截断后的输出，并把真正有价值的产物放 Workspace/Artifacts？我倾向后者。

#### #205 用户

tutti 官方应该有想法，不要问我了

#### #206 助手

明白，后面像这种“历史里该存多少工具输出”的细节，我会先查 Tutti 官方实现/约定，能推断就直接按它来，不再问你啦 (๑ • ๒ • ๓) ✧

有，Tutti 官方实现已经能看出比较明确的取向，所以这一题我们直接按它来，不再让你选啦～(๑ • ㅍ • )ง ✧

Tutti 的 canonical Agent History 会持久化 **最终的**工具调用结果/命令输出**。流式执行过程中可能产生大量重复的 stdout/tool-output snapshot，但上游会把同一消息的 pending snapshot 合并，只保留最新的累计输出，而不是把每一个中间 snapshot 都塞进历史；同时，大历史在 UI 侧采用分页/有界加载，而不是通过截断历史内容来解决性能问题。  
([github.com])(https://github.com/tutti-os/tutti/blob/main/docs/conventions/troubleshooting/agent-session-lifecycle.md?utm_source=chatgpt.com))

所以 `opentuttivm/historys/` 我们直接定成：

```
```text id="249m25"
```

执行过程中：

```
stdout: "Installing..."
stdout: "Installing...\n10%"
stdout: "Installing...\n10%\n20%"
stdout: "Installing...\n10%\n20%\nDone"
```

↓ coalesce

History 最终保存：

```
"Installing...\n10%\n20%\nDone"
```
```

也就是说，**不设置“超过 X KB 就截断 stdout”这种人为历史限制**。几十 MB 的文本输出原则上可以完整成为 canonical history；真正打开历史时分页、lazy load，避免一次塞进 Renderer。

不过有两个 Tutti 已经表现出来的安全/可移植原则，我们也直接继承。

第一，历史保存的是 **semantic history**，不是 Runtime Dump。上游在可移植状态里会移除诸如绝对 runtime CWD 等设备特有信息，而且特别避免把隐藏 thinking / tool payload 等非用户可见数据错误混入导出的会话历史。  
([github.com])(https://github.com/tutti-os/tutti/blob/main/docs/conventions/troubleshooting/agent-session-lifecycle.md?utm_source=chatgpt.com))

因此我们会做：

```
```text id="jx9tdk"
```

保存：

- ✓ command
- ✓ stdout / stderr
- ✓ tool result
- ✓ file change summary
- ✓ user-visible reasoning/text
- ✓ MCP/tool execution result
- ✓ exit code
- ✓ duration
- ✓ Workspace 相对路径

清洗：

- ✗ Host absolute path
- ✗ Provider credentials

✗ env secrets
✗ OAuth/API token
✗ Private Capability Plane paths
✗ provider-internal hidden payload
````

第二，**大的二进制内容不直接塞 JSONL**。Tutti 最近针对诊断导出的设计也明确把 image base64 等重内容剥离，并对 tool result 中 credentials/env 做脱敏。([github.com](https://github.com/tutti-os/tutti/issues/1000?utm_source=chatgpt.com))

所以我们自己的历史采用：

```
``text id="my0p3d"
messages.jsonl
```

```
ToolResult {
 output: "...text..."
}
```

```
ArtifactResult {
 artifact_ref: "...
 path: "output/video.mp4"
 hash: "sha256:..."
}
````
```

而不是：

```
``text id="o5gzib"
messages.jsonl
→ 800 MB base64 video ✗
````
```

这会让我们之前设计的：

```
`Workspace/CAS = 大型实体数据`
`.opentuttivm/historys = 语义历史与引用`
```

边界非常清晰 🤖(↔️ □ <=>)🤖

另外，上游已经明确强调历史 Session 应该通过**分页和针对性读取**处理，而不是把整个历史列表/所有 messages 一次性 hydrate；否则 AgentGUI 会在大历史下冻结。我们直接照这个经验做 `.opentuttivm/historys/` 的 Restore：先读 manifest/session index，用户真正展开某段 Conversation 时才加载对应 JSONL 段。([github.com](https://github.com/tutti-os/tutti/blob/main/docs/conventions/troubleshooting/agent-session-lifecycle.md?utm_source=chatgpt.com))

因此这一块不用再讨论，正式定为：

> **完整保存 canonical semantic history；流式 snapshot 合并；大历史分页加载；二进制改用 Artifact/CAS 引用；敏感/设备特有 runtime 信息在写入 `.opentuttivm/historys/` 前清洗。 **

接下来我会继续优先从 Tutti 仓库找答案，不再问这种已经有官方端倪的实现细节啦～(≥▽≤)♡



## #208 用户

看你没有要问的了。接下来，只能最多再问 5 个问题。挑重点问。

## #209 助手

那我只挑真正会决定实现成败的“大问题”来问，最多 5 个；能从 Tutti/上游推断出的都不占这个额度 (๑ • ๒ • ๓) ♡

****第 1 个问题：Private Capability Plane 的凭据隔离，是否允许我们做「Credential Broker / Proxy」而不是把原始凭据文件直接暴露给 Agent Session？****

原因是 Claude Code / Codex / OpenCode 的“Agent 主进程”和它执行的 shell/tool 往往处在同一个容器权限域里。单靠 `chmod` 很难做到“Provider 能用 Token，但 Agent 执行 `cat ~/.config/...` 却读不到”。

我建议最终安全结构是：

```
```text
Agent Session Container
├── Agent / shell / tools
│   └── 看不到原始 OAuth/API/SSH secrets
└── Unix socket / local broker API
    ↓
open-tutti-capability-broker
    ↓
Host Private State
├── provider credentials
├── MCP credentials
├── SSH
└── Git credentials
...```
```

能通过原生 socket/agent 转发的就转发；无法 broker 的 Provider，再退化到受限私有挂载。

\*\*你接受“Broker 优先、受限挂载兜底”作为第一版安全原则吗？\*\* 啊(˘> ౧ <˘)啊

#210 用户

好

#211 助手

第 1 个重点问题锁定：**\*\*Private Capability Plane 采用 Broker 优先、受限私有挂载兜底\*\***。这样原始 Token/SSH/MCP Secret 尽量不进入 Agent 可直接读取的文件系统，只把“可用能力”暴露给 Session (๑ • ๒ • ๓) ♡

我还剩最多 4 个问题。第 2 个我想问的是会直接影响我们以后能不能持续吃上游更新的 Fork 策略。

#212 助手

第 2 个重点问题：**\*\*Fork 是否要明确以“长期可跟随上游 Tutti 更新”为目标？\*\*** Tutti 当前是 Apache-2.0，仓库本身也欢迎 Fork；如果我们要持续吃上游的 Agent Provider、UI、App、Bugfix，我建议把“低侵入 Fork”当硬约束。
([github.com])([https://github.com/tutti-os/tutti?utm\\_source=chatgpt.com](https://github.com/tutti-os/tutti?utm_source=chatgpt.com))

也就是尽量采用：

上游 Tutti

...

\*\*你是否希望把“保持上游可合并性、尽量低侵入”作为正式工程原则？\*\* (。・ヾ-)◇

好

第 2 个重点问题也锁定：**\*\*长期跟随上游、低侵入 Fork\*\*** 作为硬工程原则。能通过新增 service/package、adapter、feature gate 实现的，就不去改上游核心语义；HostRuntime/Tutti Agent 以隐藏为主，不做破坏性删除 ☹️ ⚡️

#215 助手

\*\*第3个重点问题：MCP / Skills / Tools 的实际执行位置。\*\*

我建议采用：

Agent Session Container

↓

Capability Broker

↓

Owner Device / tuttid

↓

MCP Server / Skill / Tool

...

也就是：

- Agent Session 能“调用”所有者已经配置好的 MCP / Skills / Tools;
- MCP Secret、OAuth、SSH Key 等继续留在所有者设备的 Private Capability Plane;
- 对适合直接放进容器的纯文件型 Skill，可以以只读方式注入;
- 对 MCP、需要宿主机网络/认证/本地能力的 Tool，优先由 `tuttid` / Capability Broker 代理执行;
- Borrower 只能看到调用能力和结果，不能读取底层 Secret。

这样比“把所有 MCP 配置和凭据复制进每一个 Session Container”安全很多，也最符合我们前面定下来的 Broker-first 原则。

\*\*你是否同意：MCP/Tool 默认由 Owner Device 的 Capability Broker 提供，纯文件型 Skills 才允许只读注入 Session Container? \*\* (｡•̯•)✧

#216 用户

可以

#217 助手

第 3 个重点问题锁定：\*\*MCP / Tool 默认通过 Owner Device 的 Capability Broker 执行；纯文件型 Skills 才以只读方式注入 Agent Session Container\*\*。这样 Borrower 能用能力，但拿不到底层 Secret (๑ • ๒•)₹

还剩最多 2 个问题。第 4 个我挑 Runtime 编排这一条，因为它会决定 `tuttid` 最终是“调用 Compose”还是“直接成为 Docker 控制器”。

#218 助手

\*\*第 4 个重点问题：Room 的 Docker 编排是否允许 `tuttid` 直接调用 Docker Engine API? \*\*

我建议：\*\*允许，而且不要在运行时依赖 `docker compose` CLI。 \*\*

虽然逻辑上仍然严格保持：

`一个 Room = 一个 Compose Project = open-tutti-vm-<roomId>`

但真正实现由 `tuttid` 使用 Docker Engine API 动态创建：

```
```text id="erjram"
Project labels
com.docker.compose.project=open-tutti-vm-<roomId>

Resources
|—— Network
|—— open-tutti-vm-<roomId>-workspace
|—— room-sync
|—— agent-session-*
|—— terminal-session-*
```
```

这样创建/销毁 Agent Session、Terminal、Rebuild、端口发现都会比不断生成 Compose YAML 再 shell out 稳定很多；同时资源仍打标准 Compose labels，Docker Desktop 里看起来仍是一个 `open-tutti-vm-<roomId>` Project。

`docker-compose.yml` 只用于 \*\*Self Hosted Server 部署\*\*，不用于 Desktop Room Runtime 编排。

你是否同意这个方向？ (๑ • ๒-)₹

#219 用户

可以

#220 助手

第 4 个重点问题锁定：\*\*Desktop Room Runtime 由 `tuttid` 直接通过 Docker Engine API 编排，不依赖 `docker compose` CLI\*\*；但统一打 Compose project labels，所以 Docker Desktop 里仍按 `open-tutti-vm-<roomId>` 聚合显示 (๑ • ๒•)₹

还剩最后 \*\*1 个问题\*\*。我把它留给服务器的长期身份/创建权限模型，因为这会决定 Device Identity 和 Server Invite Code 到底怎么落库。

#221 助手

\*\*第 5 个，也是最后一个问题：服务器邀请码是“一次授权设备”，还是“每次创建 Room 都要输入”？\*\* (｡•̀-｡)✧

我建议采用\*\*一次授权设备\*\*：

```
```text id="4g43qb"
```

第一次连接某个 Server

Device Identity (Ed25519)

↓

输入 Server Invite Code

↓

Server 验证

↓

给该 Device 授予：

can_create_room = true

↓

以后创建 Room 不再输入邀请码

```
```
```

没有获得这个权限的设备仍然可以：

```
```text id="24fw0y"
```

✓ 通过 /share/... + Room Password 加入别人房间

✗ 自己创建 Room

```
```
```

管理员以后可以在 Server 侧撤销某个 Device 的 `can\_create\_room`。如果修改 `SERVER\_INVITE\_CODE`，已经授权的设备不会自动失效，新设备使用新邀请码。

这样 Server Invite Code 更像“Self Hosted Server 的成员准入码”，而不是每次开会都重新输入的密码。

\*\*你希望采用这种“一次输入，长期授予该设备创建 Room 权限”的模型吗？\*\* ฅ(̑̑> ̑̑<̑̑)ฅ

#222 用户

server 侧只有 .env 无法撤销，所以改为，服务器列表同时记住服务器地址和邀请码，如果不保存邀请码则在创建房间时输入

#223 助手

锁定最后一处：\*\*Server Invite Code 不变成服务器端长期授权\*\*。Server 只读 `.env`；Desktop 的 Server 列表可以本地保存 `server URL + invite code`，创建 Room 时随请求提交；若没保存，就现场输入。这样不需要额外的管理员后台或撤销体系 ฅ(̑̑> ̑̑<̑̑)ฅ

5 个重点问题已经问完啦，我把整套脑暴结果收束成一份实现画布给你确认～

#224 助手

好~5 个重点问题已经全部问完，现在进入你定义的脑暴收束阶段啦 ٩(̋̄>̋̄<)۶

下面就是目前完整的 **\*\*Open Tutti VM v1 脑暴画布\*\***。后续除非实现时发现硬性技术障碍，否则我会把这些都视为已经确定的设计，不再反复问你 (๑ • ๒ •)₉ ✧

Open Tutti VM v1

产品目标非常明确：

> Fork Tutti Open Source，尽量低侵入地复刻 Tutti VM 的协作体验，但 Room / Workspace / Relay / CAS / 调度基础设施全部可以 Self Hosted。

Agent 不跑在 Server。

Agent 永远运行在 Agent 所有者自己的 Docker Desktop 中。

```
``text id="s8x009"
    open-tutti-server
    Source of Truth
        |
        +-----+-----+-----+-----+
        |         |         |         |
    Anna   Leo   Bob
tuttid   tuttid tuttid
        |         |         |
Docker Desktop Docker Desktop Docker Desktop
        |         |         |
Agent Sessions Agent Sessions Agent Sessions
````
```

Server 不做 P2P，也不承担真正的 Workspace Linux 文件系统。

— — —

## ## 1. Fork 与技术栈

继续维护一个长期可跟随上游的低侵入 Fork。

```

` ``text id="gilyfw"
Desktop Electron / TypeScript / React
tuttid Go
open-tutti-server Go
open-tutti-room-sync Go
open-tutti-fs Go + hanwen/go-fuse/v2
Capability Broker Go
Database SQLite v1 → Repository abstraction → PostgreSQL future
CAS Local filesystem v1
` ``

```

、HostRuntime` 保留源码和测试，但产品 UI 完全不可达。

‘Tutti Agent’ 同样保留上游代码但隐藏。

其他上游已经支持、以后新支持的 Agent Provider 尽量原样继承。

---

## ## 2. Docker Runtime

第一版支持三个发行版本：

```
```text id="yhmt68"
```

Windows x86\_64

macOS Intel

macOS Apple Silicon

```
```
```

要求 Docker Desktop。

支持 Docker VMM 的平台默认推荐 Docker VMM，但 **绝不修改用户现有 Docker Desktop 设置，也绝不偷偷重启 Docker Desktop**。

`tuttid` 只执行 Runtime Preflight。

Room 在概念上仍然是：

```
```text id="u6uw9s"
```

open-tutti-vm-<roomId>

```
```
```

Workspace Volume：

```
```text id="mgv1rx"
```

open-tutti-vm-<roomId>-workspace

```
```
```

但 Desktop 不依赖 `docker compose` CLI。

`tuttid` 直接调用 Docker Engine API 动态创建 Network、Volume、room-sync、Agent Session 和 Terminal Session，并附加标准 Compose project labels。

Server 部署才使用：

```
```bash id="tlkg5w"
```

cp .env.example .env

docker compose up -d

```
```
```

而且 Compose 里只有裸 `open-tutti-server`，不自带 Caddy/Nginx/Traefik。

---

## ## 3. Project Environment

环境解析顺序：

```
```text id="wy08p7"
.opentuttivm/Dockerfile
```

↓ 不存在

```
.devcontainer/devcontainer.json
```

↓ 不存在

```
ubuntu:latest
```
```

`devcontainer.json` 第一版只看：

```
```text id="b21h08"
image
```

```
build.dockerfile
build.context
build.args
```
```

其他 Dev Container 字段全部不实现。

自定义镜像第一版只支持：

```
```text id="k56qi1"
Linux + glibc
```
```

Alpine/musl、distroless、scratch 不支持。

项目镜像构建完成后，由 Open Tutti 注入：

```
```text id="2tatl7"
open-tutti-fs
open-tutti-agentd
Open Tutti CA
.tutti network runtime
provider runtime bootstrap
```
```

环境定义变化后：

```
```text id="iwmue8"
Environment changed — Rebuild
```
```

不会自动重建。

用户点击 Rebuild 后，立即 Interrupt 本设备该 Room 所有正在运行的 Agent，销毁旧 Runtime → Rebuild → Resume 原 Provider Session → 对刚刚确实因为 Rebuild 被中断的 Agent 自动发送：

```
```text id="5xvont"
continue
```
```

---

## ## 4. Workspace 协作文件系统

不再使用我们最开始讨论的“Revision 三方合并作为主同步”。

主模型改为：

```
```text id="ceemga"
实时 File Operation Stream
  +
Server Authoritative Sequencer
  +
Snapshot / CAS
```
```

每个 Agent Session 都看到：

```
```text id="rk932e"
/workspace
  ↓
open-tutti-fs
  ↓
room-sync
  ↓
open-tutti-server
```
```

Agent **不能直接挂载 backing Named Volume**，否则能绕过 FUSE。

文本文件：

```
```text id="js6k17"
Server-sequenced OT / patch operations
```
```

二进制/大文件：

```
```text id="ppijjn"
4 MiB fixed chunks
+
SHA-256 CAS
+
File Manifest
```
```

目录层：



```
```text id="iybrvn"
mkdir
rename
unlink
metadata
```
```

统一经过 Server sequencing。

Operation Log 主要存在 Server 内存，不要求每次落 SQLite。

Snapshot 时才：

```
```text id="umwfvn"
CAS + SQLite metadata
```
```

Checkpoint 后压缩旧 Operation Log，不提供无限时间旅行历史。

---

## ## 5. Server 故障模型

Server 重启/崩溃：

```
```text id="565c1i"
Room 直接结束
```
```

不恢复 Room，不恢复 Operation Log，不做 P2P 补救。

普通参与者直接结束。

房主维护 Full Replica，因此自动执行：

```
```text id="gna59o"
Apply to Workspace
```
```

允许使用自己最后同步成功的状态。

我们明确接受：Server 极端崩溃瞬间，不同设备最后看到的 seq 可能略有差异。

---

## ## 6. Host Workspace

创建 Room 时允许直接选择：

```
```text id="79140p"
~/Projects/my-app
```
```

Import 后立即脱离 Host:

```
```text id="x15sws"
Host Workspace
  ↓ snapshot/import
Room Workspace
```
```

绝不 bind mount。

同一个 Host Workspace Path 同时只能绑定一个活跃 Room。

Room 最终写回动作名称统一叫:

```
```text id="a3ajbe"
Apply to Workspace
```
```

它不是 merge, 而是**镜像式直接覆盖**:

```
```text id="ocae6l"
Host Final State == Room Final State
```
```

Room 删除了文件 → Host 同样删除。

第一版不处理会议过程中用户又在 Host Workspace 外部修改代码导致的 merge。

房主退出时 Apply 自动执行, 不再二次询问。

---

## ## 7. Git

Import 时复制 `.git`, 让 Room 内所有 Agent 看到同一个 Git 状态。

但首先安全清洗 Credential、危险 Hooks、外部 Path、`core.sshCommand`、credential 配置等内容。

Git/SSH 身份本身属于 Private Capability Plane, 不进入共享 Workspace。

Linked Worktree 需要 materialize 成 Room 内独立可用的 Git Repository。

---

## ## 8. Room 生命周期与身份

没有传统用户账号。

****一台设备 = 一个用户。****

设备拥有长期 Ed25519 Device Identity, 并允许在 Settings 修改 Display Name。

Room 是会议:

```text id="s2sggm"

Create



Collaboration



所有人退出



Destroy

```

创建者是 Owner。

Owner 主动退出前必须：

```text id="8bgk69"

自动 Apply to Workspace



Disband

或

Transfer Ownership

```

Owner 意外掉线：

```text id="5qoaur"

5 分钟 Grace Period

```

5 分钟内回来 → 恢复。

超时且还有人在线 → 转让给连续在线时间最长的参与者。

无人在线 → Room 直接解散。

Owner Transfer 必须完成：

```text id="p5ov1o"

新 Owner Lazy Replica → Full Replica



选择新的 Host Workspace



初始化 Host Workspace



Server commit ownership transfer

```

否则转让不成立。

---

## 9. 邀请系统

Server `.env` 可以可选配置：

```
```dotenv id="tw8y9p"
OPEN_TUTTI_SERVER_INVITE_CODE=
```
```

它只控制：

```
```text id="b0ohkt"
是否允许创建 Room
```
```

不控制加入 Room。

Desktop 支持保存多个 Server：

```
```text id="c237pw"
Company
Home
Friend
...
```
```

每个 Server Entry 可以记住：

```
```text id="o6czt9"
Server URL
Server Invite Code (可选)
```
```

逻辑上属于 Server Entry，敏感值实际可放 OS Secure Credential Storage。

如果保存了邀请码，创建 Room 时自动提交。

没保存则：

```
```text id="4yvdsq"
Create Room
→ 输入 Server Invite Code
```
```

Server 每次创建都直接对 `.env` 当前值验证，不存在服务器端长期授权。

如果 Server 没配置邀请码，则无需输入。

Room 创建时自动生成可修改的：

```
```text id="bjvg26"
6 位数字 Room Password
```
```

只有 Owner 能看到：

```
```text id="az413q"
Share URL
Room Password
```
```

只有 Owner 能邀请新人、修改密码、撤销分享、踢人、转让 Owner、解散 Room。

加入：

```
```text id="93bo9r"
https://server/share/<share-id>
```
```

打开极简 H5 → 输入房间密码 → Server 发一次性 Join Ticket →：

```
```text id="9qtut7"
open-tutti://join?...
```
```

唤起 Desktop。

Room Password 永远不放 URL/Deep Link。

---

## ## 10. Agent Borrowing

加入 Room **不会自动共享 Agent**。

用户必须主动：

```
```text id="0tghhs"
Share with Room
```
```

才允许别人借用。

同一个 Agent 可以被多人同时借用，不设 Open Tutti 人工并发上限。

每一次 Borrow Session 都创建独立 Container：

```
```text id="kr66ec"
Claude Borrow A → agent-session-a
Claude Borrow B → agent-session-b
Codex      → agent-session-c
```
```

它们拥有独立 PID、tmp、Session State、端口，但共享同一个逻辑 `/workspace`。

Borrower 是 Session Operator。

因此：

```text id="0t7syt"

Approval → Borrower 决定

Permission Mode → Borrower 决定

```

Borrower 可以自由切换 Session 级权限模式，而不会修改 Agent 所有者的全局 Provider 设置。

必须兼容：

```text id="d1l2s5"

Claude Code Auto Mode

Codex Auto/Approval Mode

OpenCode permissions / auto

以及上游其他 Provider capability

```

Borrower 主动 Leave → Interrupt 自己的 Borrowed Sessions。

Borrower 意外掉线 → 5 分钟 Operator Grace Period。

Agent 在无需 Approval 时可以继续工作，需要 Approval 时暂停；5 分钟未回来后 Interrupt。

---

## ## 11. Private Capability Plane

最终安全原则：

```text id="iwlidx"

Broker First

Restricted Mount Fallback

```

也就是说优先：

```text id="1rcwrn"

Agent Session

↓

Unix Socket / Broker API

↓

Capability Broker

↓

Owner private credentials / MCP / SSH / Git

```

尽量不让 Agent Shell 直接读取原始 Secret。

MCP / Tool 默认在 Owner Device 通过 Capability Broker 执行。

纯文件型 Skills 可以只读注入 Session Container。

确实无法 Broker 的 Provider 私有状态才使用受限 Private Mount 兜底。

Borrower 借的是：

```
```text id="9kdj0p"
Capability
```
```

而不是：

```
```text id="pa16ni"
Secret
```
```

---

## ## 12. Terminal

每个 Terminal 都是独立 Session Container。

```
```text id="ucuzan"
terminal-session-a
terminal-session-b
...
```
```

使用相同 Project Image、`/workspace`、`.tutti` 网络。

但不继承 Agent Provider Credential。

Terminal 本身完全 Private：

```
```text id="9po8bj"
别人不能看 PTY
别人不能操作
别人不能 @ Terminal
```
```

但是它产生的：

```
```text id="c1zizc"
Workspace 文件
监听服务
Artifacts
```
```

仍然属于 Room。

---

## ## 13. `.tutti` Room Network

没有 P2P。

所有跨设备数据：

```
```text id="njjnzf"
```

Device A

↓

open-tutti-server

↓

Device B

```
```
```

Control:

```
```text id="wdbe3h"
```

HTTPS / OpenAPI

Business WebSocket

```
```
```

大量双向数据可以走:

```
```text id="1v5f4l"
```

WebSocket tunnel + yamux

```
```
```

`.tutti` 只存在:

```
```text id="w9ywi1"
```

Tutti Browser

Agent/Terminal Session Containers

```
```
```

系统 Chrome、Safari、Host Terminal 均不能访问。

设备:

```
```text id="sx89sk"
```

annasmacbookpro.tutti

```
```
```

Session:

```
```text id="cdowwf"
```

claude-a.annasmacbookpro.tutti

codex-b.annasmacbookpro.tutti

```
```
```

如果 Anna 只有一个 Session 占用 `:3000`:

```
```text id="3mrbqx"
```

annasmacbookpro.tutti:3000

```
```
```

直接解析。

如果多个 Session 占用同一个 HTTP/HTTPS 端口, 则设备短地址返回 H5 Session 选择器。



Agent 同样能读取这个 H5 并选择 canonical link。

Raw TCP 出现歧义时必须直接使用：

```
```text id="8v6f9i"
session.device.tutti:port
```
```

任何监听 TCP 端口默认都可以发布给 Room。

Open Tutti 自己生成 Local CA，只在 Tutti Browser 和 Session Container 信任，不写入 macOS/Windows 系统证书库。

---

## ## 14. Group Chat / Big @ / History

第一版保留 Tutti VM 的核心：

```
```text id="usyj9u"
Group Chat
Big @
Agent Conversation
Tasks
Artifacts
Context Graph
```
```

会议历史最终归档到：

```
```text id="sixyxp"
.opentuttivm/historys/
```
```

例如：

```
```text id="8zondx"
.opentuttivm/historys/<collaboration-session-id>/
|—— manifest.json
|—— group-chat.jsonl
|—— tasks.jsonl
|—— agents/
|   |—— <agent-session-id>/
|   |   |—— session.json
|   |   |—— messages.jsonl
```
```

保存的是 provider-neutral canonical semantic history。

不保存 Credentials、Token、Host 私有路径等。

流式 Tool Output 做 coalesce，最终 semantic output 可以完整保存；大型二进制用 Workspace/Artifact 引用，不把 base64 巨物塞进 JSONL。

下一次创建 Room 时:

```
```text id="piyc7h"
Restore History
或
Start Fresh
```
```

一次只能 Restore 一个历史协作 Session。

Restore 只恢复 Context:

```
```text id="751clz"
Chat
Agent Conversations
Tasks
Big @
Artifacts metadata
```
```

****不会把 Workspace 回滚到历史版本。****

首次保存 History 时询问 Owner:

```
```text id="265t54"
是否让 Chat History 进入 Git?
```
```

如果否, 追加:

```
```gitignore id="mzm9vz"
# Chat History Ignored by OpenTuttiVM
.opentuttivm/historys/
```
```

---

## 15. App Catalog 与 Server 网络

默认继续使用 ****Tutti 官方 App Catalog****。

但 App Catalog 是 Desktop 自己访问, 不经过 Self Hosted Server 代理。

Tutti Agent 则明确隐藏。

Open Tutti Desktop 支持多个 Self Hosted Server。

服务器允许:

```
```text id="ovm52r"
系统可信 HTTPS
用户显式信任的自签名/企业 CA
```

```

不允许：

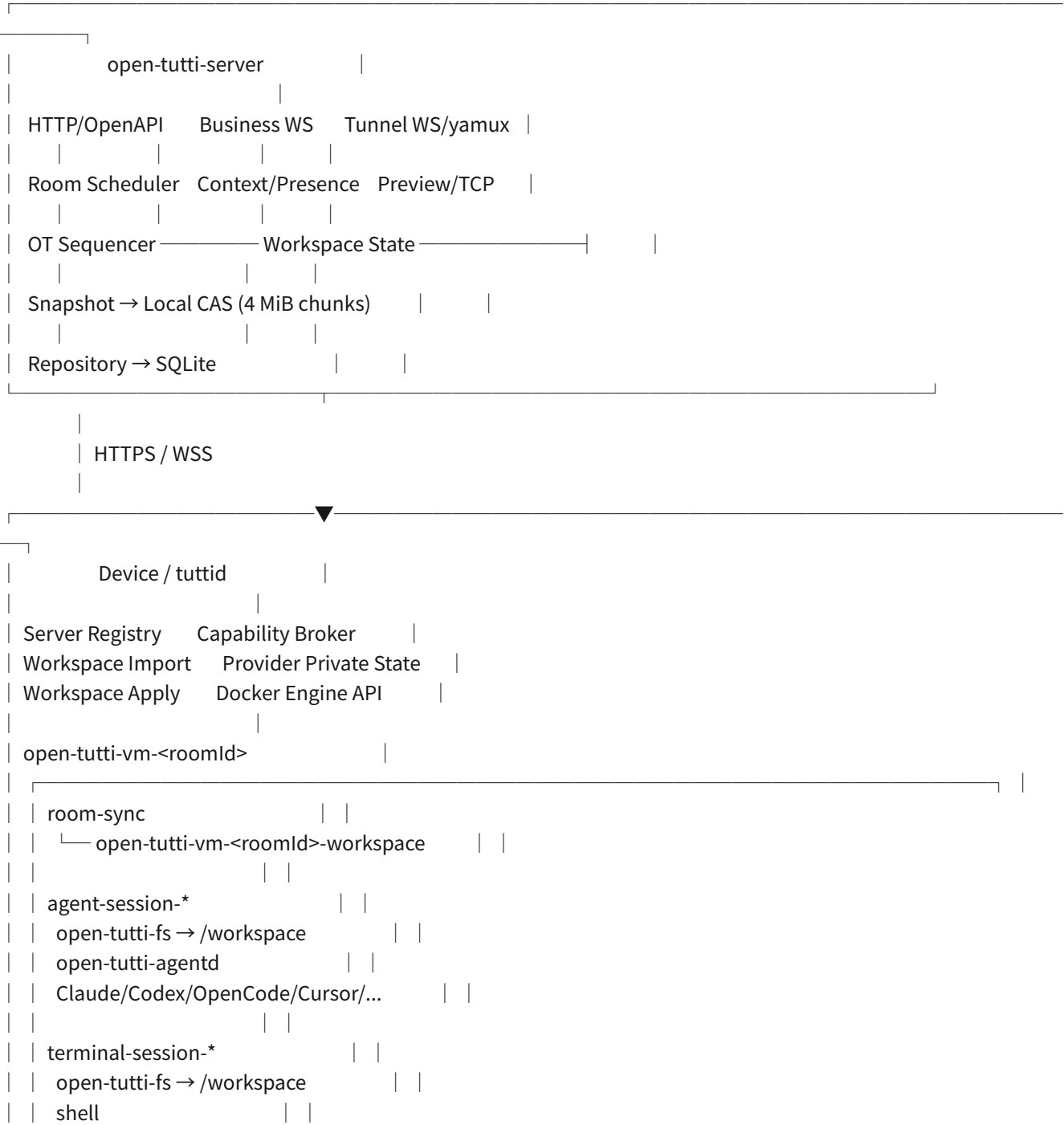
```text id="qok7ld"  
全局忽略 TLS 验证
```

自签名 Server 首次连接显式信任并做 pinning。

---

## 最终运行拓扑

```text id="me9f5q"



我认为这份设计已经足够进入实现阶段了~(≥▽≤)♡ 核心思想也已经非常稳定：

> \*\*Server 做权威协调，不做 Cloud VM；Docker Desktop 做每个设备的 Managed Local VM；FUSE 把多台本地 VM 组织成一个实时 Workspace；Capability Broker 让 Agent 能力可借、Secret 不外泄；Room 像会议，最终成果回到 Owner 的真实 Workspace。 \*\*

你看看这份脑暴画布有没有需要改的地方。如果你确认这就是我们要做的方案，下一步我就不再继续脑暴，直接按这份方案开始拆 \*\*实际实现阶段、仓库改造顺序和第一批代码任务\*\* 啦~、(✱°▽°)ノ

#225 用户

整个对话记录一字不差整理为 PDF

#226 助手

可以。我会把这段对话里\*\*所有可见的用户消息与助手回复按时间顺序原样整理\*\*成 PDF，不加入总结、不改写内容；工具调用、系统/开发者隐藏指令和内部推理不会作为对话正文写进去。

#227 助手

这份 PDF 会比较长，我会用“用户 / 助手”分段但不改正文内容，并保留代码块、链接、加粗、项目符号和颜文字。完成后我还会渲染检查，避免中文字体缺失、分页截断或代码块溢出。

#228 助手

我发现运行环境里没有保留刚才准备中的临时文件，所以我正在直接从当前会话重新生成成品；正文仍按可见对话逐条排版，不会用摘要替代。

#229 助手

正文重建已经进入排版阶段。为了避免“整理”过程中改掉原话，我会把消息正文按纯文本保留，仅额外增加“用户 / 助手”分隔和页码；链接与代码块内容本身不改。

#230 用户

继续

#231 助手

继续。我现在直接完成 PDF 生成和渲染校验；不会再用摘要替代正文。

#232 助手

我已经把前半段原始消息逐条写入源文件，包括中途的架构修正和代码块；现在继续补后半段，再统一生成 PDF。

#233 助手

我在核对时发现中段还有一组“Host Workspace 导入 / Apply / Git 清洗 / Server 崩溃”的原始消息没有进入源文件，正在补回正确的中段位置，避免 PDF 漏段。